

Enhancing IoT security: A comparative study of feature reduction techniques for intrusion detection system

Jing Li^{a,*}, Hewan Chen^b, Mohd Othman Shahizan^a, Lizawati Mi Yusuf^a

^a University of Technology Malaysia, China

^b China Jiliang University, China

ARTICLE INFO

Keywords:

Internet of things
Intrusion detection
Feature reduction
Machine learning
Attack classification

ABSTRACT

Internet of Things (IoT) devices are extensively utilized but are susceptible to cyberattacks, posing significant security challenges. To mitigate these threats, machine learning techniques have been implemented for network intrusion detection in IoT environments. These techniques commonly employ various feature reduction methods, prior to inputting data into models, in order to enhance the efficiency of detection processes to meet real-time requirements. This study provides a comprehensive comparison of feature selection (FS) and feature extraction (FE) techniques for network intrusion detection systems (NIDS) in IoT environments, utilizing the TON-IoT and BoT-IoT datasets for both binary and multi-class classification tasks. We evaluated FS methods, including Pearson correlation and Chi-square, and FE methods, such as Principal Component Analysis (PCA) and Autoencoders (AE), across five classic machine learning models: Decision Tree (DT), Random Forest (RF), Naive Bayes (NB), k-Nearest Neighbors (kNN), and Multi-Layer Perceptron (MLP). Our analysis revealed that FE techniques generally achieve higher accuracy and robustness compared to FS methods, with RF paired with AE delivering superior performance despite higher computational demands. DTs are most effective with smaller feature sets, while MLPs excel with larger sets. Chi-square is identified as the most efficient FS method, balancing performance and computational efficiency, whereas PCA outperforms AE in runtime efficiency. The study also highlights that FE methods are more effective for complex datasets and less sensitive to feature set size, whereas FS methods show significant performance improvements with more informative features. Despite the higher computational costs of FE methods, they demonstrate a greater capability to detect diverse attack types, making them particularly suitable for complex IoT environments. These findings are crucial for both academic research and industry applications, providing insights into optimizing detection performance and computational efficiency in NIDS for IoT networks.

1. Introduction

The Internet of Things (IoT) plays a significant role in enhancing various aspects of daily life (Al-Fuqaha et al., 2015). As IoT devices continue to proliferate, their impact on our lives becomes increasingly profound. According to Statista, the number of IoT devices is projected to reach 30.9 billion by 2025, up from 8.74 billion in 2020 (Zhou et al., 2019). Despite this rapid growth and the numerous benefits these devices offer, security remains a major concern. IoT devices often have limited security capabilities, making them vulnerable to cyberattacks and raising significant issues regarding data protection and privacy.

Due to the limited computing resources and low cost of IoT devices, implementing robust security schemes is essential. One such scheme is

Network Intrusion Detection Systems (NIDS), which help address these security concerns (Chaabouni et al., 2019). Data-driven NIDS powered by machine learning have emerged as a promising solution (Mishra et al., 2019). However, given the resource constraints of IoT devices, it is crucial to use limited yet informative features. Feature reduction techniques, such as feature selection and extraction, are vital in optimizing the efficiency of these security systems (Tama et al., 2019). By focusing on essential features, feature reduction helps enhance the performance of NIDS, making them more suitable for resource-limited IoT environments and effectively addressing security issues.

Two primary feature reduction strategies used in optimizing security systems for resource-limited IoT devices are feature selection and feature extraction. Feature selection involves identifying and selecting

* Corresponding author.

E-mail address: jinglihz225@163.com (J. Li).

<https://doi.org/10.1016/j.iswa.2024.200407>

Received 14 November 2023; Received in revised form 31 May 2024; Accepted 11 June 2024

Available online 14 June 2024

2667-3053/© 2024 The Author(s). Published by Elsevier Ltd. This is an open access article under the CC BY-NC-ND license (<http://creativecommons.org/licenses/by-nc-nd/4.0/>).

the most relevant features from the original dataset without altering the original data (Hall). Techniques such as filter methods, wrapper methods, and embedded methods are commonly used in feature selection. Filter methods rank features based on statistical criteria, wrapper methods evaluate feature subsets using a specific learning algorithm, and embedded methods perform feature selection during the model training process. On the other hand, feature extraction transforms the original data into a new set of features, capturing the most important information in a more compact form (Yan & Han, 2018). Principal Component Analysis (PCA) and Linear Discriminant Analysis (LDA) are widely used feature extraction techniques. PCA reduces the dimensionality of the data by projecting it onto a set of orthogonal components that maximize variance, while LDA aims to find a feature space that best separates different classes. Both methods help in enhancing the performance of machine learning models by reducing computational complexity and improving the efficiency of intrusion detection in IoT systems.

Recent studies have employed feature reduction techniques to enhance security schemes, including Network Intrusion Detection Systems (NIDS) (Aminanto et al., 2018; Amiri et al., 2011). However, a notable gap in this research is the tendency to overlook the unique characteristics of IoT environments, focusing instead on general cyber networks. Furthermore, there has been limited comprehensive evaluation and analysis of feature reduction methods specifically on IoT datasets (Zachos et al., 2021). This oversight is critical, as the effectiveness of feature reduction is a constant and pressing issue for IoT devices with limited computing capabilities. Consequently, it is significant to conduct thorough evaluations of feature reduction techniques tailored to IoT contexts, ensuring that these methods effectively address the unique challenges posed by resource-constrained IoT devices.

To this end, our research aims to fill this gap by utilizing advanced feature reduction techniques to evaluate and benchmark an evaluation framework specifically designed for IoT datasets. We will systematically assess the strengths and limitations of various feature reduction methods within the context of IoT environments (Saied et al., 2024). By doing so, we intend to provide a comprehensive guide on the topic of feature reduction in machine learning-based models tailored for IoT applications. This guide will be invaluable for both academic researchers and industry practitioners, offering insights into the most effective feature reduction strategies to enhance the performance and security of IoT devices. Our work will contribute to the development of more robust and efficient security schemes, addressing the unique challenges posed by resource-constrained IoT systems.

This study reveals the applicability of feature reduction techniques in IoT scenarios, aiming to strike a balance between runtime efficiency and performance scores. By applying these techniques to real-world IoT datasets, we demonstrate how feature reduction can optimize the computational requirements of IoT devices without compromising the accuracy and effectiveness of security measures. Our findings highlight the importance of selecting the right feature reduction strategy to ensure that IoT systems can operate efficiently within their limited computational resources while maintaining robust performance in detecting and mitigating security threats. This balance is crucial for the practical deployment of machine learning-based security models in IoT environments, where resource constraints are a significant consideration.

Based on the findings of this study, a comprehensive table based on feature reduction methods has been provided. This Table 29 serves as a practical guide for selecting appropriate reduction techniques for IoT classification models aimed at enhancing IoT security. The guide highlights the strengths and limitations of each method, considering factors such as classification accuracy, robustness, computational efficiency, and suitability for different feature set sizes. By offering detailed insights into the performance of feature selection and feature extraction methods under various conditions, this table equips researchers and practitioners with the knowledge to make informed decisions when developing security frameworks for IoT environments. This guide is particularly

valuable for future work in IoT security, helping to optimize detection performance while managing the computational constraints inherent in IoT devices.

The key contributions in this paper are provided as follows.

- 1) Comprehensive Evaluation of Feature Reduction Techniques: This study thoroughly implements and measures various feature reduction techniques, including feature selection methods such as Pearson correlation and Chi-square, and feature extraction methods like Principal Component Analysis (PCA) and Autoencoders (AE). These techniques are ranked and selected for their effectiveness in improving the resilience and accuracy of attack classification models, with a focus on public IoT datasets.
- 2) Development of a 3-Phase Machine Learning Pipeline: A robust 3-phase machine learning pipeline framework is created, encompassing data preprocessing, feature reduction, class balancing, and classification procedures. This framework is used for a comprehensive performance evaluation of classic machine learning classifiers, including Decision Tree (DT), Random Forest (RF), Naive Bayes (NB), k-Nearest Neighbors (kNN), and Multi-Layer Perceptron (MLP).
- 3) Application and Evaluation on IoT Datasets: The study applies and evaluates the proposed feature reduction techniques using two representative IoT datasets, Network TON-IoT (Moustafa, 2021) and BoT-IoT (Koroniotis et al., 2019), for both binary and multi-class classification tasks. The analysis demonstrates that feature extraction techniques generally offer higher accuracy and robustness, particularly in complex IoT environments, while feature selection techniques show significant improvements with more informative feature sets.

This paper is structured into six sections. Section 2 reviews related works on feature reduction techniques in IoT security, identifying gaps this study addresses. Section 3 details the proposed three-phase evaluation framework, including the methodology and criteria for assessing each reduction method. Section 4 covers the configurations and pre-conditions for evaluation. Section 5 presents the results and analysis, comparing the performance of different feature reduction methods, with a summary of findings. Finally, Section 6 concludes, discusses limitations, and suggests future research directions.

2. Related works

This section reviews related studies on Network Intrusion Detection Systems (NIDSs) utilizing feature reduction methods. Feature selection is commonly employed in NIDS to simplify traffic data. Several studies use filter-based methods to identify key features for classification. For example, a study (Amiri et al., 2011) compared Mutual Information (MI)-based feature selection with linear and non-linear techniques, finding MI superior to correlation-based methods in attack detection accuracy. Another study (Ambusaidi et al., 2016) combined MI with a variant support vector machine, enhancing accuracy and reducing model complexity on datasets like KDD Cup 99, NSL-KDD, and Kyoto 2006+ (Song et al., 2011). Study (Kasongo & Sun, 2020) analyzed the UNSW-NB15 dataset (Moustafa & Slay, 2015) using a filter-based technique with XGBoost, while study (Disha & Waheed, 2022) employed a Gini Impurity-based ranking method by Random Forest on the TON-IoT dataset to evaluate classification performance, though it did not extensively consider computational costs. However, many datasets used for NIDS benchmarking are outdated, limiting their relevance for IoT security.

Many studies employ wrapper-based feature selection to enhance classification performance by identifying optimal feature subsets. A method called CorAUC (Shafiq et al., 2021) uses the area under the curve (AUC) metric to select effective features for machine learning (ML) algorithms, tested on the Bot-IoT dataset (Koroniotis et al., 2019).

Despite its effectiveness, it showed lower precision for certain attacks like keylogging. Heuristic optimization algorithms, such as genetic algorithms (GA) (Aslahi-Shahri et al., 2016; Khammassi & Krichen, 2017; Samadi Bonab et al., 2020), have been used to identify optimal feature subsets, demonstrating lower false alarm rates on datasets like UNSW-NB15 and KDD99. For instance, study (Halim et al., 2021) used the Pigeon Inspired Optimizer (PIO) for feature selection, showing high detection rates and accuracy with minimized false alarms on KDDCUP99, NSL-KDD, and UNSW-NB15 datasets. Lightweight models tailored for IoT networks have also been designed. One study (Liu et al., 2021) proposed Particle Swarm Optimization (PSO) with one-class Support Vector Machine (SVM) (Chohra et al., 2022), and another (Naseri & Gharehchopogh, 2022) introduced a Binary Farmland Fertility Algorithm (BFFA) for feature selection, enhancing detection accuracy and computational efficiency. However, these techniques often incur high computational costs, which can negatively impact resource-constrained IoT systems and networks. Further research is needed to validate these methods across diverse datasets and explore hybrid approaches for improved intrusion detection.

In addition, since traditional Intrusion Detection Systems (IDS) often struggle with the high dimensionality and complexity of network traffic data, prompting the development of multi-objective-based IDS to address these challenges. For instance, a study (Gharehchopogh et al., 2023) introduced the Multi-objective Mutation-based Dynamic Harris Hawks Optimization (MODHHO) algorithm, which enhances botnet detection in IoT environments by optimizing feature selection, improving detection accuracy, and computational efficiency. The MODHHO algorithm outperforms existing methods with high true positive rates and low false alarm rates across various datasets. Another approach (Hosseini et al., 2023) combines Artificial Ecosystem-based Optimization (AEO) and the Sine Cosine Algorithm (SCA) for feature selection in IoT botnet detection, incorporating mechanisms like Bitwise operations and Opposition-based learning to improve detection accuracy and efficiency. However, its performance may vary with different IoT devices and attack patterns, necessitating further validation. Additionally, (Hosseini et al., 2022) proposed a botnet detection system using the Slime Mold Algorithm (SMA) and Salp Swarm Algorithm (SSA), integrating chaos theory and the Disruption operator mechanism to balance exploration and exploitation. Tested on standard UCI datasets, this method shows high botnet detection accuracy with a low error rate, although further validation in diverse real-world scenarios is required.

Moreover, many studies have employed hybrid feature selection methods to enhance attack classifiers' performance while reducing overfitting during model training. For instance, study (Moustafa & Slay, 2015) combined association rule mining and central attribute values, outperforming NSLKDD on the UNSW-NB15 dataset. Additionally, ensemble feature selection techniques have been used to identify significant features; study (Moustafa et al., 2019) combined Decision Trees (DT), Artificial Neural Networks (ANN), and Naive Bayes (NB) to learn optimal features from statistical flow data, while study (Leevy et al., 2022) utilized information gain, information gain ratio, and Chi-squared (Chi2) ranking techniques. However, these methods often overlook computational costs in favor of performance metrics. To address this issue, researchers have investigated correlation-based feature selection for its computational efficiency in NIDS by considering feature correlations (Hall). This method was initially applied to the KDD99 and UNSW-NB15 datasets in study (Moustafa & Slay, 2016). Recently, an improved multivariate correlation-based method for network anomaly detection was proposed (Moustafa et al., 2019), and study (Gavel et al., 2022) used a correlation-based fitness function with ant lion optimization for the AWID dataset in wireless networks. Study (Zhou et al., 2022) further refined this approach by removing redundant features and selecting the most informative ones based on a correlation threshold. These methods have significantly improved NIDS accuracy but added complexity.

Unlike feature selection, which retains a subset of initial features in

Network Intrusion Detection Systems (NIDS), feature extraction condenses the original features into a lower-dimensional vector while preserving much of the data. Methods such as principal component analysis (PCA), linear discriminant analysis (LDA), and autoencoders (AE) based on neural networks are commonly used for this purpose. For instance, study (Xu & Wang, 2005) applied PCA to the KDD99 dataset, significantly reducing dimensionality and enhancing NIDS performance and accuracy in attack classification via support vector machines. Various PCA variants, including hierarchical PCA neural networks on the 1998 DARPA dataset (Liu et al., 2007) and kernel PCA with genetic algorithms (Kuang et al., 2014), have been used to improve precision for less common attacks. PCA has also been applied to recent datasets like UNSW-NB15 and CICIDS2017 (Abdulhammed et al., 2019; Qi et al., 2022). LDA has been used to reduce computational complexity in NIDS, as demonstrated in (Tan et al., 2010). Studies (Pajouh et al., 2017) and (Pajouh et al., 2019) combined PCA and LDA in a two-layer dimension reduction approach, effectively reducing dimensionality and detecting low-frequency malicious activities over the NSLKDD dataset.

To improve the efficiency of feature extraction in Network Intrusion Detection Systems (NIDS), various research works have applied autoencoder (AE)-based neural networks. For instance, study (Yan & Han, 2018) introduced a stacked sparse AE approach to create non-linear mappings from high-dimensional to low-dimensional data using the NSLKDD dataset. Study (Khan et al., 2019) employed a deep stacked AE to reduce the number of features for both binary and multiclass classification, achieving higher accuracy than previous methods. Additionally, several AE-based networks incorporating long short-term memory (LSTM), including variational LSTM (Zhou et al., 2021) and bidirectional LSTM (Popoola et al., 2021), have been developed to effectively address imbalances and high-dimensional problems in NIDS. However, AE-based methods, derived from deep neural networks, involve higher computational costs in both training and testing compared to statistical-based PCA and LDA algorithms. To mitigate the computational cost issue, study (Dao & Lee, 2022) proposed a network pruning algorithm to build a lightweight detection model, significantly reducing the complexity of AE structures for feature extraction in NIDS, using UNSW-NB15 and CICIDS datasets. Moreover, study (D'Angelo & Palmieri, 2021) integrated an AE network with convolutional and recurrent neural networks to extract spatial and temporal features without human intervention.

Despite numerous studies employing various feature reduction techniques, few have comprehensively compared the performance and efficiency of feature selection and feature extraction methods, particularly for IoT data. For instance, study (Aminanto et al., 2018) combined AE-based feature extraction with supervised machine learning feature selection without performance comparison, while study (Ngo et al., 2023) only compared these methods using traditional networking data from the UNSW-NB15 dataset. Given the need for real-time, low-latency detection, we emphasize correlation-based and chi-square feature selection methods for their efficiency and effectiveness, which will be comprehensively evaluated using IoT-specific datasets. For feature extraction, Principal Component Analysis (PCA) and Autoencoders (AE) are selected due to their demonstrated effectiveness and complementary strengths. Evaluating both PCA and AE will leverage PCA's computational efficiency and AEs' advanced feature extraction abilities, providing a comprehensive understanding of optimizing IoT classification models for robust performance within IoT constraints.

In related works, various machine learning classifiers have been employed due to their specific strengths. Decision Trees (DT) are frequently chosen for their simplicity and interpretability, making them ideal for feature reduction studies (Padmashree & Krishnamoorthi, 2022). Random Forests (RF) are valued for their robustness and ability to handle large, high-dimensional datasets, which is beneficial for cybersecurity applications (Pavaiyarkarasi et al., 2022). k-Nearest Neighbors (kNN) is effective for classification tasks with non-linear data distributions (Aburomman & Ibne Reaz, 2016). Naive Bayes (NB) is

avored for its computational efficiency and simplicity, suitable for real-time applications in resource-constrained environments (Ismail & Reza, 2022). Multi-Layer Perceptron (MLP) is used for its ability to model complex patterns through deep learning, essential for detecting sophisticated cyber threats (Dina & Manivannan, 2021). Despite the focus on improving detection accuracy and reducing computational complexity using these classifiers and feature reduction techniques, there is a lack of comprehensive comparison studies specifically on

feature selection (FS) and feature extraction (FE) using current IoT network datasets. Our research aims to address this gap by thoroughly evaluating these methods within the context of IoT.

In this study, feature selection methods, including Pearson correlation and chi-square, as well as feature extraction methods like Principal Component Analysis (PCA) and Autoencoders (AE), are evaluated. A three-phase machine learning-based framework is developed for the standard evaluation of these feature reduction methods. Two renowned

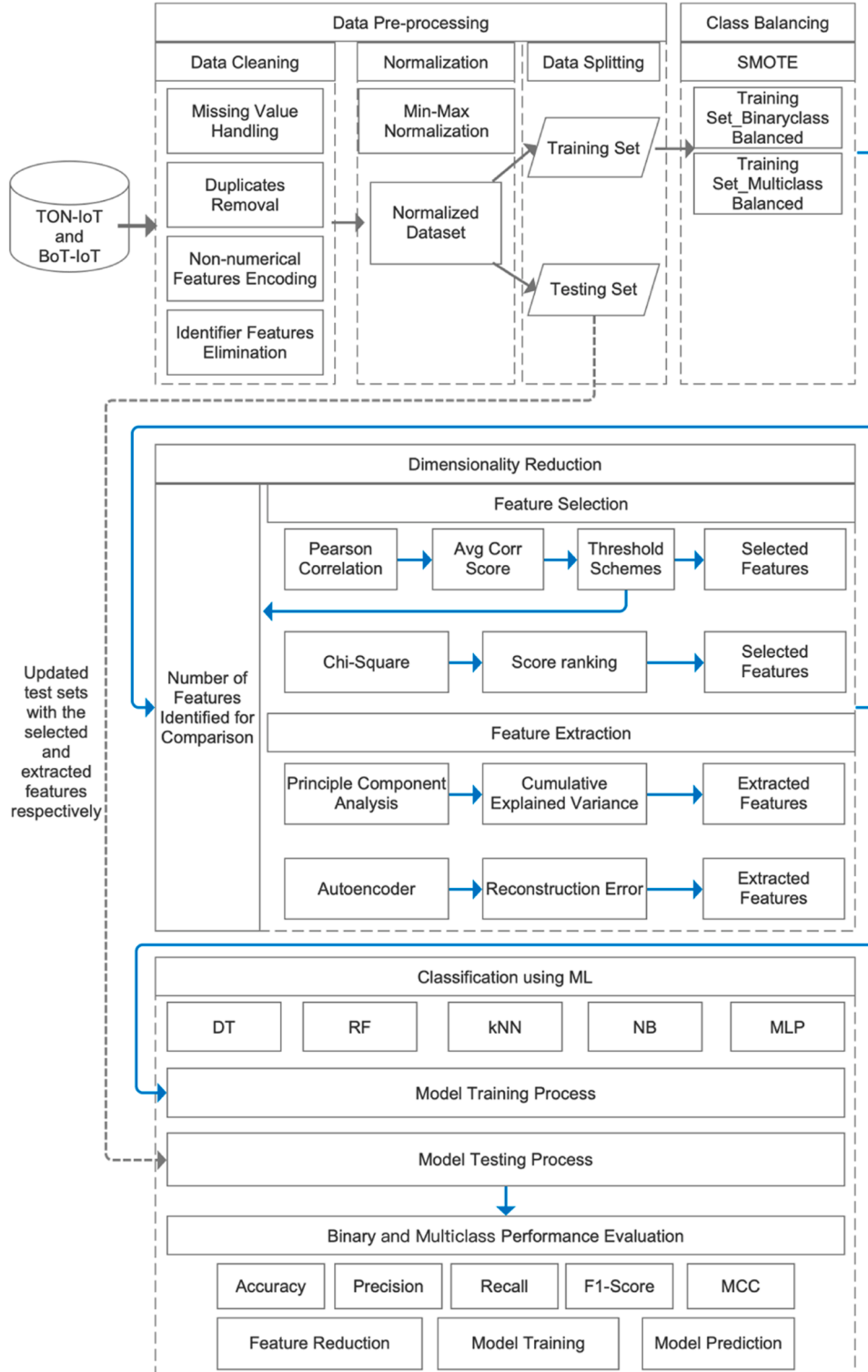


Fig. 1. Framework of proposed NIDS for comparison of feature reduction methods.

IoT datasets, TON-IoT and BoT-IoT, are used to simulate real IoT characteristics. Five machine learning models—Decision Trees (DT), Random Forests (RF), k-Nearest Neighbors (kNN), Naive Bayes (NB), and Multi-Layer Perceptron (MLP)—are trained and tested. Performance metrics such as accuracy, precision, recall, F1-score, Matthews correlation coefficient (MCC), and runtime for feature reduction, model building, and testing are assessed. Both binary and multi-class classification scenarios, including attack-wise scores, are involved to ensure comprehensive evaluation.

3. Methodology

In this section, we put feature reduction technique as module factor into the pipeline of machine learning-based network intrusion detection system (NIDS) respectively, according to the final performance metrics of the classification models. Here is the framework of the methodology according to Fig. 1, which illustrates the process of data preprocessing, class balancing, feature reduction, and classification in a machine learning pipeline, with a particular focus on the evaluation and comparison of feature reduction techniques. A detailed explanation of the three main workflow of the proposed model is provided as following, particularly for two feature reduction methods.

a) Phase 1: Data Preprocessing and Class Balancing

In this phase, the raw data from TON-IoT and BoT-IoT datasets undergo several preprocessing steps to prepare it for analysis. Data Cleaning involves handling missing values, removing duplicates, encoding non-numerical features, and eliminating identifying features. Next, the data is split into Training Set and Testing Set. The Normalization step scales the features to ensure uniformity, and SMOTE (Synthetic Minority Over-sampling Technique) is applied to balance the classes within the training set, addressing issues related to class imbalance by creating synthetic samples for minority classes. A detailed description is presented in Section 3.2.

b) Phase 2: Dimensionality Reduction Evaluation

This phase focuses on reducing the dimensionality of the data to enhance model performance and reduce computational complexity. It includes two main techniques: Feature Selection and Feature Extraction.

Feature Selection methods involve Pearson Correlation to measure the linear relationship between features, Average Correlation Score to summarize the importance of each feature, and various Threshold Schemes to select the most relevant features. Additionally, Chi-Square tests are used for ranking features based on their statistical significance.

Feature Extraction employs Principal Component Analysis (PCA) to transform data into a lower-dimensional space by maximizing explained variance and Autoencoders to capture non-linear relationships by minimizing reconstruction error. Both methods generate a compact representation of the data, making it more manageable for subsequent analysis.

This phase is crucial for evaluating and comparing different feature reduction techniques to identify the most effective method for the specific dataset and problem context. Section 3.3 offers an in-depth description of feature reduction methods.

c) Phase 3: Classification using

In the final phase, the reduced feature set is used to train and test various machine learning classifiers, including Decision Trees (DT), Random Forests (RF), k-Nearest Neighbors (kNN), Naive Bayes (NB), and Multi-Layer Perceptron (MLP). The Model Training Process involves fitting these classifiers to the training data, while the Model Testing Process evaluates their performance on the test data. The evaluation metrics include accuracy, precision, recall, F1-score, feature reduction time, Model training, and model prediction time. This phase allows for a detailed comparison of the classifiers' performance, highlighting how effectively each feature reduction

technique contributes to the accuracy and robustness of the attack classification models. Section 3.4 offers a detailed description of attack classification methods.

3.1. Datasets

To build and evaluate IoT-oriented classification models, two public published IoT datasets with IoT nature, named TON-IoT (Moustafa, 2021) and BoT-IoT (Koroniotis et al., 2019) are used in this study, following subsection presents the background and the characteristics of the two IoT datasets.

1) TON-IoT

The TON_IoT dataset is an innovative and comprehensive dataset designed to evaluate AI-based security systems, particularly in the context of IoT networks. This dataset was developed as part of a novel distributed architecture that integrates edge, fog, and cloud computing layers using technologies such as Software-Defined Networking (SDN), Network Function Virtualization (NFV), and Service Orchestration (SO). The architecture allows for the dynamic creation and management of testbed networks that can simulate real-world scenarios. The TON_IoT dataset includes heterogeneous data sources, such as telemetry data from IoT services, Windows and Linux operating systems, and network traffic data. The primary motivation for choosing this dataset lies in its ability to reflect actual behaviors of IoT networks under normal and attack conditions, providing a robust ground truth for the validation of cybersecurity solutions. Compared to existing datasets, TON_IoT offers a more realistic and diverse set of data, essential for developing and testing AI models aimed at securing complex IoT environments.

The testbed for generating the TON_IoT datasets simulates real-world IoT and edge networks within smart cities, utilizing three layers: edge, fog, and cloud. The edge layer manages IoT/IoT devices and gateways, the fog layer provides computing and analytics through virtualization technologies, and the cloud layer integrates cloud services for telemetry data management and security event simulations. Fig. 2 presents the testbed in detailed for generating and extracting TON-IoT dataset.

Below is the key information about the TON-IoT Network dataset, which will be employed in our experiments detailed in Section 4. Subsequently, a comprehensive discussion on data preprocessing for this dataset will be provided.

TON-IoT dataset was generated from heterogeneous data sources collected from Telemetry datasets of IoT and IIoT sensors, operating systems datasets of Windows as well as Ubuntu network traffic datasets. It was first introduced in (Moustafa, 2021), and this dataset comprises 22,339,021 instances of data and includes two target classes: the "label" class, containing normal and attack data, and another class with ten categories—normal and nine attack types, such as Backdoor, DDoS, DoS, Injection, Password, Ransomware, Scanning, XSS, and MITM. There are six feature groups: Connection, Statistical, DNS, SSL, HTTP, Violation, and Labeling, holding a total of 46 features in the original data. However, in this research, our data analysis involves the "Train_Test_Network.csv" dataset, comprising both training and testing sets, totaling 461,043 records. Table 1 displays the distribution of labels for the binary class and types for the multiple-class, while Table 2 shows the dataset's respective features.

2) BoT-IoT

The Bot-IoT dataset was developed to address the critical need for realistic and comprehensive data for network forensic analytics, particularly in the context of IoT networks. This dataset was created to simulate a wide range of IoT and network activities, including both normal traffic and various types of cyber-attacks such as Distributed Denial of Service (DDoS), data theft, and keylogging. The testbed for the

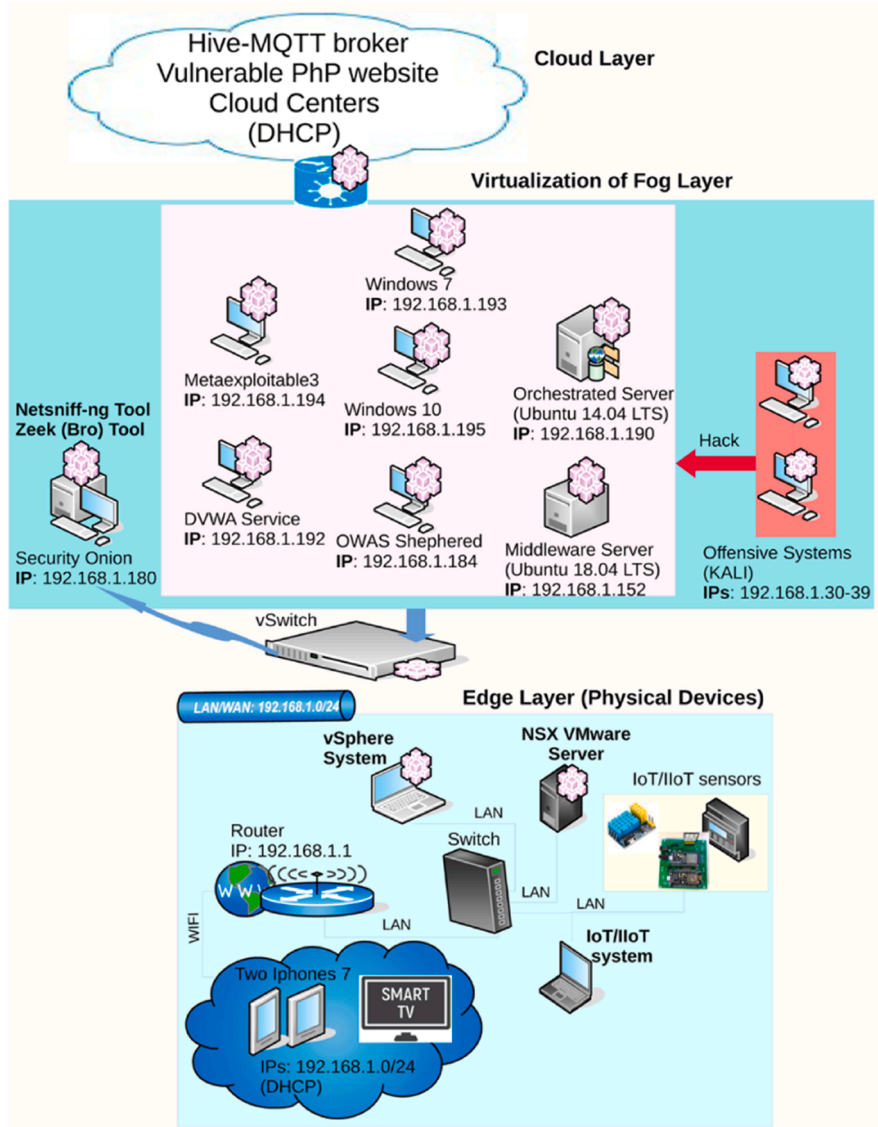


Fig. 2. Simulated testbed of TON-IoT dataset.

Table 1

Classes description of network TON_IoT.

No.	Type of classes	Amount
1	Normal	300,000
2	backdoor	20000
3	ddos	20000
4	dos	20000
5	injection	20000
6	password	20000
7	ransomware	20000
8	scanning	20000
9	xss	20000
10	mitm	1043

Bot-IoT dataset was meticulously designed to incorporate a mixture of legitimate and malicious activities using a variety of tools and IoT devices, to ensure that the dataset accurately reflects the complexity and diversity of real-world IoT environments. The primary motivation for choosing the Bot-IoT dataset is its detailed and well-documented testbed configuration, the inclusion of recent and complex attack scenarios, and the comprehensive labeling of data. These features make the Bot-IoT

dataset an invaluable resource for developing and evaluating advanced network forensic methods, as it provides a realistic basis for training and validating machine learning algorithms aimed at detecting and mitigating IoT-specific cyber threats.

The proposed testbed consists of three main components: network platforms, simulated IoT services, and feature extraction with forensic analytics. The network platforms include virtual machines (VMs) configured with both normal and attacking roles, connected through firewalls and network devices to simulate realistic network traffic. Simulated IoT services are implemented using the Node-red tool to mimic various IoT scenarios, such as a weather station and smart home devices, while data features are extracted and analyzed using the Argus tool to discriminate between normal and abnormal traffic for forensic purposes. Fig. 3 presents the testbed for generating and extracting Bot-IoT dataset.

Below is the key information about the Bot-IoT dataset, which is 5 % of the original dataset, and will be employed in our experiments detailed in Section 4. Subsequently, a comprehensive discussion on data pre-processing for this dataset will be provided.

The Bot-IoT dataset was first put forward in (Koroniotis et al., 2019). It consists of 73,360,900 data instances and is divided into two target classes: one class called "label" that contains both normal and attack

Table 2
Features description in Network TON_IoT.

No.	Feature	Description	No.	Feature	Description
1	ts	Timestamp of connection between flow identifiers	24	dns_rejected	DNS rejection, where the DNS queries are rejected by the server
2	src_ip	Source IP addresses which originate endpoints' IP addresses	25	ssl_version	SSL version which is offered by the server
3	Src_port	Source ports which Originate endpoint's TCP/ UDP ports	26	ssl_cipher	SSL cipher suite which the server chose
4	Dst_ip	Destination IP addresses which respond to endpoint's IP addresses	27	ssl_resumed	SSL flag indicates the session that can be used to initiate new connections, where T refers to the SSL connection is initiated
5	Dst_port	Destination ports which respond to endpoint's TCP/UDP ports	28	ssl_established	SSL flag indicates establishing connections between two parties, where T refers to establishing the connection
6	proto	Transport layer protocols of flow connections	29	ssl_subject	Subject of the X.509 cert offered by the server
7	service	Dynamically detected protocols, such as DNS, HTTP and SSL	30	ssl_issuer	Trusted owner/originator of SLL and digital certificate (certificate authority)
8	duration	The time of the packet connections, which is estimated by subtracting 'time of the last packet seen' and 'time of the first packet seen'	31	http_trans_depth	Pipelined depth into the HTTP connection
9	src_bytes	Source bytes which are originated from payload bytes of TCP sequence number	32	http_method	HTTP request methods such as GET, POST and HEAD
10	dst_bytes	Destination bytes which are responded payload bytes from TCP sequence numbers	33	http_uri	URIs used in the HTTP request
11	conn_state	Various connection states, such as S0 (connection without replay), S1 (connection established), and REJ (connection attempt rejected)	34	http_referrer	Values of the 'referer' header in the HTTP protocol
12	missed_bytes	Number of missing bytes in content gaps	35	http_version	The HTTP versions utilized such as V1.1
13	src_pkts	Number of original packets which is estimated from source systems	36	http_request_body_len	Actual uncompressed content sizes of the data transferred from the HTTP client
14	src_ip_bytes	Number of original IP bytes which is the total length of IP header field of source systems	37	http_response_body_len	Actual uncompressed content sizes of the data transferred from the HTTP server
15	dst_pkts	Number of destination packets which is estimated from destination systems	38	http_status_code	Status codes returned by the HTTP server
16	dst_ip_bytes	Number of destination IP bytes which is the total length of IP header field of destination systems	39	http_user_agent	Values of the UserAgent header in the HTTP protocol
17	dns_query	Domain name subjects of the DNS queries	40	http_orig_mime_types	Ordered vectors of mime types from source system in the HTTP protocol
18	dns_qclass	Values which specifie the DNS query classes	41	http_resp_mime_types	Ordered vectors of mime types from destination system in the HTTP protocol
19	dns_qtype	Value which specifies the DNS query types	42	weird_name	Names of anomalies/violations related to protocols that happened
20	dns_rcode	Response code values in the DNS responses	43	weird_addl	Additional information is associated to protocol anomalies/violations
21	dns_AA	Authoritative answers of DNS, where T denotes server is authoritative for query	44	weird_notice	It indicates if the violation/anomaly was turned into a notice
22	dns_RD	Recursion desired of DNS, where T denotes request recursive lookup of query	45	label	Tag normal and attack records, where 0 indicates normal and 1 indicates attacks
23	dns_RA	Recursion available of DNS, where T denotes server supports recursive queries	46	type	Tag attack categories, such as normal, DoS, DDoS and backdoor attacks, and normal records

data, and another class with five main categories that includes four different attack types in addition to normal class, like DDoS, DoS, Reconnaissance, and Theft. With a total of 43 features in the original data, there are two main feature groups: default features and produced flow features. In this study, however, we are analyzing data from the 5 % original dataset, which consists of 3,668,309 records overall from both training and testing sets. Table 3 illustrates the distribution of labels for the binary class and types for the multiple-class, while Table 4 presents the features of the corresponding dataset.

3.2. Phase 1 data preprocessing and class balancing

Data preprocessing refers to the process of transforming raw data into a clean, consistent, and meaningful format that can be used for analysis. It plays a vital role in ensuring the quality and suitability of data for machine learning based classification models in IoT security (Kotsiantis et al., 2007). Thus, to achieve accurate and reliable results, proper data preprocessing of IoT datasets is crucial. As described in the methodology framework, feature elimination, missing value handling, duplicates removal, non-numerical features encoding, and normalization, after that, data splitting is implemented to split the original data into training set and test set, in which, the training set is used for following normalization, feature reduction, and model training process,

while the test set will be set aside for final model prediction for both binary and multi-class classification.

1) Missing value handling

For TON-IoT, since all the “-” values among the features means not available from the perspective of networking domain knowledge, for example, the connection feature named “service” that has “-” value, which means the instance does not have the service value. Similarly, the instances that have “-” value in DNS features means that the instances are not DNS-capable instances. In the same way, the remaining features involving SSL, HTTP and Violation features that contain “-” value, means these instances do not support the SSL, HTTP and Violation capability. Thus, we replace it with the value “n/a”, and will create a new feature, which means not available for this feature, in the following non-numerical encoding step.

As for BoT-IoT, there are two types of missing values: the “-1” value in the feature “sport” and the “dport” value in all ARP instances. Based on the analysis implemented by (Peterson et al. 2021), we remove all the ARP instances since all should be labeled in the benign class. In addition, we changed all the hexadecimal values in the instances with the feature “proto” valued “icmp” to “-1” since ICMP traffic has no available port value.

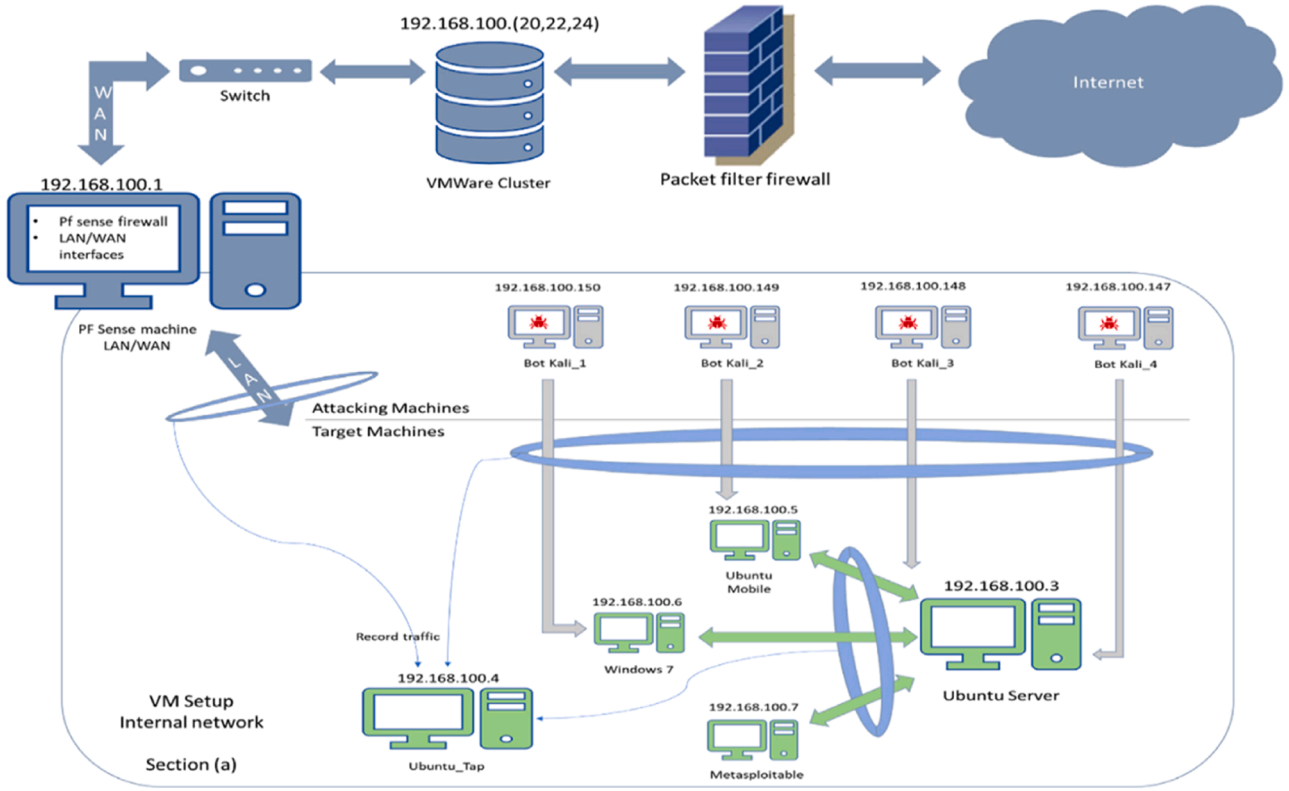


Fig. 3. Simulated testbed of BoT-IoT dataset.

Table 3
Classes description of BoT-IoT.

No.	Type of Classes	Amount
1	Normal	430
2	DDoS	1926589
3	DoS	1650211
4	Reconnaissance	91004
5	Theft	75

2) Duplicates Removal

For TON-IoT, after investigate the dataset, there are 11071 rows duplicated in the data set, thus, we need consider the mechanism to handle. Because duplicate instances cannot contribute to meaningful data to the model building process, we directly drop the duplicate instances. Here we remove the duplicated instances and leave the unique ones in the dataset. Now the remaining the dataset of 449,972 rows with unique instances is generated.

As for BoT-IoT, there is no any duplicates in the dataset, so the process then moves to the next step.

3) Non-numerical Features Encoding

For TON-IoT, since non-numerical data can be used for model training process, while part of the original dataset of Network TON-IoT has 44 features, including 18 numerical features and 26 non-numerical features. Since there are many categorical features in the dataset, so we need to convert the non-numerical features into numerical ones, so that the followed reduction and machine learning algorithms can process the data. Label encoding and one-hot encoding are methods for handling categorical variables in machine learning. The choice between them depends on the specific dataset and the ML algorithm we use. Label encoding is simpler and more space-efficient, but it may introduce an

arbitrary order to categorical values. One-hot encoding avoids this issue by creating binary columns for each category, but it can lead to high-dimensional data (Gad et al., 2021). In our work, we implement different encoding scheme considering the characteristics of various features in the dataset.

We use one-hot encoding for the connection features "proto," "service," and "conn_state" due to their distinct and finite values, creating binary features like "proto_icmp," "proto_tcp," and "proto_udp." The "service_n/a" and "conn_state_n/a" will also be generated for the original "n/a" values. For DNS features like "dns_query," binary encoding is applied to indicate the presence of DNS requests. Features such as "dns_AA," "dns_RD," "dns_RA," and "dns_rejected" are converted using one-hot encoding. Non-numerical SSL features "ssl_version," "ssl_cipher," "ssl_resumed," "ssl_established," "ssl_subject," and "ssl_issuer" are encoded using binary encoding, except "ssl_resumed" and "ssl_established," which use one-hot encoding. For HTTP features, binary encoding is used for "http_uri," "http_user_agent," "http_orig_mime_types," and "http_resp_mime_types," while one-hot encoding is applied to "http_trans_depth," "http_method," and "http_version." For weird features, "weird_name" is binary encoded, and "weird_addr" and "weird_notice" use one-hot encoding.

As for BoT-IoT, since learning algorithms can only recognize numerical values, we need to transform non-numerical features in the dataset. An algorithm checks for non-numerical features, identifying "flgs," "proto," "saddr," "sport," "daddr," "dport," "state," "category," and "subcategory" as non-numerical. The "flgs" feature is redundant since it has a numerical counterpart "flgs_number," so only "flgs_number" is used. The "proto" feature, indicating networking protocols, is transformed using one-hot encoding into features like "proto_icmp," "proto_ipv6_icmp," "proto_tcp," and "proto_udp." Its numerical counterpart "proto_number" is removed as redundant. For identifiers like "saddr," "sport," "daddr," and "dport," "saddr" and "daddr" are removed, and "sport" and "dport" are converted to numerical format (int64). The "state" feature, representing transaction states, also has a numerical

Table 4
Features description in network TON_IoT.

No.	Features	Description	No.	Features	Description
1	pkSeqID	Row Identifier	24	Dpkts	Destination-to-source packet count
2	Stime	Record start time	25	Sbytes	Source-to-destination byte count
3	Flgs	Flow state flags seen in transactions	26	Dbytes	Destination-to-source byte count
4	flgs_number	Numerical representation of feature flags	27	Rate	Total packets per second in transaction
5	Proto	Textual representation of transaction protocols present in network flow	28	Srate	Source-to-destination packets per second
6	proto_number	Numerical representation of feature proto	29	Drate	Destination-to-source packets per second
7	Saddr	Source IP address	30	TnBPSrcIP	Total Number of bytes per source IP
8	Sport	Source port number	31	TnBPDstIP	Total Number of bytes per Destination IP.
9	Daddr	Destination IP address	32	TnP_PSrcIP	Total Number of packets per source IP.
10	Dport	Destination port number	33	TnP_PDstIP	Total Number of packets per Destination IP.
11	Pkts	Total count of packets in transaction	34	TnP_PerProto	Total Number of packets per protocol.
12	Bytes	Totan number of bytes in transaction	35	TnP_PerDport	Total Number of packets per dport
13	State	Transaction state	36	AR_P_Proto_P_SrcIP	Average rate per protocol per Source IP. (calculated by pkts/dur)
14	state_number	Numerical representation of feature state	37	AR_P_Proto_P_DstIP	Average rate per protocol per Destination IP.
15	Ltime	Record last time	38	N_IN_Conn_P_SrcIP	Number of inbound connections per source IP.
16	Seq	Argus sequence number	39	N_IN_Conn_P_DstIP	Number of inbound connections per destination IP.
17	Dur	Record total duration	40	AR_P_Proto_P_Sport	Average rate per protocol per sport
18	Mean	Average duration of aggregated records	41	AR_P_Proto_P_Dport	Average rate per protocol per dport
19	Stddev	Standard deviation of aggregated records	42	Pkts_P_State_P_Protocol_P_DstIP	Number of packets grouped by state of flows and protocols per destination IP.
20	Sum	Total duration of aggregated records	43	Pkts_P_State_P_Protocol_P_SrcIP	Number of packets grouped by state of flows and protocols per source IP.
21	Min	Minimum duration of aggregated records	44	Attack	Class label: 0 for Normal traffic, 1 for Attack Traffic
22	Max	Maximum duration of aggregated records	45	Category	Traffic category
23	Spkts	Source-to-destination packet count	46	Subcategory	Traffic subcategory

counterpart "state_number." One-hot encoding is applied to "state," and "state_number" is removed. After handling these features, 24 new numerical feature sets are added, leading to a total of 67 feature sets.

As a result, after the above features are encoded, the number of features will increase from the original 39 features to 77 features for TON-IoT and 43 features to 67 features for BoT-IoT in this stage, where many of them are not really significant in classifying attacks. Therefore, it is necessary to reduce such a large number of features to a few of the most important ones, which will reduce the complexity of machine learning models in the classification phase. Based on the characteristics of the features in the dataset, diverse encoding schemes are applied to the non-numerical features above, so that the transformed data is moved to the next step.

4) Feature Elimination

To maintain the generalization of the models that can be used for real-scenario classification models in IoT networks, the features that represent the identifiers of the test environment in which the data was generated are eliminated.

For TON-IoT, the "ts" feature represents the timestamp of each connection, while 'src_ip', 'src_port', 'dst_ip', 'dst_port' stands for the identifier of each instance, all these features are not significant as the predictors for following model training (Guo, 2022), therefore, after eliminating the unnecessary features in this stage, 39 features are left in the data set, with the exception of the two labels.

As for BoT-IoT, based on data analysis by (Peterson et al., 2021), the features "pkSeqID," "seq," "stime," "ltime," "saddr," and "daddr" are invalid as they limit generalization and cause overfitting, reducing the predictive model's effectiveness. The features "pkSeqID" and "seq" are row or sequence identifiers with only ordinal information, so they should be removed. The "stime" and "ltime" features, indicating start and last packet times, are less generalizable than derived features like "dur," "rate," "srate," and "drate," and should be eliminated. Additionally, all attack traffic in the BoT-IoT dataset contains private IPs for "saddr" and "daddr," which are removed because they do not reflect the general nature of IoT networks. Lastly, the numerical counterpart features 'flgs_number,' 'proto_number,' and 'state_number' should also be

removed. As a result, all 9 invalid features are deemed invalid and eliminated in this step, changing the feature sets from the original 63 to the current 54 features. The updated dataset is then advanced to the following stage.

5) Normalization

Normalization is used to keep the scale of the feature without bias to the features with large values. In machine learning, two commonly used feature scaling techniques are normalization and standardization. The studies (Liu et al., 2021; Popoola et al., 2021) used normalization technique to scale the features, thus, we use min-max scaling to normalize the data. The data in this experiment were normalized between the range of 0 and 1 using min-max scaling. The normalization formula is shown as Eq. (1):

$$X_{normalized} = \frac{X - X_{min}}{X_{max} - X_{min}} \quad (1)$$

Where, X is the original value of the data point, $X_{normalized}$ is the normalized value of the data point. X_{min} is the minimum value of the variable, while X_{max} is the maximum value of the variable of the dataset. Consequently, the same Min-Max normalization procedure is implemented on TON-IoT and BoT-IoT datasets preprocessed from the last step, respectively.

6) Data Splitting

Data splitting is to split the original data into two sets, one is training set which is used for model training, while the other sets named test sets is used for model test, or the final performance evaluation for the trained model. However, to avoid data leakage in the following step of data transformation, such as normalization and feature reduction, and following machine learning process, data splitting was implemented before that (Gad et al., 2021).

Moreover, in order to verify the effectiveness of the trained model, the proportion of the classes of the test data set keeps nearly the same class distribution as the training set to simulate the real scenario of IoT networks. Thus, we use stratified splitting scheme to split the dataset

into training and test data with the proportion of 80:20, in which the 80 % of the dataset will be used for model training to improve the performance of the final model, while the remaining percent will be used for model evaluation.

After the data split, Figs. 4 and 5 illustrate the distribution and number of instances of the 10 classes in the TON-IoT training and test sets and the distribution and number of instances of the 5 classes in the BoT-IoT training and test sets, respectively.

As demonstrated in Algorithm 1, we developed our preprocessing technique based on the preceding procedures discussed above in the phase 1 data pre-processing.

7) Class balancing

Class imbalance is a significant issue in IoT attack classification, where the majority class (normal traffic) vastly outnumbers the minority class (attack traffic). This imbalance can lead to biased machine learning models that favor the majority class, resulting in poor detection rates for attacks and undermining the effectiveness of security measures (Wang et al., 2023). To address this, the Synthetic Minority Over-sampling Technique (SMOTE) is employed to balance the classes by generating synthetic samples for the minority class. SMOTE works by selecting a minority class instance, identifying its k-nearest neighbors, and creating synthetic samples along the line segments between the original instance and its neighbors. This increases the number of minority class instances, making the dataset more balanced and improving the model's ability to learn from the minority class.

SMOTE is applied only to the training data to prevent data leakage and ensure the integrity of the evaluation process as presented in Algorithm 2. This method maintains the real-world representativeness of the test data. SMOTE is applied separately to both binary and multi-class datasets, with the exact balancing tailored to the specific class distributions within each dataset. This selective balancing helps retain the original data distribution, ensuring that subsequent feature reduction techniques accurately capture the true characteristics of the original data. By effectively handling class imbalance, SMOTE plays a crucial role in developing robust and effective models for attack classification in IoT networks.

3.3. Phase 2 feature reduction

1) Feature Selection – Pearson Correlation

There are various feature selection techniques implemented in Network Intrusion Detection Systems (NIDS) for IoT security, such as Gini-impurity (Chaabouni et al., 2019), Chi-square (Mishra et al., 2019), Information Gain (Tama et al., 2019), Mutual Information (Liu et al., 2021), and Feature Correlation (Hall). Pearson correlation-based ranking techniques have been shown to provide competitive detection accuracy and complexity compared to other methods in cybersecurity datasets (Gavel et al., 2022; Ngo et al., 2023). Therefore, this study utilizes Pearson correlation-based feature selection to identify informative features based on a defined threshold. This approach calculates the correlation between each feature and the target variable, selecting features that are not highly correlated with each other to reduce multicollinearity. The correlation score threshold is iteratively adjusted to determine the optimal number of features for building classifiers using five machine learning models, as explained in phase 3. Pearson's correlation coefficient (PCC) is employed to evaluate feature interdependencies, using a correlation matrix computed from the preprocessed training set (Zhou et al., 2021). The PCC between features is calculated to guide the feature selection process with the equations as following:

$$PCC(f_1, f_2) = \frac{cov(f_1, f_2)}{\sigma_{f_1} \times \sigma_{f_2}} \quad (2)$$

$$\frac{cov(f_1, f_2)}{\sigma_{f_1} \times \sigma_{f_2}} = \frac{\sum_{i=1}^N (x_i - M_{f_1})(y_i - M_{f_2})}{\sqrt{\sum_{i=1}^N (x_i - M_{f_1})^2} \times \sqrt{\sum_{i=1}^N (y_i - M_{f_2})^2}} \quad (3)$$

$$M_{f_1} = 1 / N \sum_i x_i \quad (4)$$

$$M_{f_2} = 1 / N \sum_i y_i \quad (5)$$

where cov is the covariance and σ is the standard deviation, while M_{f_1} and M_{f_2} indicate the means of f_1 and f_2 respectively.

The Pearson correlation coefficient measures the linear correlation between two variables, with values ranging from -1 to 1. A coefficient of 1 indicates a perfect positive correlation, -1 indicates a perfect negative correlation, and 0 indicates no correlation. The closer the value is to 1 or -1, the stronger the correlation. During the training phase, we use this coefficient to select features from the high-dimensional dataset, which are then used to generate the reduced-dimensional test data. In contrast, feature extraction methods like Principal Component Analysis (PCA)

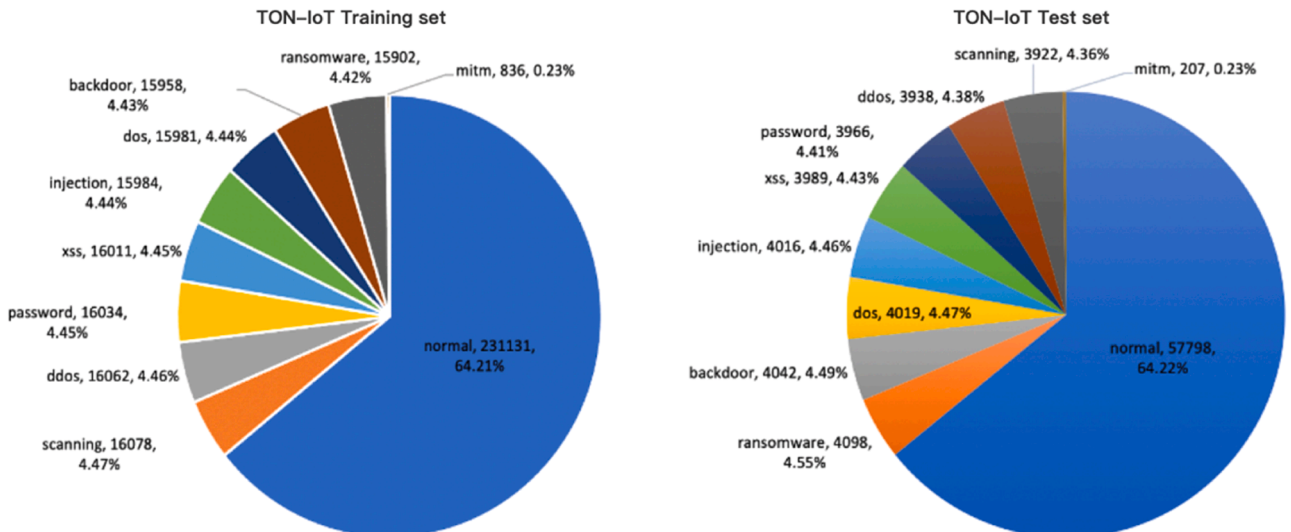


Fig. 4. Proportions of 10 classes in training and test datasets of TON-IoT.

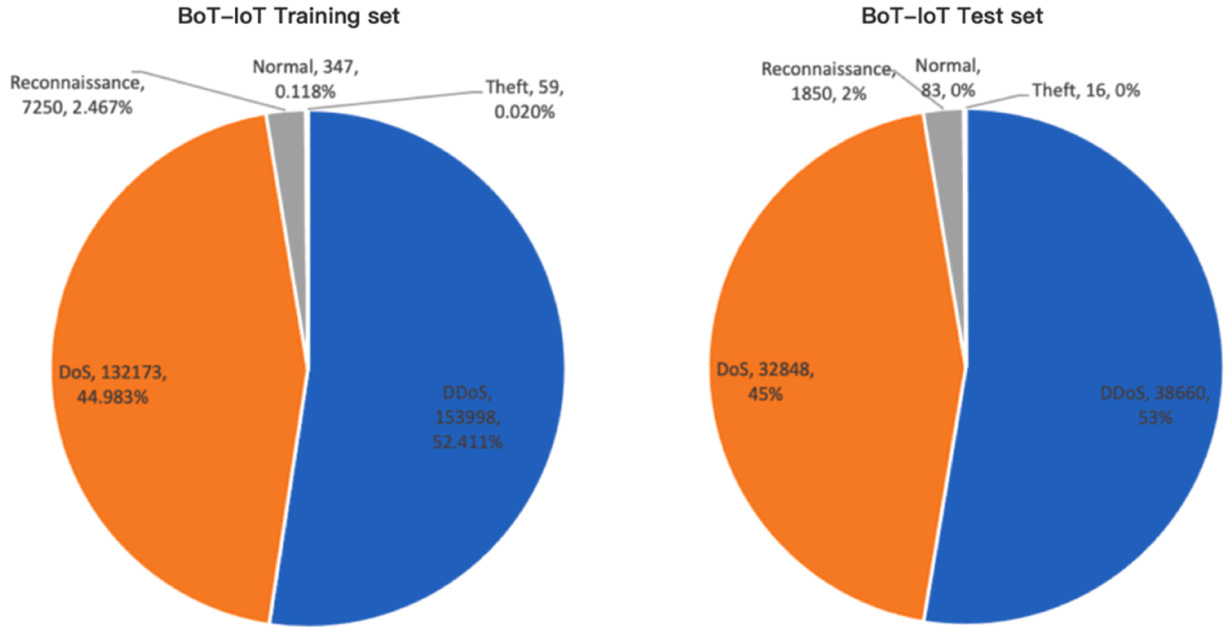


Fig. 5. Proportions of 5 classes in training and test datasets of BoT-IoT.

Algorithm 1

Pre-processing of TON-IoT and BoT-IoT Datasets in Phase 1.

Input: TON-IoT, BoT-IoT
Output: TON-IoT_train, TON-IoT_test, BoT-IoT_train, BoT-IoT_test

- Missing Value Handling
 - For TON-IoT:
 - Replace "-" values with "n/a" in relevant features (service, DNS, SSL, HTTP, Violation).
 - For BoT-IoT:
 - Remove ARP instances with "-1" in "sport" and "dport".
 - Change hexadecimal values to "-1" in ICMP "proto" instances.
- Duplicates Removal
 - For TON-IoT:
 - Remove 11071 duplicate rows.
 - For BoT-IoT:
 - No duplicates found.
- Non-numerical Features Encoding
 - For TON-IoT:
 - Apply one-hot encoding for "proto," "service," "conn_state," "dns_AA," "dns_RD," "dns_RA," "dns_rejected," "ssl_resumed," "ssl_established," "http_trans_depth," "http_method," "http_version," "weird_addl," "weird_notice."
 - Apply binary encoding for "dns_query," "ssl_version," "ssl_cipher," "ssl_subject," "ssl_issuer," "http_uri," "http_user_agent," "http_orig_mime_types," "http_resp_mime_types," "weird_name."
 - For BoT-IoT:
 - Apply one-hot encoding for "proto," "state."
 - Convert "sport," "dport" to numerical (int64).
 - Remove redundant features: "flgs," "proto_number," "state_number."
- Feature Elimination
 - For TON-IoT:
 - Remove features: "ts," "src_ip," "src_port," "dst_ip," "dst_port."
 - For BoT-IoT:
 - Remove features: "pkSeqID," "seq," "stime," "ltime," "saddr," "daddr," "flgs_number," "proto_number," "state_number."
- Normalization
 - Apply Min-Max normalization to scale data between 0 and 1.
- Data Splitting
 - Use stratified splitting to divide each dataset into training (80%) and test (20%) sets, maintaining class distribution.

End Algorithm

and Autoencoders apply dimensionality reduction to both training and test datasets, with the duration of this operation considered as part of the feature reduction process.

As demonstrated in Algorithm 3, we developed and analyzed our feature selection algorithm using Pearson correlation-based technique in the phase 2 feature reduction.

2) Feature Selection – Chi-Square

The Chi-square (χ^2) test is a statistical method used to determine the independence of two categorical variables. In the context of feature selection for IoT datasets, it assesses how strongly each feature is associated with the target variable, which is typically the class label in classification problems. The mechanism of the Chi-square test involves calculating the Chi-square statistic for each feature by comparing the observed frequency of each feature-category combination with the expected frequency if the feature and the target variable were independent.

Algorithm 2

Applying SMOTE to Pre-processed Training Data.

Input: *TON-IoT_Train*, *BoT-IoT_Train*
Output: *TON-IoT_X_train_p1*, *TON-IoT_X_test_p1*, *BoT-IoT_X_train_p1*, *BoT-IoT_X_test_p1*

1. **For** each dataset in [*TON-IoT_Train*, *BoT-IoT_Train*]:
2. Split the dataset into features (*X*) and target (*y*)
3. Split *X* and *y* into training and testing sets:
4. *X_train*, *X_test*, *y_train*, *y_test* $\leftarrow$ *train_test_split*(*X*, *y*, *test_size*=0.2, *random_state*=42)
5. Initialize SMOTE:
6. *smote_binary* $\leftarrow$ *SMOTE*(*sampling_strategy*='auto', *random_state*=42)
7. *smote_multiclass* $\leftarrow$ *SMOTE*(*sampling_strategy*='not majority', *random_state*=42)
8. Apply SMOTE to the training data:
9. If dataset is binary classification:
10. *X_train_resampled*, *y_train_resampled* $\leftarrow$ *smote_binary.fit_resample*(*X_train*, *y_train*)
11. Else if dataset is multi-class classification:
12. *X_train_resampled*, *y_train_resampled* $\leftarrow$ *smote_multiclass.fit_resample*(*X_train*, *y_train*)
13. Store the resampled training data:
14. If dataset is TON-IoT:
15. *TON-IoT_X_train_p1*, *TON-IoT_X_test_p1* $\leftarrow$ *X_train_resampled*, *X_test*
16. Else if dataset is BoT-IoT:
17. *BoT-IoT_X_train_p1*, *BoT-IoT_X_test_p1* $\leftarrow$ *X_train_resampled*, *X_test*
18. **End for** (each dataset)

Algorithm 3

Pearson correlation-based feature ranking and selection in Phase 2.

Input:
TON-IoT_X_train_p1, *TON-IoT_X_test_p1*, *BoT-IoT_X_train_p1*, *BoT-IoT_X_test_p1*
(TON-IoT_Binary: 462,262 instances, 77 features;
TON-IoT_Multi: 374,141 instances, 77 features;
BoT-IoT_Binary: 296,480 instances, 55 features;
BoT-IoT_Multi: 300,171 instances, 55 features)

Output:
(*TON-IoT_X_train_p2*, *TON-IoT_X_test_p2*, *BoT-IoT_X_train_p2*, *BoT-IoT_X_test_p2*
TON-IoT_Binary_FS: reduced 8, 22, 28, 43, 77 selected features;
TON-IoT_Multi_FS: reduced 10, 22, 34, 49, 77 selected features;
BoT-IoT_Binary_FS: reduced 3, 15, 32, 40, 55 selected features;
BoT-IoT_Multi_FS: reduced 3, 16, 33, 40, 77 selected features)

1. **For** each dataset in [*TON-IoT_Binary*, *TON-IoT_Multi*, *BoT-IoT_Binary*, *BoT-IoT_Multi*]:
2. [*Array_train*] $\leftarrow$ *X_train_p1*, [*Array_test*] $\leftarrow$ *X_test_p1*
3. **For** each feature in [*Array_train*]:
4. Start a timer to record the feature selection runtime
5. Initialize an empty list *correlation_scores* = []
6. Calculate the Pearson correlation score between feature and every other_feature
7. Append each correlation score to *correlation_scores*
8. **End for** (feature correlation)
9. Calculate the average correlation score for each feature: *average_correlation* = *sum*(*correlation_scores*) / *length*(*correlation_scores*)
10. Get the *feature_correlation_list* in descending order based on the *average_correlation*
11. If dataset is TON-IoT:
12. Set threshold ranges: *threshold_ranges* = [[-0.01, 0.01], [-0.015, 0.015], [-0.02, 0.02], [-0.03, 0.03], [*min*(*correlation_scores*), *max*(*correlation_scores*)]]
13. Else if dataset is BoT-IoT:
14. Set threshold ranges: *threshold_ranges* = [[-0.01, 0.01], [-0.02, 0.02], [-0.05, 0.05], [-0.05, 0.15], [*min*(*correlation_scores*), *max*(*correlation_scores*)]]
15. **For** each threshold range in *threshold_ranges*:
16. Get the reduced features based on *threshold_ranges*: *reduced_features* = []
17. Get the number of reduced features: *No_Feature* = *length*(*reduced_features*)
18. Get *X_train_p2*, *X_test_p2* with reduced features applied by feature selection
19. Store the feature selection runtime
20. **End for** (feature selection)
21. **End for** (each dataset)

(Mishra et al., 2019). The formula for the Chi-square statistic is given by:

$$\chi^2 = \sum \frac{(O_i - E_i)^2}{E_i} \quad (6)$$

Where O_i represents the observed frequency and E_i represents the expected frequency under the assumption of independence.

For feature ranking, each feature's Chi-square statistic is computed, and features are then ranked based on their Chi-square values. Higher Chi-square values indicate a stronger association with the target variable, suggesting that the feature is more relevant for classification. This ranking allows for the selection of the most significant features, thereby reducing the dimensionality of the IoT dataset while retaining the most informative variables. For evaluation and comparison purpose, the

number of the selected features are based on the number of the features identified by the thresholds of Pearson correlation in the Algorithm 3, following is the Algorithm 4 of the ranking and selecting the features based on Chi-square technique.

3) Feature Extraction - PCA

In the IoT security domain, feature extraction methods such as PCA (Kuang et al., 2014), LDA (Tan et al., 2010), and Autoencoders (AE) (Fatani et al., 2021) are widely used, with PCA and AE being the most common in NIDS for IoT security. Unlike feature selection, which retains selected features from the original dataset, feature extraction techniques condense high-dimensional data into lower dimensions using a projection matrix or an Autoencoder-based neural network trained on the

Algorithm 4

Feature ranking and selection using Chi-square in Phase 2.

Input: $TON\text{-}IoT_X_{train,p1}$, $TON\text{-}IoT_X_{test,p1}$, $BoT\text{-}IoT_X_{train,p1}$, $BoT\text{-}IoT_X_{test,p1}$
Output: $TON\text{-}IoT_X_{train,cs}$, $TON\text{-}IoT_X_{test,cs}$, $BoT\text{-}IoT_X_{train,cs}$, $BoT\text{-}IoT_X_{test,cs}$

1. **For** each dataset in $[TON\text{-}IoT_Binary, TON\text{-}IoT_Multi, BoT\text{-}IoT_Binary, BoT\text{-}IoT_Multi]$:
2. $[Array_train] \leftarrow X_{train,p1}$, $[Array_test] \leftarrow X_{test,p1}$
3. Set the number of Chi-square components: $Chi_features = No_Feature$ (from Algorithm 3)
4. Start a timer to record the feature ranking and selection runtime
5. Apply the Chi-square algorithm on $[Array_train]$ to calculate Chi-square statistics for each feature
6. Rank the features based on their Chi-square statistics in descending order
7. Select the top Chi score features
8. Apply the selected features to $[Array_train]$ and $[Array_test]$ to get $X_{train,cs}$, $X_{test,cs}$
9. Store the feature selection runtime
10. **End for** (each dataset)

dataset. The AE approach, often associated with deep neural networks (DNN), has higher computational complexity and greater latency compared to PCA. Given the importance of resource-constrained IoT devices and the need for low latency in NIDS to protect against cyber-attacks, this study prioritizes the more efficient PCA-based feature extraction method.

Principal Component Analysis (PCA) is a statistical method used to reduce the dimensionality of a dataset while retaining as much variance as possible. It identifies principal components, which are orthogonal directions capturing the most variance in the data. The process begins by centering the data by subtracting the mean of each feature. Then, the covariance matrix of the centered data is computed, and its eigenvalues and eigenvectors are calculated. The eigenvectors represent the principal components, and the eigenvalues indicate the variance captured by each component. The data is then projected onto these principal components, reducing its dimensionality. As presented in the Fig. 6, in a 3D dataset with three features, PCA standardizes the data, computes the covariance matrix, and derives eigenvectors and eigenvalues to find the principal components. The dataset is then transformed onto a lower-dimensional plane, such as a 2D plane defined by the first two principal components, capturing the most significant information from the original data in a simpler form. This transformation facilitates easier analysis and visualization while preserving essential data patterns and structures.

The equation that underlies PCA is as follows:

$$E = \frac{1}{n-1} X^T * X \quad (7)$$

Mathematically, X is the centered data matrix, the covariance matrix is E . The eigenvalue equation

$$\sum v = \lambda v \quad (8)$$

is solved to find the eigenvalues λ and eigenvectors v . The data is then transformed into the lower-dimensional space using:

$$Z = XV \quad (9)$$

where V is the matrix of is the matrix of selected eigenvectors, this transformation results in a reduced dataset Z that retain the most significant variance from the original data.

In the training phase, we prepare the dataset for PCA by standardizing the data—subtracting the mean and dividing by the standard deviation of each feature to ensure uniform scale. We then compute the covariance matrix E and derive its eigenvalues and eigenvectors. By sorting the eigenvalues in descending order, we select the most significant components and construct a projection matrix from these eigenvectors, projecting the data onto a lower-dimensional subspace. This results in a compact representation of the data, capturing essential information while reducing dimensionality, which simplifies machine learning models. For comparison, the number of PCA components matches the number of features selected by feature selection in each iteration. In the testing phase, the same standardization and projection process is applied to ensure consistency in the number of extracted features. Unlike feature selection, where the runtime for the test set is often ignored, the runtime for PCA on the test data is included in the overall performance evaluation in phase 3.

As demonstrated in Algorithm 5, we developed and analyzed our feature reduction algorithm based on the preceding procedures discussed above in the phase 2 feature reduction.

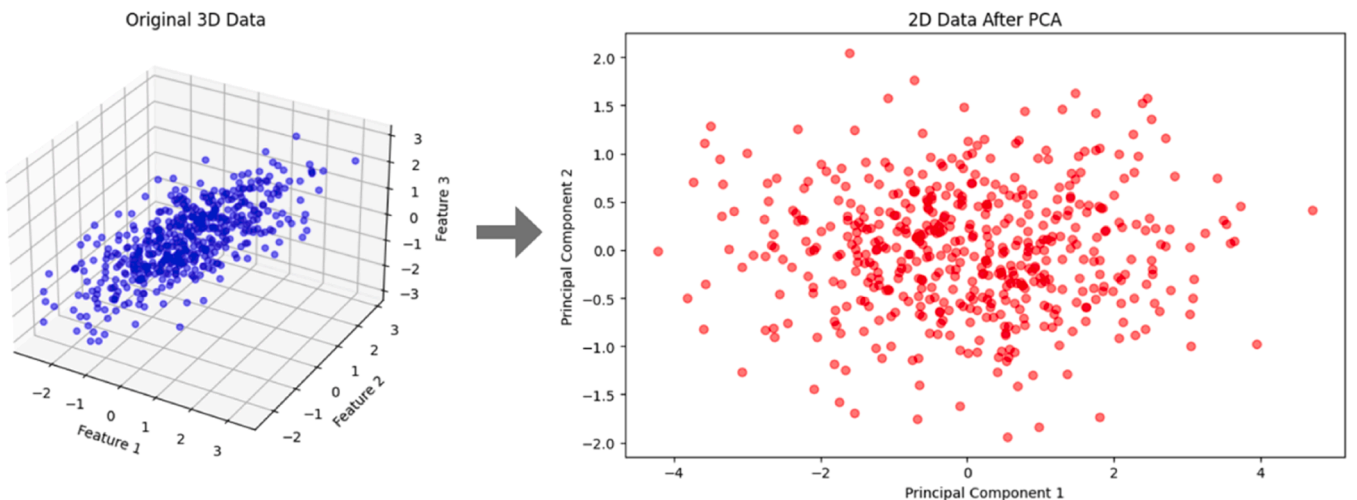
4) Feature Extraction – Autoencoder

Fig. 6. Intuition of PCA for dimensionality reduction.

Algorithm 5

Feature Extraction using PCA in Phase 2.

Input: $TON-IoT_X_{train_p1}$, $TON-IoT_X_{test_p1}$, $BoT-IoT_X_{train_p1}$, $BoT-IoT_X_{test_p1}$
Output: $TON-IoT_X_{train_pca}$, $TON-IoT_X_{test_pca}$, $BoT-IoT_X_{train_pca}$, $BoT-IoT_X_{test_pca}$

1. **For** each dataset in $[TON-IoT_Binary, TON-IoT_Multi, BoT-IoT_Binary, BoT-IoT_Multi]$:
2. $[Array_train] \leftarrow X_{train_p1}$, $[Array_test] \leftarrow X_{test_p1}$
3. Set the same number of PCA components: $PCA_components = No_Feature$ (from Algorithm 3)
4. **For** each number of PCA components in $PCA_components$:
5. Start a timer to record the feature extraction runtime
6. Apply the PCA algorithm on $[Array_train]$ and $[Array_test]$
7. Get X_{train_p2} , X_{test_p2} with $PCA_components$ applied by feature extraction
8. Store the feature extraction runtime
9. **End for** (feature extraction)
10. **End for** (each dataset)

Autoencoders (AE), as a supplement to PCA in this study, which extracts features based on linear logic, are employed to capture more complex data transformation. Autoencoders, being a type of artificial neural network, can learn non-linear mappings, making them particularly suited for handling the intricate and diverse nature of IoT data. This capability allows AEs to uncover deeper, more meaningful patterns in the data, which can be crucial for accurately classifying sophisticated cyber-attacks (Fatani et al., 2021). By utilizing PCA and AEs respectively, we can evaluate the strengths of linear and non-linear feature extraction methods, thereby enhancing the robustness and accuracy of attack classification in IoT networks.

The mechanism of an Autoencoder to extract features and reduce dimensionality involves two main components: the encoder and the decoder. The encoder compresses the input data into a latent space representation by passing it through multiple layers of neurons, effectively capturing the essential features of the data. This latent space is of lower dimensionality compared to the original input space. The decoder then attempts to reconstruct the original data from this compressed representation, ensuring that the most critical information is retained. During training, the Autoencoder adjusts its weights to minimize the difference between the input and the reconstructed output, thereby learning an efficient encoding of the data (Aminanto et al., 2018). This process allows Autoencoders to uncover complex, non-linear relationships in the data, making them highly effective for feature extraction and dimensionality reduction in diverse datasets.

The Fig. 7 illustrates the process of using an Autoencoder for feature extraction and dimensionality reduction. For example, the original data on the left is the original 3D data represented by blue points. This dataset has three features and shows the initial high-dimensional space. The encoded data in the middle, the green points represent the encoded data. The Autoencoder compresses the original data into a 2D latent space, reducing its dimensionality while retaining the most significant features. This step demonstrates how the Autoencoder learns a lower-dimensional representation of the original data. Then, the reconstructed data on the

right are the red points show the reconstructed data from the 2D encoded space back to a 3D space. Although the reconstructed data may not perfectly match the original data, it retains the essential structure and patterns, highlighting the Autoencoder's ability to compress and then decompress the data effectively. The arrows indicate the transformation process from the original high-dimensional data to the lower-dimensional encoded representation and then back to the reconstructed data. This process captures the essential features of the data while reducing its dimensionality, making it useful for simplifying machine learning models, particularly for resource-constraint IoT environment.

The equations in the Autoencoder (AE) process to extract features and reduce dimensionality are presented as following.

a) Encoder Function:

The encoder compresses the input data x into a latent space representation z . This is typically done using a series of affine transformations followed by non-linear activation functions. The equation can be written as:

$$z = f_{encoder}(x) = \sigma(W_e x + b_e) \quad (10)$$

where W_e and b_e are the weights and biases of the encoder, respectively, and σ is the activation function, and ReLU is used in the encoder function in this study.

b) Latent Representation:

$$z = h \quad (11)$$

The latent space representation z is a lower-dimensional encoding of the input data x , this latent space representation is used for further

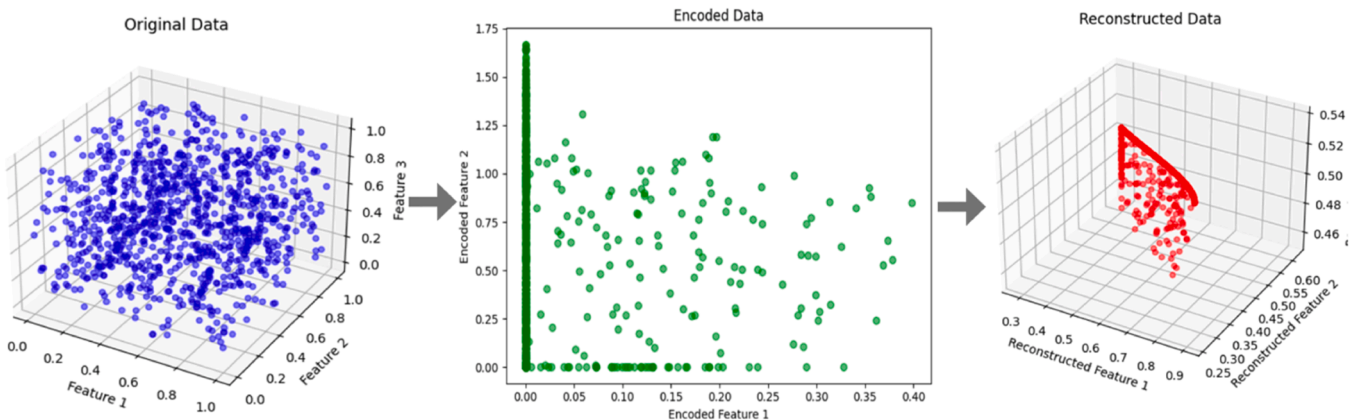


Fig. 7. Intuition of autoencoder for dimensionality reduction.

analysis, such as classification or clustering, as it retains the most important information while reducing the dimensionality of the data, in this study, we will use this as the input data with extracted features for classification evaluation.

c) Decoder Function:

The decoder reconstructs the original data x from the latent space representation z . This is also done using a series of affine transformations followed by activation functions. The equation can be written as:

$$\hat{x} = f_{\text{decoder}}(z) = \sigma'(W_d z + b_d) \quad (12)$$

where W_d and b_d are the weights and biases of the decoder, respectively, and σ' is the activation function, in which sigmoid function is used in the decoder function.

d) Reconstruction Error:

The difference between the original input x and the reconstructed output $\hat{x}$ is measured using a loss function, Mean Squared Error (MSE) is used in this study:

$$L(x, \hat{x}) = \|x - \hat{x}\|^2 \quad (13)$$

e) Optimization Objective:

During training, the Autoencoder optimizes its parameters (W_e , b_e , W_d , b_d) to minimize the reconstruction error over all training samples:

$$\text{Min}_{(W_e, b_e, W_d, b_d)} = \sum_{i=1}^N L(x^{(i)}, \hat{x}^{(i)}) \quad (14)$$

where N is the number of training samples.

These equations collectively describe the process by which an Autoencoder learns to extract features and reduce the dimensionality of the input data.

In the training phase, the preprocessed data is passed through the encoder part of the Autoencoder, which compresses the data into a lower-dimensional latent space by applying multiple layers of affine transformations and non-linear activations. This latent space captures the essential features of the data. The decoder then reconstructs the original data from this compressed representation, ensuring minimal reconstruction error. This results in a compact representation that simplifies machine learning models. For comparison, the latent space dimensions of the Autoencoder match the number of features selected by feature selection in each iteration. In the testing phase, the same process

is applied to the test dataset to maintain consistency in the number of extracted features. The runtime of the Autoencoder for the test data is included in the overall runtime calculation for feature reduction in phase 3. Algorithm 6 demonstrates the steps for feature extraction using an Autoencoder, ensuring the number of features matches those identified by Pearson correlation.

3.4. Phase 3 attack classification

Five machine learning classifiers are applied respectively and are fed with the pre-processed data after applying each feature reduction technique. These classifiers include Decision Trees (DT), Random Forests (RF), k-Nearest Neighbors (kNN), Naive Bayes (NB), and Multi-Layer Perceptron (MLP). The hyperparameters for each classifier will be detailed in Section 4.

1) Decision Tree (DT)

A Decision Tree (DT) is a supervised machine learning algorithm used for classification and regression tasks. It works by recursively splitting the dataset into subsets based on the most significant feature at each node, forming a tree-like structure. Each internal node represents a feature (or attribute), each branch represents a decision rule, and each leaf node represents an outcome (or class label) (Ingre et al., 2018). The process begins with the root node, where the feature that provides the highest information gain or the lowest Gini impurity is selected to split the data. Gini impurity is used in this work for a split calculated as Eq. (14) illustrates:

$$G(D) = \sum_{i=1}^C (P(i) + (1 - P(i))) \quad (15)$$

where $P(i)$ is the probability of a class I in dataset D , with class labels C .

For IoT classification, DTs are beneficial due to their simplicity, interpretability, and low computational cost, making them suitable for resource-constrained IoT devices. They can efficiently handle high-dimensional data and provide clear decision rules, facilitating quick and accurate classification of IoT network traffic, essential for detecting and mitigating security threats in real-time.

2) Random Forest (RF)

Random Forest (RF) is an ensemble machine learning algorithm used for classification and regression tasks. It works by constructing multiple decision trees during training and outputting the mode of the classes (classification) or mean prediction (regression) of the individual trees (Negandhi et al., 2019). Each tree in the forest is built from a random subset of the training data (bootstrapping), and at each node, a random

Algorithm 6

Feature Extraction using Autoencoder in Phase 2.

Input: $TON\text{-}IoT_X_{train_p1}$, $TON\text{-}IoT_X_{test_p1}$, $BoT\text{-}IoT_X_{train_p1}$, $BoT\text{-}IoT_X_{test_p1}$
Output: $TON\text{-}IoT_X_{train_ae}$, $TON\text{-}IoT_X_{test_ae}$, $BoT\text{-}IoT_X_{train_ae}$, $BoT\text{-}IoT_X_{test_ae}$
1. For each dataset in $[TON\text{-}IoT_Binary, TON\text{-}IoT_Multi, BoT\text{-}IoT_Binary, BoT\text{-}IoT_Multi]$:
2. $[Array_train] \leftarrow X_{train_p1}$, $[Array_test] \leftarrow X_{test_p1}$
3. Set the same number of Autoencoder dimensionality: $AE_dim = No_Feature$ (from Algorithm 3)
4. Define the Autoencoder architecture:
5. Input layer $\leftarrow \text{Input}(\text{shape}=(Array_train.\text{shape}[1]),)$
6. Encoded layer $\leftarrow \text{Dense}(AE_dim, \text{activation}='relu')(Input\ layer)$
7. Decoded layer $\leftarrow \text{Dense}(Array_train.\text{shape}[1], \text{activation}='sigmoid')(Encoded\ layer)$
8. Compile the Autoencoder model with optimizer='adam' and loss='mse'
9. Train the Autoencoder model on $Array_train$ with $Array_train$ as both input and output
10. Get the encoded data by creating an Encoder model from the trained Autoencoder:
11. Encoder model $\leftarrow \text{Model}(Input\ layer, Encoded\ layer)$
12. $TON\text{-}IoT_X_{train_ae}, TON\text{-}IoT_X_{test_ae} \leftarrow \text{Encoder model.predict}(Array_train), \text{Encoder model.predict}(Array_test)$
13. Store the feature extraction runtime
14. End for (feature extraction)
15. End for (each dataset)

subset of features is considered for splitting. This randomness helps in reducing overfitting and improving generalization. The prediction of the Random Forest for classification is given by the majority vote among the trees:

$$y = \text{mode} \{h_t(x)\}, t = 1, 2, \dots, T \quad (16)$$

Where, $h_t(x)$ is the prediction of the t -th tree for input x , and T is the total number of trees.

For IoT classification, Random Forests are particularly beneficial due to their robustness, ability to handle large datasets with high dimensionality, and resistance to overfitting. They can process complex, non-linear relationships in data, making them effective for analyzing the diverse and noisy data typical of IoT environments. Moreover, their ensemble nature ensures high accuracy and reliability, which is crucial for real-time IoT security applications, where accurate detection and rapid response to threats are essential.

3) k-Nearest Neighbors (kNN)

k-Nearest Neighbors (kNN) is a simple, non-parametric, lazy learning algorithm used for classification and regression tasks. It works by identifying the k closest training examples to a given test instance in the feature space and predicting the class label based on the majority vote of these neighbors (for classification) or the average value (for regression) (Almseidin et al., 2017). The distance between instances is typically measured using Euclidean distance:

$$d(x, x') = \sqrt{\sum_{i=1}^n (x_i - x'_i)^2} \quad (17)$$

Where x and x' are two instances, and n is the number of features.

For IoT classification, kNN is beneficial due to its simplicity and effectiveness in handling non-linear data distributions. It does not make any assumptions about the underlying data distribution, making it flexible for various types of IoT data. kNN is particularly useful in scenarios where the decision boundaries are complex, as it relies on the local structure of the data. Additionally, kNN's lazy learning approach means it has low training time, which is advantageous for rapidly changing IoT environments. However, it is essential to note that kNN can be computationally intensive during the prediction phase, which may require optimization for use in resource-constrained IoT devices.

4) Naive Bayes (NB)

Naive Bayes is a probabilistic machine learning algorithm based on Bayes' theorem, used for classification tasks. It assumes that the features are conditionally independent given the class label, which is a "naive" assumption but simplifies the computation significantly (Mukherjee & Sharma, 2012). The algorithm calculates the posterior probability of each class given the input features and assigns the class with the highest probability. The posterior probability $P(L | X)$ is computed using Bayes' theorem:

$$P(L | X) = \frac{P(X | L) P(L)}{P(X)} \quad (18)$$

Where $P(L | X)$ is the prior probability of class L , $P(X | L)$ is the likelihood of features X given class L , and $P(X)$ is the evidence of marginal likelihood of features X .

For IoT classification, Naive Bayes is particularly beneficial due to its simplicity, computational efficiency, and effectiveness with large datasets. Its assumption of feature independence, while not always accurate, often yields good performance in practice, especially for high-dimensional data. Naive Bayes classifiers are highly scalable and can handle real-time classification tasks efficiently, making them suitable for resource-constrained IoT environments.

5) Multi-Layer Perceptron (MLP)

A Multi-Layer Perceptron (MLP) is a type of artificial neural network used for classification and regression tasks. It consists of an input layer, one or more hidden layers, and an output layer. Each layer is made up of neurons (nodes), and each neuron in one layer is connected to every neuron in the next layer, forming a fully connected network (Amato et al., 2017). The neurons apply a weighted sum of their inputs, followed by a non-linear activation function such as ReLU (Rectified Linear Unit):

$$f(x) = \max(0, x) \quad (19)$$

The output of each neuron is passed through to the next layer until the final output layer. The network is trained using backpropagation, where the error between the predicted output and the actual target is minimized by adjusting the weights using gradient descent. The objective is to minimize a loss function, typically the mean squared error (MSE) for regression or cross-entropy for classification:

$$L = - \sum_{i=1}^n y_i \log(\hat{y}_i) \quad (20)$$

Where y_i is the true label and $\hat{y}_i$ is the predicted probability for class i .

For IoT classification, MLPs are beneficial due to their ability to model complex, non-linear relationships in the data. They can learn intricate patterns and representations, making them highly effective for detecting sophisticated cyber threats and anomalies in IoT networks. Despite requiring more computational resources compared to simpler models, their deep learning capabilities enable them to provide high accuracy and adaptability, which is crucial for the dynamic and diverse nature of IoT environments.

As shown in Algorithm 7, we created above five classifiers and trained it with 80 % of the dataset samples, and tested it with the remaining 20 % samples for performance evaluation between feature selection and feature extraction.

4. Experimental setup and analysis

This section details the experimental configuration, including the code implementation environment, the performance metrics for both binary and multi-class classification, and the hyperparameters of the applied classifiers. Each classifier's hyperparameters are presented as defaults for performance comparison. Performance metrics such as accuracy, precision, recall, F1-score, Matthews correlation coefficient (MCC), and runtime of feature reduction and modeling are evaluated to assess the effectiveness of the feature reduction methods. The benchmark schemes for the number of feature sets are identified by the correlation technique, and the feature sets are then derived using chi-square for feature selection and PCA, or autoencoders (AE), for feature extraction. These methods are applied to the preprocessed TON-IoT and BoT-IoT datasets to simulate real IoT characteristics and ensure robust evaluation of the classifiers under different feature reduction scenarios.

4.1. Experimental setup

The computing platform utilized is Google Colab Pro, featuring a 2-core Xeon 2.2GHz processor, 16 GB of RAM, and 100 GB of disk space. The operating system is Linux, running on an x86_64 machine architecture. The Python version used is 3.10.12. Several essential packages are employed, including Pandas, Numpy, Scikit-learn, Keras, Matplotlib, Scipy, Scikit-plot, and Time, facilitating various aspects of data processing, machine learning and deep learning models building, and visualization. This setup ensures robust performance and efficient handling of the computational demands associated with the implementation and evaluation of the NIDS framework.

Algorithm 7

Normal/Attack Classification in Phase 3.

Input: $TON-IoT_X_train_p2$, $TON-IoT_X_test_p2$, $BoT-IoT_X_train_p2$, $BoT-IoT_X_test_p2$
Output: Performance metrics and corresponding run time for each case

1. Initialize sets of datasets: $TON-IoT_X_train_p2$, $TON-IoT_X_test_p2$, $BoT-IoT_X_train_p2$, $BoT-IoT_X_test_p2$ with reduced features $[]$ from [Pearson correlation, Chi-square, PCA, Autoencoder] respectively, and y_train , y_test
2. Initialize a list of machine learning models: [DT, RF, kNN, NB, MLP]
3. Initialize classification functions: [binary, multiclass]
4. For each dataset in [$TON-IoT_X_train_p2$, $TON-IoT_X_test_p2$, $BoT-IoT_X_train_p2$, $BoT-IoT_X_test_p2$]:
5. For each feature reduction technique in [Pearson correlation, Chi-square, PCA, Autoencoder]:
6. X_train_p2 , $X_test_p2 \leftarrow$ dataset with feature reduction technique
7. For each model in models:
8. For each classification in classification functions:
9. start a timer to record the model training and testing runtime
10. train the model on the training set: $model.fit(X_train_p2, y_train)$
11. make predictions on the test set: $y_pred = model.predict(X_test_p2)$
12. stop the timer to calculate the runtime
13. calculate performance metrics: accuracy, precision, recall, $f1_score$
14. store the performance metrics and runtime
15. End for (classification tasks)
16. End for (models)
17. End for (feature reduction techniques)
18. End for (datasets)

4.2. Performance evaluation

The classification performance metrics including accuracy, precision, recall, F1-score, and Matthews correlation coefficient (MCC) are evaluated in this study, since they are critical metrics using true positive (TP), true negative (TN), false negative (FN), and false positive (FP) values, for evaluating attack classification in IoT systems. Accuracy is overall correctness of the classifier, calculated as accuracy in Eq. (21). Precision indicates the proportion of true positive results among all positive predictions, highlighting the classifier's ability to avoid false positives in Eq. (22). Recall reflects the proportion of true positive results among all actual positive instances, emphasizing the detection rate of actual attacks in Eq. (23). The F1-score, combines precision and recall into a single metric, providing a balanced measure of the classifier's performance, particularly in imbalanced datasets in Eq. (24). The MCC, offers a comprehensive measure of the classifier's quality, considering true and false positives and negatives in Eq. (25). These metrics are crucial indicators for assessing the effectiveness of IoT attack classification, ensuring accurate detection and minimizing false alarms, which are vital for maintaining the security and reliability of IoT networks.

$$Accuracy = \frac{(TP + TN)}{(TP + FN + FP + TN)} \quad (21)$$

$$Precision = \frac{TP}{(TP + FP)} \quad (22)$$

$$Recall = \frac{TP}{(TP + FN)} \quad (23)$$

$$F1 - Score = \frac{2 * Precision * Recall}{Precision + Recall} \quad (24)$$

$$MCC = \frac{(TP \times TN) - (FP \times FN)}{\sqrt{(TP + FP)(TP + FN)(TN + FP)(TN + FN)}} \quad (25)$$

In addition, feature reduction time, model training time, and model test time are essential metrics for evaluating the efficiency of attack classification in IoT systems. Reduction time comprises the duration for calculating feature scores and identifying significant features, defined as Eq. (26). This metric is crucial for assessing the preprocessing overhead. Model training time, given by Eq. (27), measures the duration required to train the classification model on the training dataset. This metric is indicative of the computational cost and feasibility of deploying the model in resource-constrained IoT environments. Model test time as Eq. (28), refers to the duration needed to make predictions on the test

dataset. This is a critical indicator of the model's performance in real-time detection scenarios. Together, these metrics provide a comprehensive understanding of the time efficiency of the feature reduction techniques and machine learning models in IoT attack classification, ensuring that the systems can operate effectively and promptly in practical deployments.

$$Reduction\ Time = Time_{Feature\ Calculation} + Time_{Feature\ Selection} \quad (26)$$

$$Training\ Time = Time_{Model\ Training} \quad (27)$$

$$Inference\ Time = Time_{Model\ Testing} \quad (28)$$

4.3. Hyperparameter settings of classifiers

To perform binary and multiclass classification tasks, we utilize five machine learning models from the Python Scikit-learn library: Decision Tree (DT), Random Forest (RF), k-Nearest Neighbors (kNN), Gaussian Naive Bayes (NB), and Multi-layer Perceptron (MLP). The hyperparameter settings for each model are presented. For the DT model, we use the Gini criterion, best splitter, no maximum depth, and a random state of 42. The RF model is configured with 100 estimators, the Gini criterion, a maximum depth of 5, and a random state of 42. The kNN model is set with 3 neighbors. The NB model uses its default parameters. The MLP model includes a hidden layer size of 100, ReLU activation, an alpha of 0.0001, auto batch size, constant learning rate, and a maximum of 200 iterations. These configurations ensure that each model is optimally tuned for our classification tasks, providing a comprehensive evaluation of their performance on IoT datasets.

4.4. Features selected based on correlation thresholds

The feature selection scheme for both preprocessed binary and multi-class TON-IoT and BoT-IoT data involve selecting features based on their average scores within predefined ranges, as shown in the Figs. 8 and 9. For both datasets, features within Range 1 (red dashed lines) are of the highest importance and are selected first. Features within Range 2 (pink dashed lines) are moderately important and are included next. Features within Range 3 (yellow dashed lines) are of lower importance but are optionally included if needed. Features within Range 4 (green dashed lines) are minimally important and generally excluded unless specifically required. Features within Range 5 (blue dashed lines) are likely noise and avoided. While the selection process is consistent for both binary and multi-class datasets, the slight differences in the number of selected features arise due to the varying importance of features in

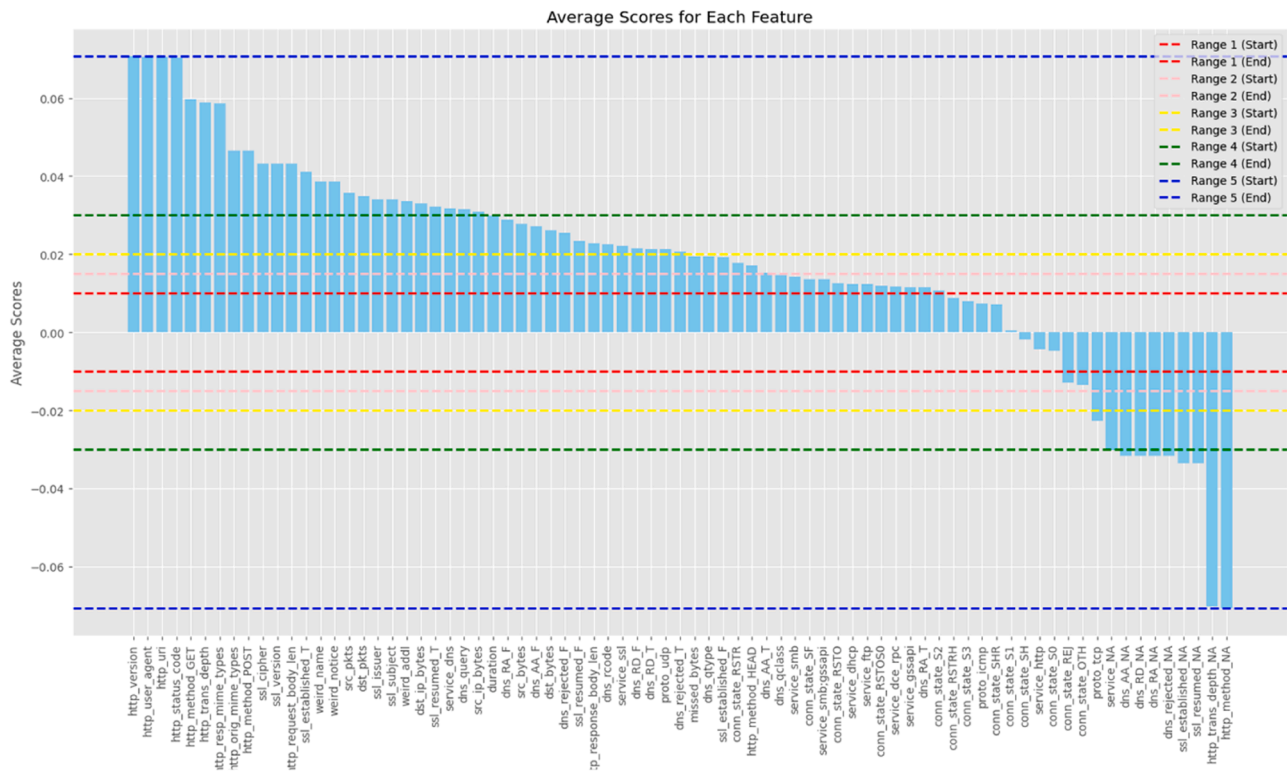


Fig. 8. Average correlation score for each feature in pre-processed binary class TON-IoT.

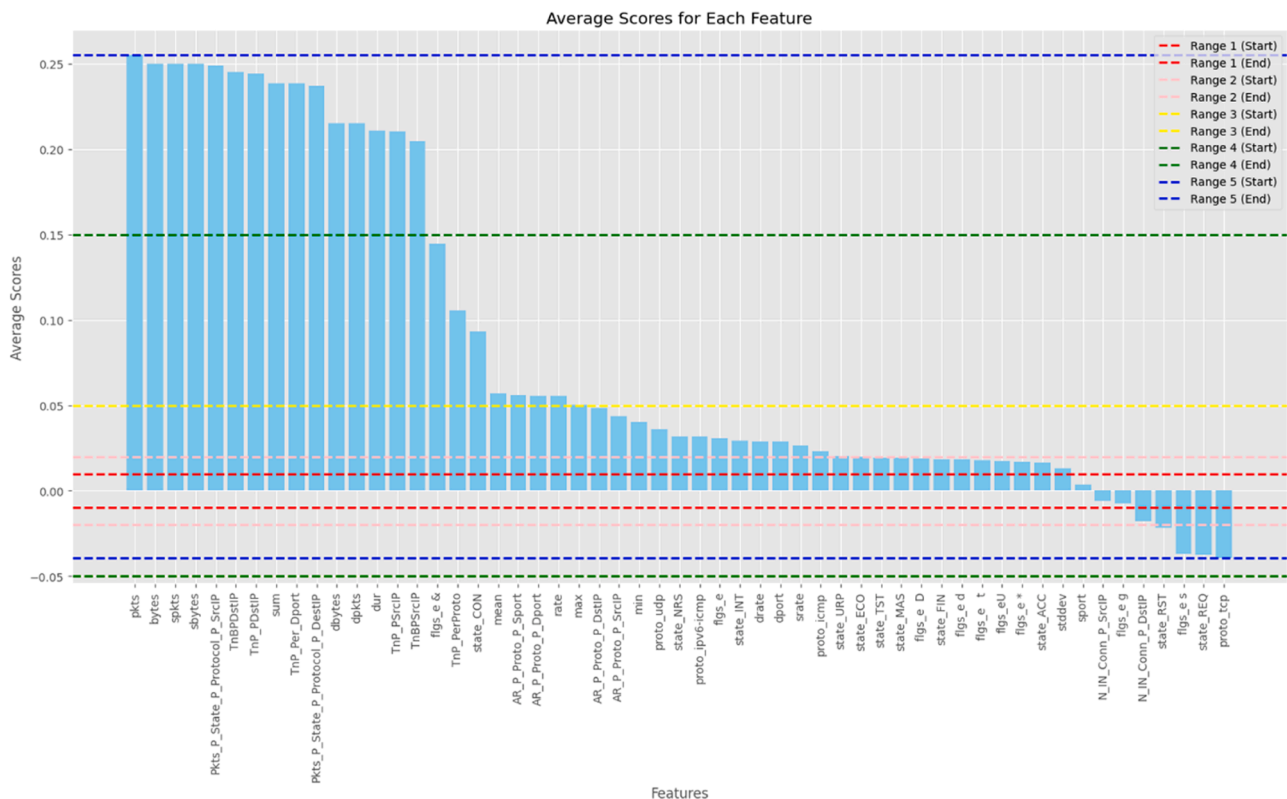


Fig. 9. Average correlation score for each feature in pre-processed binary class BoT-IoT.

distinguishing between two classes versus multiple classes. Consequently, the binary dataset might select fewer features as it only needs to differentiate between normal and attack instances, whereas the multi-

class dataset requires a broader feature set to distinguish between multiple attack types.

Based on this correlation score, we select the features within the

value range to determine the number of reduced features. With Chi-square serving as the feature selection technique and PCA and Autoencoder as the feature extraction methods, a certain number of features will be selected, and then the same number of components will be computed and selected for the performance comparison study.

As a result, in Tables 5–8, we present lists of 8, 22, 28, 43, and 77 (full features) selected features, as well as the corresponding average correlation thresholds used to achieve those numbers of selected features, in order to implement comprehensive comparison for a better understanding of two feature reduction methods.

5. Result and analysis

5.1. Binary classification for TON-IoT

Initially, we explore the performance and runtime of feature selection and extraction methods in binary classification for TON-IoT dataset, presented in Tables 9, 10, 11, 12, and 13. These tables showcase the performance metrics and run time for 8, 22, 28, 43, and 77 features (full features) selected or extracted, respectively. The highlighted values, in bold and red, denote the superior outcomes among the feature reduction techniques. The results from the tables comparing various feature reduction techniques (FS and FE) for binary classification are summarized below. The classifiers used include Decision Tree (DT), Random

Table 5

Correlation threshold with the features selected for TON-IoT Binary Class Preprocessed Data.

TON-IoT Binary Class Preprocessed Data		
Correlation Threshold	No. of Features	Selected Features
[-0.01 0.01]	8	'conn_state_RSTRH', 'conn_state_S3', 'proto_icmp', 'conn_state_SHR', 'conn_state_S1', 'conn_state_SH', 'service_http', 'conn_state_S0'
[-0.015 0.015]	22	'dns_qclass', 'service_smb', 'conn_state_SF', 'service_smb:gssapi', 'conn_state_RSTO', 'service_dhcp', 'service_ftp', 'conn_state_RSTOS0', 'service_dce_rpc', 'service_gssapi', 'dns_RA_T', 'conn_state_S2', 'conn_state_RSTRH', 'conn_state_S3', 'proto_icmp', 'conn_state_SHR', 'conn_state_S1', 'conn_state_SH', 'service_http', 'conn_state_S0', 'conn_state_REJ', 'conn_state_OTH'
[-0.02 0.02]	28	'missed_bytes', 'dns_qtype', 'ssl_established_F', 'conn_state_RSTR', 'http_method_HEAD', 'dns_AA_T', 'dns_qclass', 'service_smb', 'conn_state_SF', 'service_smb:gssapi', 'conn_state_RSTO', 'service_dhcp', 'service_ftp', 'conn_state_RSTOS0', 'service_gssapi', 'dns_RA_T', 'conn_state_S2', 'conn_state_RSTRH', 'conn_state_S3', 'proto_icmp', 'conn_state_SHR', 'conn_state_S1', 'conn_state_SH', 'service_http', 'conn_state_S0', 'conn_state_REJ', 'conn_state_OTH'
[-0.03 0.03]	43	'duration', 'dns_RA_F', 'src_bytes', 'dns_AA_F', 'dst_bytes', 'dns_rejected_F', 'ssl_resumed_F', 'http_response_body_len', 'dns_rcode', 'service_ssl', 'dns_RD_F', 'dns_RD_T', 'proto_udp', 'dns_rejected_T', 'missed_bytes', 'dns_qtype', 'ssl_established_F', 'conn_state_RSTR', 'http_method_HEAD', 'dns_AA_T', 'dns_qclass', 'service_smb', 'conn_state_SF', 'service_smb:gssapi', 'conn_state_RSTO', 'service_dhcp', 'service_ftp', 'conn_state_RSTOS0', 'service_dce_rpc', 'service_gssapi', 'dns_RA_T', 'conn_state_S2', 'conn_state_RSTRH', 'conn_state_S3', 'proto_icmp', 'conn_state_SHR', 'conn_state_S1', 'conn_state_SH', 'service_http', 'conn_state_S0', 'conn_state_REJ', 'conn_state_OTH', 'proto_tcp'
N/A	77	All the features of the dataset transformed by pre-processing stage.

Table 6

Correlation threshold with the features selected for TON-IoT Multi Class Preprocessed Data.

TON-IoT Multi Class Preprocessed Data		
Correlation Threshold	No. of Features	Selected Features
[-0.01 0.01]	10	'conn_state_RSTRH', 'dns_RA_T', 'conn_state_S3', 'proto_icmp', 'conn_state_SHR', 'conn_state_S1', 'conn_state_SH', 'service_http', 'conn_state_S0', 'conn_state_REJ'
[-0.015 0.015]	22	'dns_qclass', 'service_smb', 'dns_AA_T', 'conn_state_SF', 'service_smb:gssapi', 'conn_state_RSTO', 'service_ftp', 'service_dhcp', 'service_dce_rpc', 'conn_state_RSTOS0', 'service_gssapi', 'conn_state_S2', 'conn_state_RSTRH', 'dns_RA_T', 'conn_state_S3', 'proto_icmp', 'conn_state_SHR', 'conn_state_S1', 'conn_state_SH', 'service_http', 'conn_state_S0', 'conn_state_REJ'
[-0.02 0.02]	34	'missed_bytes', 'dns_RD_F', 'dns_rejected_T', 'dns_qtype', 'conn_state_RSTR', 'http_method_HEAD', 'ssl_established_F', 'dns_RD_T', 'proto_udp', 'service_ssl', 'dns_qclass', 'service_smb', 'dns_AA_T', 'conn_state_SF', 'service_smb:gssapi', 'conn_state_RSTO', 'service_ftp', 'service_dhcp', 'service_dce_rpc', 'conn_state_RSTOS0', 'service_gssapi', 'conn_state_S2', 'conn_state_RSTRH', 'dns_RA_T', 'conn_state_S3', 'proto_icmp', 'conn_state_SHR', 'conn_state_S1', 'conn_state_SH', 'service_http', 'conn_state_S0', 'conn_state_REJ', 'conn_state_OTH', 'proto_tcp'
[-0.03 0.03]	49	'dns_query', 'src_bytes', 'service_dns', 'dst_bytes', 'dns_RA_F', 'http_response_body_len', 'dns_AA_F', 'dns_rejected_F', 'ssl_resumed_F', 'dns_rcode', 'missed_bytes', 'dns_RD_F', 'dns_rejected_T', 'dns_qtype', 'conn_state_RSTR', 'http_method_HEAD', 'ssl_established_F', 'dns_RD_T', 'proto_udp', 'service_ssl', 'dns_qclass', 'service_smb', 'dns_AA_T', 'conn_state_SF', 'service_smb:gssapi', 'conn_state_RSTO', 'service_ftp', 'service_dhcp', 'service_dce_rpc', 'conn_state_RSTOS0', 'service_gssapi', 'conn_state_S2', 'conn_state_RSTRH', 'dns_RA_T', 'conn_state_S3', 'proto_icmp', 'conn_state_SHR', 'conn_state_S1', 'conn_state_SH', 'service_http', 'conn_state_S0', 'conn_state_REJ', 'conn_state_OTH', 'proto_tcp', 'dns_rejected_NA', 'dns_RA_NA', 'dns_RD_NA', 'dns_AA_NA', 'service_NA'
N/A	77	All the features of the dataset transformed by pre-processing stage.

Forest (RF), k-Nearest Neighbors (kNN), Naive Bayes (NB), and Multi-Layer Perceptron (MLP). Performance metrics are provided for accuracy, precision, recall, F1 score, and Matthew correlation coefficient (MCC). Additionally, training times measured in seconds (s) and prediction times measured in microsecond (μ s) are recorded. The results from the tables comparing various feature reduction techniques for binary classification are summarized below:

For 8 features, Pearson Correlation shows MLP and DT with similar performance (70.71 % accuracy), while RF has the lowest accuracy (56.27 %). Chi-square technique shows RF and DT with identical performance (70.45 % accuracy). PCA significantly boosts performance, with DT achieving 86.15 % accuracy and kNN showing high precision (81.59 %). AE allows RF to outperform all with the highest accuracy (89.60 %) and F1 score (89.20 %).

For 22 features, Pearson Correlation results in DT and MLP both achieving the highest accuracy (80.74 %), while NB has a lower MCC (53.24 %). Chi-square results in DT and RF both achieving over 85 % accuracy and high MCC. PCA shows DT slightly outperforming RF with 86.21 % accuracy, and MLP maintaining high performance (71.27 % MCC). AE allows RF to excel with an accuracy of 89.26 %.

Table 7

Correlation threshold with the features selected for BoT-IoT Binary Class Preprocessed Data.

BoT-IoT Binary Class Preprocessed Data		
Correlation Threshold	No. of Features	Selected Features
[-0.01 0.01] [-0.02 0.02]	3	['sport', 'N_IN_Conn_P_SrcIP', 'flgs_e_g']
	15	['state_ECO', 'state_TST', 'state_MAS', 'flgs_e_D', 'state_FIN', 'flgs_e_d', 'flgs_e_t', 'flgs_eU', 'flgs_e_*', 'state_ACC', 'stddev', 'sport', 'N_IN_Conn_P_SrcIP', 'flgs_e_g', 'N_IN_Conn_P_DstIP']
[-0.05 0.05]	32	['AR_P_Proto_P_DstIP', 'AR_P_Proto_P_SrcIP', 'min', 'proto_udp', 'state_NRS', 'proto_ipv6_icmp', 'flgs_e', 'state_INT', 'drate', 'dport', 'srate', 'proto_icmp', 'state_URP', 'state_ECO', 'state_TST', 'state_MAS', 'flgs_e_d', 'flgs_e_t', 'flgs_eU', 'flgs_e_*', 'state_ACC', 'stddev', 'sport', 'N_IN_Conn_P_SrcIP', 'flgs_e_g', 'N_IN_Conn_P_DstIP', 'state_RST', 'flgs_e_s', 'state_REQ', 'proto_tcp']
	40	['flgs_e_&', 'TnP_PerProto', 'state_CON', 'mean', 'AR_P_Proto_P_Sport', 'AR_P_Proto_P_Dport', 'rate', 'max', 'AR_P_Proto_P_DstIP', 'AR_P_Proto_P_SrcIP', 'min', 'proto_udp', 'state_NRS', 'proto_ipv6_icmp', 'flgs_e', 'state_INT', 'drate', 'dport', 'srate', 'proto_icmp', 'state_URP', 'state_ECO', 'state_TST', 'state_MAS', 'flgs_e_D', 'state_FIN', 'flgs_e_d', 'flgs_e_t', 'flgs_eU', 'flgs_e_*', 'state_ACC', 'stddev', 'sport', 'N_IN_Conn_P_SrcIP', 'flgs_e_g', 'N_IN_Conn_P_DstIP', 'state_RST', 'flgs_e_s', 'state_REQ', 'proto_tcp']
N/A	55	All the features of the dataset transformed by pre-processing stage.

Table 8

Correlation threshold with the features selected for BoT-IoT Multi Class Preprocessed Data.

BoT-IoT Multi Class Preprocessed Data		
Correlation Threshold	No. of Features	Selected Features
[-0.01 0.01] [-0.02 0.02]	3	['sport', 'N_IN_Conn_P_SrcIP', 'flgs_e_g']
	16	['state_MAS', 'state_ECO', 'state_TST', 'flgs_e_D', 'state_URP', 'flgs_e_t', 'flgs_e_d', 'flgs_e_*', 'flgs_eU', 'state_ACC', 'state_FIN', 'stddev', 'sport', 'N_IN_Conn_P_SrcIP', 'flgs_e_g', 'N_IN_Conn_P_DstIP']
[-0.05 0.05]	33	['max', 'AR_P_Proto_P_DstIP', 'AR_P_Proto_P_SrcIP', 'min', 'proto_udp', 'state_NRS', 'proto_ipv6_icmp', 'flgs_e', 'state_INT', 'drate', 'dport', 'srate', 'proto_icmp', 'state_MAS', 'state_ECO', 'state_TST', 'flgs_e_D', 'state_URP', 'flgs_e_t', 'flgs_e_d', 'flgs_e_*', 'flgs_eU', 'state_ACC', 'state_FIN', 'stddev', 'sport', 'N_IN_Conn_P_SrcIP', 'flgs_e_g', 'N_IN_Conn_P_DstIP', 'state_RST', 'flgs_e_s', 'state_REQ', 'proto_tcp']
	40	['flgs_e_&', 'TnP_PerProto', 'state_CON', 'AR_P_Proto_P_Sport', 'mean', 'AR_P_Proto_P_Dport', 'rate', 'max', 'AR_P_Proto_P_DstIP', 'AR_P_Proto_P_SrcIP', 'min', 'proto_udp', 'state_NRS', 'proto_ipv6_icmp', 'flgs_e', 'state_INT', 'drate', 'dport', 'srate', 'proto_icmp', 'state_MAS', 'state_ECO', 'state_TST', 'flgs_e_D', 'state_URP', 'flgs_e_t', 'flgs_e_d', 'flgs_e_*', 'flgs_eU', 'state_ACC', 'state_FIN', 'stddev', 'sport', 'N_IN_Conn_P_SrcIP', 'flgs_e_g', 'N_IN_Conn_P_DstIP', 'state_RST', 'flgs_e_s', 'state_REQ', 'proto_tcp']
N/A	55	All the features of the dataset transformed by pre-processing stage.

For 28 features, Pearson Correlation results in DT and RF showing close performance with accuracy around 80-85 %, while NB's accuracy drops significantly. Chi-square results in DT and RF both achieving over 85 % accuracy. PCA shows DT and RF with accuracy around 85-86 %. AE allows RF to achieve the highest accuracy (89.53 %) and F1 score (89.15 %).

For 43 features, Pearson Correlation results in DT and RF achieving high performance with accuracy around 84-86 %. Chi-square results in DT and RF both achieving around 86 % accuracy. PCA allows DT to achieve the highest accuracy (87.44 %). AE allows RF to outperform other classifiers with 88.66 % accuracy.

For full 77 features, RF maintains top performance with 88.43 % accuracy. PCA allows RF to show the highest accuracy (89.17 %). AE allows RF to lead with 88.94 % accuracy.

Overall, Random Forest (RF) consistently shows superior performance across all feature reduction techniques, particularly with AE and PCA. Decision Tree (DT) performs well with PCA, especially as the number of features increases. k-Nearest Neighbors (kNN) and Naive Bayes (NB) exhibit varied performance, often showing high precision but lower MCC. Multi-Layer Perceptron (MLP) maintains stable high performance, particularly with PCA. In summary, feature extraction techniques like PCA and AE generally lead to better classification performance compared to feature selection methods, with Random Forest being the most robust classifier across different techniques and feature set sizes.

The effects of adding more features on metrics for performance are discussed. The Fig. 10 illustrates the performance metrics (Accuracy, Precision, Recall, F1-score, and MCC) for various feature reduction methods across different numbers of features. It highlights two primary trends: First, the performance of Feature Selection (FS) models, particularly those using Chi-square (denoted as CS), generally improves as the number of features increases, with a significant boost observed from DT_CS_8 to RF_CS_22. Second, the Feature Extraction (FE) models, represented by RF_AE (Autoencoder) and RF_PCA (Principal Component Analysis), show consistent performance regardless of the number of features, with RF_AE_8 achieving the highest performance among FE schemes. This indicates that while FS benefits from more features by selecting relevant ones, FE maintains stable performance across different feature counts. Furthermore, the performance of FE in classification surpasses that of FS, particularly when considering a small number of features. The comparison between the two methods in Fig. 7 reveals that when the number of reduced features is relatively small—8, 22, 28, and 43—the classification performance of FE notably outperforms that of FS. This advantage is particularly pronounced for the case of 8 features (the fewest features).

In addition, two key observations can be made regarding the preferred models for each feature reduction method based on the Fig. 10. For FS methods, specifically those using CS, show a clear preference for the Random Forest (RF) classifier as the number of features increases. The performance metrics (Accuracy, Precision, Recall, F1-score, and MCC) notably improve when moving from DT_CS_8 to RF_CS_22, indicating that RF performs better with an increased number of selected features. This trend continues up to RF_CS_77, showing consistent improvement in performance metrics. As for the FE methods, particularly those using Autoencoder (RF_AE) and Principal Component Analysis (RF_PCA), exhibit stable performance across different feature schemes. Among the FE methods, RF_AE_8 stands out as achieving the best performance with the fewest features. This suggests that for FE methods, the Random Forest classifier maintains consistent performance regardless of the feature count, but performs exceptionally well with the minimal feature set in RF_AE_8. Thus, the results indicate that the Random Forest (RF) classifier is preferred for both feature reduction methods, with FS methods benefiting from more features and FE methods achieving optimal performance with fewer features.

As for run time performance, we firstly investigate the run time of the two feature reduction methods. It is shown from Fig. 11, which

Table 9
Feature reduction techniques for binary classification with 8 features.

Classifier	FR Time	Training Time (s)	Prediction Time (μs)	Accuracy (%)	Precision (%)	Recall (%)	F1 Score (%)	MCC (%)
Pearson Correlation								
DT	7.83	0.96	6.99	70.71	68.67	63.70	64.15	31.98
RF		8.94	484.89	56.27	64.23	63.01	56.13	27.21
kNN		1.54	700293.46	66.14	81.61	52.70	44.75	18.49
NB		0.92	13.85	70.70	68.66	63.69	64.14	31.96
MLP		40.81	68.90	70.71	68.67	63.70	64.15	31.98
Chi-square								
DT	2.52	0.92	6.20	70.45	77.19	76.90	70.45	54.09
RF		9.12	540.79	70.45	77.19	76.90	70.45	54.09
kNN		1.73	422198.13	75.57	85.92	65.90	66.16	47.80
NB		0.91	9.87	69.59	74.11	74.86	69.57	48.97
MLP		97.11	132.78	70.45	77.19	76.90	70.45	54.09
PCA								
DT	2.34	1.60	7.98	86.15	84.73	86.63	85.38	71.33
RF		18.70	626.97	85.49	84.16	86.44	84.80	70.56
kNN		1.92	7591.11	81.59	80.00	81.59	80.54	61.58
NB		0.97	17.78	83.43	81.90	82.23	82.06	64.13
MLP		175.75	92.24	85.73	84.31	86.32	84.97	70.61
AE								
DT	534.18	2.35	9.05	84.48	83.90	86.80	84.05	70.64
RF		22.56	615.12	89.60	88.58	91.57	89.20	80.09
kNN		1.85	47838.20	86.61	85.24	87.38	85.92	72.59
NB		0.93	18.64	84.11	82.57	83.95	83.10	66.51
MLP		127.72	46.22	86.52	85.09	86.41	85.63	71.48

Table 10
Feature reduction techniques for binary classification with 22 features.

Classifier	FR Time	Training Time (s)	Prediction Time (μs)	Accuracy (%)	Precision (%)	Recall (%)	F1 Score (%)	MCC (%)
Pearson Correlation								
DT	7.52	1.31	11.12	80.74	79.13	78.65	78.88	57.78
RF		11.37	539.37	80.41	78.77	78.27	78.50	57.04
kNN		0.89	57366.56	78.65	85.26	70.66	72.19	53.98
NB		1.02	24.20	78.34	85.02	70.24	71.69	53.24
MLP		52.70	77.43	80.74	79.13	78.65	78.88	57.78
Chi-square								
DT	2.53	1.14	9.04	85.37	84.04	86.29	84.67	70.29
RF		12.12	581.82	85.77	84.31	86.09	84.95	70.39
kNN		0.85	57052.61	82.43	86.15	76.42	78.48	61.81
NB		1.00	24.13	69.52	75.44	75.54	69.52	50.98
MLP		71.98	162.40	85.36	84.03	86.29	84.66	70.28
PCA								
DT	4.32	3.64	10.17	86.21	84.79	86.70	85.44	71.46
RF		31.71	636.41	85.64	84.30	86.55	84.95	70.82
kNN		0.85	56652.62	71.13	71.84	73.72	70.72	45.52
NB		1.06	33.86	70.30	76.89	76.68	70.30	53.57
MLP		138.37	250.60	86.16	84.73	86.57	85.38	71.27
AE								
DT	523.18	6.15	9.29	75.36	74.21	76.10	74.43	50.27
RF		39.12	615.40	89.26	88.34	91.42	88.88	79.69
kNN		0.84	55478.14	82.42	81.84	84.57	81.93	66.35
NB		0.96	33.74	84.33	82.79	83.90	83.25	66.68
MLP		82.52	40.29	86.14	84.71	86.62	85.37	71.31

illustrates the runtime of four feature reduction methods (FS_PC, FS_CS, FE_PCA, and FE_AE) with varying numbers of features in a binary classification task. The graph highlights that FS_PC (Feature Selection using Principal Component) and FS_CS (Feature Selection using Chi-square) have the shortest runtimes across all feature sets, indicating high efficiency in feature selection. In contrast, FE_PCA (Feature Extraction using Principal Component Analysis) shows slightly a few longer runtimes but remains reasonably efficient for feature extraction. However, FE_AE (Feature Extraction using Autoencoder) has the longest runtime across all feature sets, indicating a more resource-intensive process. In resource-constrained IoT environments, where processing power and energy resources are limited, the quick runtimes of FS_PC, FS_CS and FS_PCA make them more suitable as they can swiftly identify and select relevant features with minimal computational overhead. Conversely, FE_AE, despite its robust feature extraction capabilities, may not be ideal

for IoT applications due to its higher resource demands.

Runtime of model training and test time is evaluated considering the efficiency of the classification models. When examining the classifiers with the fewest run times for model training and testing for each number of features, we find that with 8 features, NB generally shows the shortest training times, and DT consistently has the shortest prediction times. For 22 features, kNN and NB have minimal training times, and NB and DT maintain the quickest prediction times. With 28 features, kNN and NB are the fastest in training, and DT and NB again show the shortest prediction times. For 43 features, kNN and NB lead in shortest training times, and DT and NB have the quickest prediction times. With 77 features, kNN and NB remain the fastest in training, and NB and DT consistently have the shortest prediction times.

As for the run time of model training and test among the feature reduction methods, Pearson Correlation shows that training times for

Table 11
Feature reduction techniques for binary classification with 28 features.

Classifier	FR Time	Training Time (s)	Prediction Time (μs)	Accuracy (%)	Precision (%)	Recall (%)	F1 Score (%)	MCC (%)
Pearson Correlation								
DT	7.52	1.40	12.45	80.95	79.24	79.53	79.38	58.76
RF		12.31	540.39	80.12	78.36	79.33	78.76	57.69
kNN		0.90	62123.26	78.66	85.31	70.67	72.20	54.03
NB		1.09	29.69	78.60	85.04	70.64	72.16	53.78
MLP		234.85	154.42	80.29	78.53	79.44	78.91	57.96
Chi-square								
DT	2.44	1.28	10.72	85.49	84.16	86.43	84.80	70.55
RF		13.62	591.28	85.84	84.40	86.20	85.03	70.57
kNN		0.86	66878.56	83.30	86.68	77.66	79.74	63.71
NB		1.02	30.61	59.44	71.11	67.69	59.06	38.65
MLP		148.42	177.76	85.48	84.15	86.42	84.79	70.54
PCA								
DT	4.69	4.78	11.36	86.21	84.78	86.70	85.44	71.46
RF		38.38	641.70	85.73	84.39	86.62	85.04	70.98
kNN		0.87	63255.79	83.72	83.70	80.18	81.41	63.78
NB		1.10	40.01	70.22	76.84	76.61	70.21	53.45
MLP		59.22	99.10	85.73	84.39	86.61	85.04	70.97
AE								
DT	582.28	7.88	10.09	77.05	77.16	79.55	76.60	56.67
RF		47.22	624.39	89.53	88.57	91.62	89.15	80.13
kNN		0.84	61328.53	80.35	79.66	82.18	79.77	61.79
NB		1.01	37.84	83.97	82.41	83.52	82.87	65.92
MLP		105.11	51.73	86.14	84.72	86.63	85.37	71.32

Table 12
Feature reduction techniques for binary classification with 43 features.

Classifier	FR Time	Training Time (s)	Prediction Time (μs)	Accuracy (%)	Precision (%)	Recall (%)	F1 Score (%)	MCC (%)
Pearson Correlation								
DT	7.57	2.30	18.26	84.18	83.10	82.07	82.53	65.16
RF		18.57	601.10	86.26	84.83	85.87	85.28	70.70
kNN		1.01	78306.91	84.68	85.73	80.59	82.21	66.13
NB		1.32	49.59	81.20	84.14	75.14	77.02	58.60
MLP		146.10	146.39	86.06	84.64	86.57	85.30	71.18
Chi-square								
DT	2.51	1.62	14.38	86.11	84.68	86.59	85.34	71.25
RF		15.08	594.28	86.00	84.55	86.32	85.19	70.85
kNN		0.91	70914.04	83.91	87.19	78.47	80.58	65.08
NB		1.18	46.17	70.71	75.98	76.46	70.70	52.44
MLP		152.35	166.86	85.74	84.39	86.62	85.04	70.97
PCA								
DT	6.36	8.95	13.50	87.44	86.04	87.62	86.65	73.64
RF		51.64	669.02	85.74	84.40	86.63	85.05	71.00
kNN		0.88	70282.88	70.59	73.12	74.63	70.44	47.72
NB		1.20	53.93	81.66	79.98	80.51	80.22	60.48
MLP		141.77	242.92	86.14	84.71	86.63	85.37	71.31
AE								
DT	597.81	11.47	11.34	78.00	76.17	77.25	76.58	53.41
RF		50.93	659.31	88.66	87.89	91.05	88.30	78.87
kNN		0.86	70134.03	79.83	79.60	82.19	79.36	61.73
NB		1.07	58.10	69.42	73.47	74.40	69.38	47.86
MLP		84.20	52.31	86.15	84.72	86.63	85.38	71.33

classifiers like RF and MLP tend to increase significantly with more features, and prediction times for RF also rise notably. Chi-square demonstrates relatively stable training and prediction times for NB and DT, but RF and kNN exhibit increases with more features. PCA shows a gradual increase in both training and prediction times across most classifiers, particularly for RF and MLP. AE has the steepest increase in training times, significantly impacting DT and RF, with prediction times also increasing but not as drastically as training times. Overall, Pearson Correlation and Chi-square are more efficient for quicker model training and testing, PCA has moderate efficiency, while AE, despite its robust feature extraction capabilities, is the least efficient due to its significantly longer run times, making it less suitable for resource-constrained environments like IoT.

Finally, in order to better understand the attack detection performance among the feature reduction techniques, we evaluate and

compare the class-wise f1-score, namely normal and attack, using the best model of the feature reduction techniques respectively in each feature setting, involving 8, 22, 28, 43, and 77 full features. The Fig. 12 illustrates the class-wise F1-scores for different feature reduction techniques across various feature schemes (8, 22, 28, 43, and 77 features). In the 8 features setting, the DT model with Chi-square (DT_CS_8) shows significantly lower F1-scores for both normal and attack classes compared to the DT model with Autoencoder (DT_AE_8). This trend continues with the 22 features setting, where the RF model with Chi-square (RF_CS_22) performs well, but the RF model with Autoencoder (RF_AE_22) achieves even higher F1-scores, especially in the normal class. For 28 features, the RF model with Autoencoder (RF_AE_28) again outperforms the DT model with Chi-square (DT_CS_28), maintaining high F1-scores for both classes. Similarly, in the 43 features setting, while both the DT model with Chi-square (DT_CS_43) and the RF model

Table 13
Feature reduction techniques for binary classification with full (77) features.

Classifier	FR Time	Training Time (s)	Prediction Time (μs)	Accuracy (%)	Precision (%)	Recall (%)	F1 Score (%)	MCC (%)
Pearson Correlation/Chi-Square								
DT	/	3.67	23.70	80.27	78.52	79.53	78.93	58.04
RF		21.17	620.92	88.43	87.71	90.88	88.07	78.53
kNN		1.07	91175.45	80.31	80.60	83.28	79.96	63.83
NB		1.56	78.95	59.57	71.04	67.75	59.20	38.65
MLP		192.02	468.92	86.10	84.68	86.61	85.33	71.25
PCA								
DT	8.60	15.18	16.82	75.71	75.52	77.76	75.16	53.23
RF		66.12	705.94	89.17	88.20	91.24	88.78	79.38
kNN		0.89	77331.60	79.01	78.16	80.50	78.34	58.62
NB		1.28	68.62	70.32	76.36	76.44	70.31	52.80
MLP		195.08	502.21	86.15	84.73	86.63	85.38	71.33
AE								
DT	608.79	14.06	11.90	74.70	73.17	74.72	73.51	47.87
RF		58.67	698.77	88.94	88.08	91.19	88.57	79.21
kNN		0.85	77371.35	79.23	78.77	81.26	78.69	59.98
NB		1.11	67.90	69.35	73.39	74.32	69.31	47.71
MLP		94.88	54.44	86.14	84.72	86.62	85.37	71.31

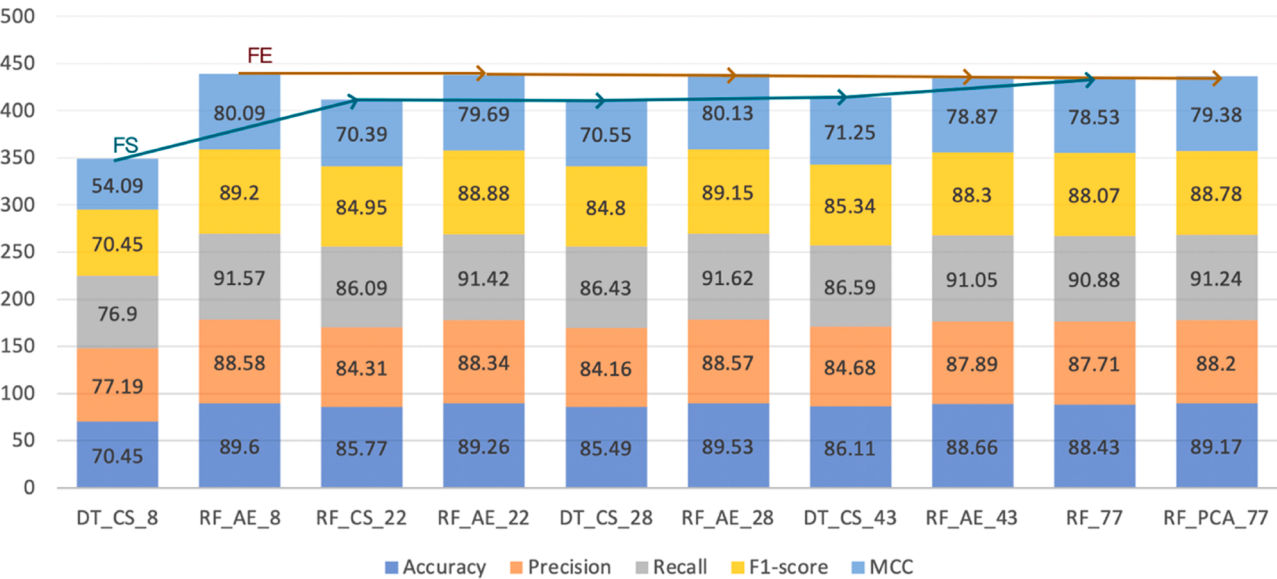


Fig. 10. The performance comparison of FS and FE models for binary classification.

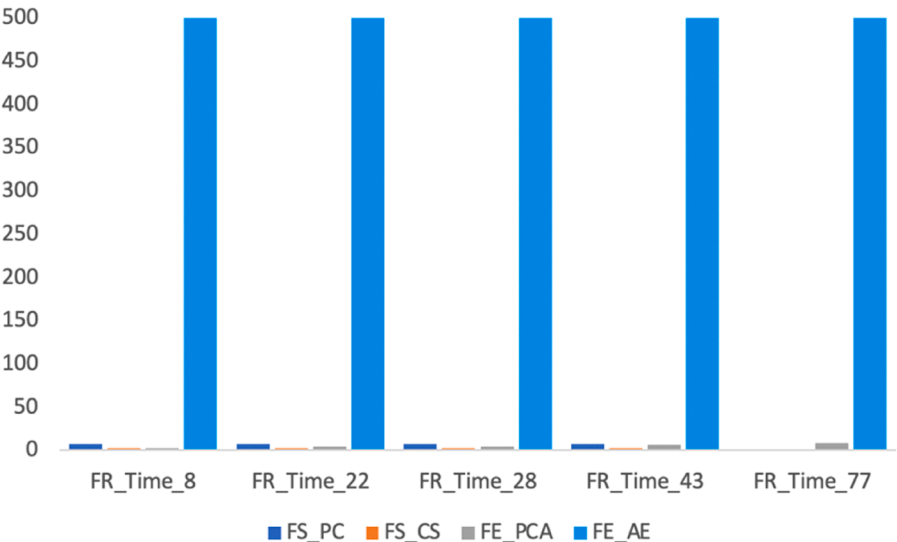


Fig. 11. The run time of FS and FE with reduced features in binary classification.

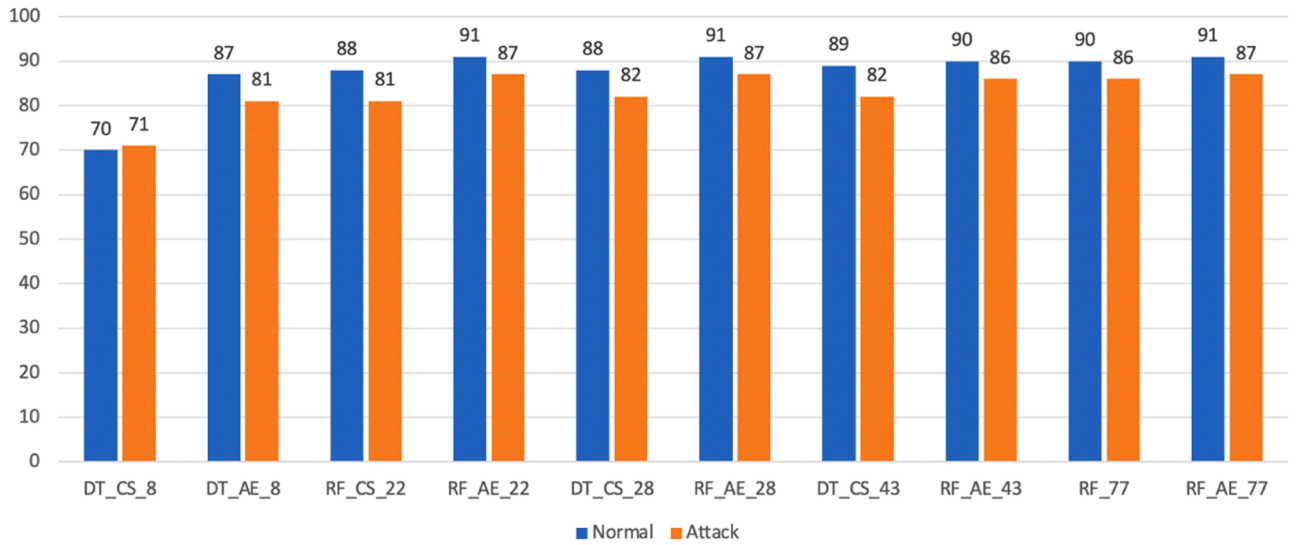


Fig. 12. The class-wise F1-score among the feature reduction techniques for reduced features.

with Autoencoder (RF_AE_43) perform well, RF_AE_43 shows superior class-wise F1-scores. Finally, with 77 features, the RF model with Autoencoder (RF_AE_77) achieves the highest F1-scores for both normal and attack classes, indicating its robustness and effectiveness in handling a larger number of features.

The consistent superior performance of the RF model with Autoencoder (RF_AE) across all feature schemes can be attributed to several factors. Firstly, Autoencoders excel at capturing the underlying structure of the data through non-linear transformations, which allows them to extract more meaningful and relevant features compared to linear methods like Chi-square. This leads to a more robust feature representation that enhances the classifier's ability to distinguish between normal and attack instances. Additionally, Autoencoders are capable of reducing noise and redundancies in the data, further improving the quality of the features used for classification. This noise reduction is particularly beneficial in complex datasets where the signal-to-noise ratio is low. The ability of Autoencoders to learn and compress features in an unsupervised manner also contributes to their effectiveness

in various feature schemes, making them more adaptable and capable of achieving better class-wise F1-scores consistently.

5.2. Binary classification for BoT-IoT

Next, we examine how well feature selection and extraction techniques perform in binary classification for BoT-IoT, as shown in Tables 14, 15, 16, 17, and 18. The performance metrics and run times for 3, 15, 32, 40, and 55 features (complete features) that were chosen or extracted using the same output format as the tables in Subsection 5.1 are displayed in these tables, accordingly. The following is an overview of the findings from the tables contrasting different feature reduction methods for binary classification:

For the table with 3 features, it is observed that the Chi-square method achieves the best performance across all classifiers, with 100 % accuracy, precision, recall, F1 score, and MCC, indicating perfect classification. Among the classifiers, the Random Forest and k-Nearest Neighbors also show high performance but with slightly lower precision

Table 14
Feature reduction techniques for binary classification with 3 features.

Classifier	FR Time	Training Time (s)	Prediction Time (μ s)	Accuracy (%)	Precision (%)	Recall (%)	F1 Score (%)	MCC (%)
Pearson Correlation								
DT	2.70	0.89	7.02	99.74	60.94	75.80	65.35	33.60
RF		16.15	441.81	99.90	80.72	59.63	64.65	34.40
kNN		0.81	5359.91	99.75	60.83	72.80	64.67	31.42
NB		0.59	7.16	56.83	50.13	78.39	36.48	3.85
MLP		146.13	61.66	99.89	78.53	59.63	64.39	33.15
Chi-square								
DT	1.11	0.56	3.23	100.00	100.00	100.00	100.00	100.00
RF		7.99	348.26	100.00	100.00	99.40	99.70	99.40
kNN		0.56	9955.19	100.00	100.00	100.00	100.00	100.00
NB		0.40	3.28	99.99	98.78	98.19	98.48	96.97
MLP		15.18	24.62	99.99	98.78	98.19	98.48	96.97
PCA								
DT	1.90	1.82	5.11	99.98	96.34	95.78	96.06	92.11
RF		26.48	476.21	99.97	99.22	88.55	93.24	87.12
kNN		0.82	3729.51	99.98	96.34	95.78	96.06	92.11
NB		0.60	10.44	99.87	73.15	87.90	78.74	59.24
MLP		42.24	58.27	99.88	74.13	92.12	80.67	63.76
AE								
DT	356.48	1.35	9.42	99.99	97.02	97.59	97.30	94.61
RF		26.22	418.02	99.98	98.58	91.56	94.80	89.87
kNN		0.87	4033.14	99.99	97.67	99.39	98.52	97.05
NB		0.59	10.18	99.71	63.12	93.84	70.15	47.97
MLP		32.13	31.30	99.99	98.73	96.38	97.53	95.09

Table 15
Feature reduction techniques for binary classification with 15 features.

Classifier	FR Time	Training Time (s)	Prediction Time (μs)	Accuracy (%)	Precision (%)	Recall (%)	F1 Score (%)	MCC (%)
Pearson Correlation								
DT	2.54	0.84	5.92	99.98	92.55	98.18	95.19	90.56
RF		14.07	406.87	99.94	89.21	80.71	84.44	69.40
kNN		1.58	535491.43	99.95	87.89	93.36	90.44	81.06
NB		0.67	15.16	90.81	50.59	94.19	48.76	10.22
MLP		82.18	108.07	99.93	82.60	95.15	87.86	76.74
Chi-square								
DT	1.16	1.29	3.19	100.00	100.00	100.00	100.00	100.00
RF		16.86	383.56	100.00	100.00	99.40	99.70	99.40
kNN		1.26	82556.29	100.00	100.00	99.40	99.70	99.40
NB		0.45	9.00	99.96	88.32	99.38	93.15	87.00
MLP		23.71	34.98	100.00	99.40	99.40	99.40	98.79
PCA								
DT	2.24	9.98	5.95	100.00	100.00	98.80	99.39	98.79
RF		73.43	389.74	100.00	100.00	99.40	99.70	99.40
kNN		1.86	4544.77	100.00	99.40	100.00	99.70	99.40
NB		0.66	21.22	99.18	55.81	97.18	60.16	33.13
MLP		28.08	49.68	100.00	99.40	100.00	99.70	99.40
AE								
DT	388.09	9.66	5.11	100.00	99.40	100.00	99.70	99.40
RF		74.72	375.12	99.99	99.99	95.18	97.47	95.05
kNN		1.94	4827.37	100.00	99.40	98.79	99.09	98.18
NB		0.62	22.82	99.99	96.02	98.79	97.37	94.77
MLP		24.61	35.66	100.00	100.00	98.80	99.39	98.79

Table 16
Feature reduction techniques for binary classification with 32 features.

Classifier	FR Time	Training Time (s)	Prediction Time (μs)	Accuracy (%)	Precision (%)	Recall (%)	F1 Score (%)	MCC (%)
Pearson Correlation								
DT	2.53	1.42	8.04	100.00	100.00	98.80	99.39	98.79
RF		19.36	452.99	99.99	99.36	96.99	98.15	96.32
kNN		0.64	40086.66	99.99	98.21	98.79	98.50	97.00
NB		0.77	32.11	90.12	50.55	93.85	48.49	9.82
MLP		36.86	119.22	99.99	96.55	98.79	97.64	95.32
Chi-square								
DT	1.09	1.55	3.79	100.00	100.00	98.80	99.39	98.79
RF		22.92	332.24	100.00	100.00	100.00	100.00	100.00
kNN		0.51	28710.17	100.00	100.00	99.40	99.70	99.40
NB		0.51	22.41	99.76	66.08	99.28	74.20	56.30
MLP		21.80	42.48	100.00	100.00	99.40	99.70	99.40
PCA								
DT	3.80	12.83	7.39	100.00	100.00	98.19	99.08	98.17
RF		103.63	432.33	100.00	100.00	98.19	99.08	98.17
kNN		0.57	36018.73	100.00	99.39	98.79	99.09	98.18
NB		0.75	35.75	99.03	55.18	98.91	59.13	31.82
MLP		32.65	43.53	100.00	100.00	100.00	100.00	100.00
AE								
DT	359.12	11.34	4.51	99.99	98.07	95.18	96.58	93.21
RF		104.85	411.61	100.00	100.00	100.00	100.00	100.00
kNN		0.54	31749.19	100.00	100.00	100.00	100.00	100.00
NB		0.65	32.64	99.87	73.47	96.33	81.14	65.95
MLP		25.70	37.88	100.00	100.00	100.00	100.00	100.00

and recall when compared to the Chi-square method. In contrast, Naive Bayes performs the worst with only 56.83 % accuracy.

For 15 features, similar trends are observed, with Chi-square and PCA methods showing excellent performance. The Random Forest and k-Nearest Neighbors, in particular, perform extremely well with high precision and recall.

As the number of features increases to 32, 40, and 55, the overall performance remains high for most classifiers, especially with the Chi-square and PCA methods, which maintain high accuracy, precision, and recall. Notably, for 55 features, the full feature set without any feature selection applied also achieves perfect performance across all classifiers.

Overall, the Chi-square method consistently yields the best results in terms of classification performance metrics across different numbers of features. PCA also performs well, especially for higher feature numbers.

While the Naive Bayes classifier tends to underperform compared to other classifiers, the Decision Tree, Random Forest, and k-Nearest Neighbors generally show robust performance across all metrics. Multi-Layer Perceptron also demonstrates high effectiveness, particularly with the Chi-square method. These observations suggest that the choice of feature reduction technique and classifier can significantly impact the effectiveness of binary classification tasks.

The Fig. 13 presents the performance metrics (Accuracy, Precision, Recall, F1-score, and MCC) for different models and feature reduction methods across varying numbers of features for BoT-IoT dataset. Initially, different from the result of TON-IoT, with fewer features (3 features), the models demonstrate a balanced performance, with Accuracy, Precision, Recall, F1-score, and MCC all showing perfect values. As the number of features increases to 32 and 43, we observe that the metrics generally maintain high levels, indicating that the models

Table 17
Feature reduction techniques for binary classification with 40 features.

Classifier	FR Time	Training Time (s)	Prediction Time (μs)	Accuracy (%)	Precision (%)	Recall (%)	F1 Score (%)	MCC (%)
Pearson Correlation								
DT	2.54	2.44	8.82	100.00	100.00	100.00	100.00	100.00
RF		25.75	396.45	100.00	100.00	100.00	100.00	100.00
kNN		0.64	41645.40	100.00	99.39	98.79	99.09	98.18
NB		0.82	39.71	95.46	51.18	96.52	51.15	14.85
MLP		38.99	128.06	100.00	100.00	100.00	100.00	100.00
Chi-square								
DT	1.08	1.79	4.29	99.99	98.79	98.79	98.79	97.59
RF		24.88	325.64	100.00	100.00	100.00	100.00	100.00
kNN		0.39	26918.27	100.00	100.00	100.00	100.00	100.00
NB		0.52	21.81	99.80	67.90	99.30	76.23	59.42
MLP		21.12	40.54	100.00	100.00	100.00	100.00	100.00
PCA								
DT	3.84	17.27	8.82	99.99	98.78	98.19	98.48	96.97
RF		113.48	425.44	100.00	100.00	98.19	99.08	98.17
kNN		0.58	39156.00	100.00	99.40	99.40	99.40	98.79
NB		0.78	47.47	99.55	59.67	96.77	65.94	42.54
MLP		24.40	38.55	100.00	100.00	100.00	100.00	100.00
AE								
DT	356.13	11.44	5.06	100.00	100.00	99.40	99.70	99.40
RF		119.17	413.71	100.00	100.00	98.19	99.08	98.17
kNN		0.54	34579.56	100.00	99.40	99.40	99.40	98.79
NB		0.66	39.73	99.55	59.67	96.77	65.94	42.54
MLP		24.40	38.55	100.00	98.82	100.00	99.40	98.82

Table 18
Feature reduction techniques for binary classification with 55 features.

Classifier	FR Time	Training Time (s)	Prediction Time (μs)	Accuracy (%)	Precision (%)	Recall (%)	F1 Score (%)	MCC (%)
Full features without FS applied								
DT	/	3.58	10.80	100.00	100.00	100.00	100.00	100.00
RF		32.21	365.98	100.00	100.00	100.00	100.00	100.00
kNN		0.65	43090.86	100.00	99.39	98.79	99.09	98.18
NB		0.92	57.46	99.78	67.02	98.69	75.18	57.57
MLP		33.46	117.23	100.00	100.00	100.00	100.00	100.00
PCA								
DT	0.774	18.70	7.54	99.99	99.38	98.19	98.78	97.57
RF		104.97	403.69	100.00	100.00	98.19	99.08	98.17
kNN		0.42	26789.10	100.00	99.39	98.79	99.09	98.18
NB		0.54	34.04	99.27	56.66	99.03	61.55	36.13
MLP		22.79	41.74	100.00	98.82	100.00	99.40	98.82
AE								
DT	343.43	18.70	7.54	99.99	99.38	98.19	98.78	97.57
RF		104.97	403.69	100.00	100.00	98.19	99.08	98.17
kNN		0.42	26789.10	100.00	99.39	98.79	99.09	98.18
NB		0.54	34.04	99.27	56.66	99.03	61.55	36.13
MLP		22.79	41.74	100.00	98.82	100.00	99.40	98.82

effectively utilize the additional features to maintain performance. However, when the feature count reaches full features (77) for both feature selection and feature extraction methods, there is a noticeable decrease in performance metrics among different models. Specifically, models such as DT, RF and MLP with feature reduction techniques like PCA and MLP with PCA or AE achieve high performance metrics consistently. This suggests that while adding more irrelevant features can decrease performance, and the choice of feature reduction technique plays a crucial role in effectively to maintain the classification performance.

As for the preferred models for each feature reduction method is also explained. When using Chi-square (CS), the Decision Tree (DT) model performs well with fewer features (3, 22, and 32 features), maintaining high performance metrics across Accuracy, Precision, Recall, F1-score, and MCC. In Autoencoder (AE), The Multi-Layer Perceptron (MLP) model shows strong performance, particularly with 3, 32, and 77 features. MLP_AE consistently achieves high scores in all performance metrics. For Principal Component Analysis (PCA), both Decision Tree (DT) and Random Forest (RF) models perform exceptionally well with 32 and 77 features. For instance, MLP_PCA and RF_PCA show superior

performance, particularly in the higher feature count settings. Combined Methods: Models like RF, ANN, and MLP with feature sets involving either CS or PCA demonstrate high performance with larger feature sets (32, 43 features). These combined approaches leverage the strengths of multiple feature reduction techniques, resulting in robust performance metrics. Thus, the Fig. 10 indicates that while adding more features generally enhances performance, the preferred models for each feature reduction method are those that can effectively handle the increased feature dimensionality. MLP with AE and PCA, and DT with CS, emerge as strong performers across different feature sets, showcasing their ability to maintain high performance across various metrics.

As for run time performance, the same findings as the result of 5.2 in TON-IoT dataset, the Fig. 14 presents the feature reduction (FR) times for various feature reduction techniques (FS_CS, FS_CS, FE_PCA, FE_AE) across different feature sets (3, 22, 32, 43, 77). It is evident that FE_AE (Feature Extraction using Autoencoder) consistently exhibits significantly higher FR times compared to the other techniques, regardless of the number of features. The FR times for FE_AE are around 350 seconds across all feature sets, indicating a substantial computational cost. In contrast, FS_CS (Feature Selection using Chi-square) and FS_CS (another

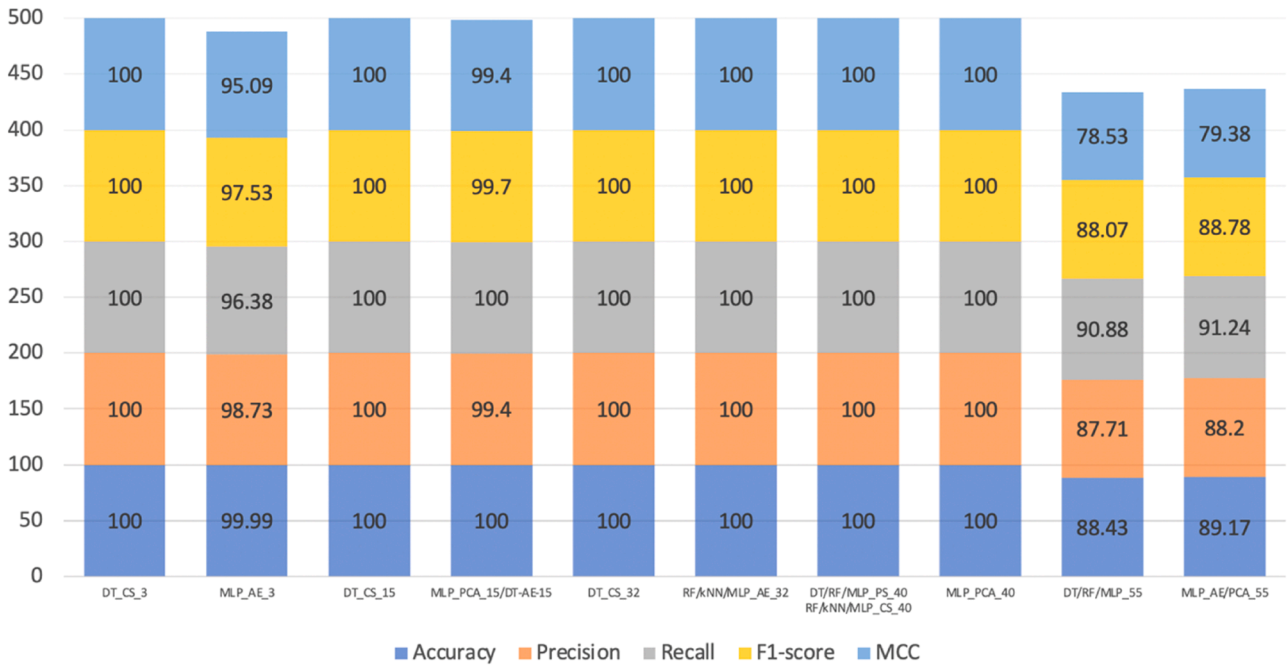


Fig. 13. The performance comparison of FS and FE models for binary classification.

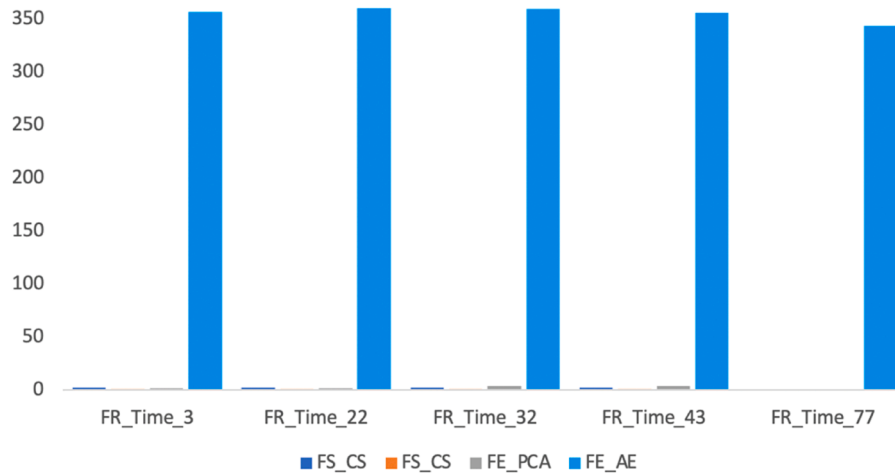


Fig. 14. The run time of FS and FE with reduced features in binary classification.

variant of Feature Selection) show minimal FR times, consistently under 10 seconds, highlighting their efficiency in feature reduction. FE_PCA (Feature Extraction using Principal Component Analysis) has moderate FR times, higher than FS_CS but considerably lower than FE_AE. This trend underscores the resource-intensive nature of Autoencoders, which, despite their robust feature extraction capabilities, demand significantly more computational time, making them less suitable for time-sensitive or resource-constrained environments compared to the more efficient FS methods.

The runtime of model training and test times is also evaluated in binary classification using BoT-IoT dataset. From the provided Tables 14, 15, 16, 17, and 18, the same as the results of TON-IoT, it is evident that the Decision Tree (DT) and Naive Bayes (NB) models consistently show the shortest training times across different feature sets. For instance, the DT model with Pearson Correlation feature reduction and 3 features has a training time of only 0.89 seconds, while the NB model with Chi-square feature reduction and 3 features has an even shorter training time of 0.40 seconds. In contrast, the Multi-Layer Perceptron (MLP) model generally exhibits longer training times,

particularly with larger feature sets. However, despite the longer training times, MLP often achieves higher performance metrics, indicating a trade-off between training time and model accuracy. The prediction times across models also vary, with DT and NB models showing significantly lower prediction times compared to models like k-Nearest Neighbors (kNN), which has very high prediction times, especially with larger feature sets. Overall, the DT and NB models are highly efficient in terms of runtime, making them suitable for applications requiring quick model training and testing.

The feature reduction methods also impact the runtime of model training and testing. The same as the results of the binary classification using TON-IoT dataset, Chi-square (CS) feature reduction consistently shows minimal feature reduction (FR) times and efficient training times across various classifiers and feature sets. For example, the FR time for Chi-square with 3 features is only 1.11 seconds, and it maintains low FR times even with larger feature sets. In contrast, Autoencoder (AE) feature reduction has significantly higher FR times, often exceeding 300 seconds, which impacts the overall training time. Despite the longer FR times, AE often results in higher performance metrics, suggesting that

while it is computationally intensive, it provides a better feature set for classification. Principal Component Analysis (PCA) shows moderate FR times and training times, striking a balance between computational efficiency and model performance. Overall, while Chi-square is preferred for its efficiency in FR time, AE is favored for its superior performance metrics, albeit at the cost of longer computational times. This indicates that the choice of feature reduction method should consider the computational resources available and the performance requirements of the classification task.

Finally, in the same way as the result analysis in TON-IoT, in order to better understand the attack detection performance among the feature reduction techniques, we evaluate and compare the class-wise f1-score, namely normal and attack, using the best model of the feature reduction techniques respectively in each feature setting, involving 3, 15, 32, 40, and 55 full features. The Figure illustrates the F1-scores for normal and attack classes in the BoT-IoT dataset across various feature reduction methods and models. The Decision Tree (DT) with Chi-square (DT_CS_3) achieves perfect F1-scores (100 %) for both normal and attack classes, demonstrating its effectiveness with minimal features. The MLP with Autoencoder (MLP_AE_3) shows slightly lower performance in the normal class (95 %), but still maintains a high score in the attack class (100 %). As the number of features increases to 15, 32, 40, and 55, the models consistently achieve perfect F1-scores across both classes. This indicates that minimal number of the features identified by feature selection or extraction can achieve perfect performance for the binary classification of BoT-IoT dataset, while the models can maintain or achieve optimal classification performance even with varying numbers of features.

Comparing the two Figs. 12 and 15, it is evident that the BoT-IoT dataset results in higher and more consistent F1-scores across both normal and attack classes compared to the TON-IoT dataset. In the TON-IoT dataset, while models like RF_AE_22 and RF_AE_28 achieve high F1-scores (91 %), other models exhibit more variability in performance, particularly in the attack class. For instance, DT_CS_8 has significantly lower F1-scores (70 for normal and 71 for attack) compared to the BoT-IoT models. This discrepancy can be attributed to several factors. The BoT-IoT dataset may have more distinct and separable features, making it easier for models to classify instances accurately. Additionally, the feature reduction techniques, especially Autoencoder and Principal Component Analysis (PCA), seem to be more effective in enhancing the classification performance for BoT-IoT. The consistency of perfect scores in BoT-IoT suggests that its data is more conducive to these techniques, whereas the TON-IoT dataset may present more complex patterns or noise that challenge the models. Therefore, while both datasets benefit

from feature reduction, the inherent characteristics of the BoT-IoT dataset make it more amenable to achieving higher classification performance.

5.3. Multi classification for TON-IoT

In this subsection, we use Tables from Tables 19–23 to examine the multi-class classification performance and computational time analysis of both FS and FE approaches for the TON-IoT dataset. Using the same criteria as the binary classification described in Subsection 5.1, the tables emphasize the best values in bold and the superior outcomes for both feature selection and extraction in red.

When analyzing the classifiers with 10 features, Pearson Correlation shows moderate accuracy for most classifiers, with kNN achieving the highest at 71.88 %. However, the prediction time for kNN is exceptionally high at 698,364.41 μ s, making it less suitable for real-time applications. Chi-square results are similar, with slightly better precision (DT: 36.56 %, kNN: 36.57 %) and recall (DT: 42.18 %, kNN: 29.39 %) for some classifiers. PCA provides a balance with high accuracy (MLP: 74.84 %) and moderate prediction time (MLP: 106.45 μ s), while AE struggles with lower accuracy (NB: 43.59 %) and higher prediction times (kNN: 50,218.27 μ s).

For 22 features, PCA outperforms other techniques in accuracy (DT: 76.25 %), precision (DT: 70.14 %), recall (DT: 54.31 %), and F1 Score (DT: 43.22 %) across most classifiers. Chi-square also shows strong performance, particularly with Decision Tree (accuracy: 75.30 %) and kNN (accuracy: 75.74 %). AE's performance improves with more features, achieving the highest accuracy (MLP: 76.13 %) and MCC (MLP: 56.29 %), but its training and prediction times remain high (RF: 947.91 μ s, MLP: 75.48 μ s).

With 34 features, PCA continues to deliver the best overall performance, particularly in terms of accuracy (DT: 76.59 %) and MCC (DT: 57.04 %). Chi-square maintains consistent performance, while AE and Pearson Correlation lag in accuracy (AE DT: 59.42 %, Pearson Correlation RF: 69.69 %).

For 49 and 55 features, PCA and AE techniques show competitive results, with AE catching up in terms of accuracy (MLP: 76.22 %, 76.16 %) and MCC (MLP: 56.41 %, 56.34 %). The training and prediction times increase with more features, highlighting the need for efficient algorithms in resource-constrained environments.

Overall, PCA consistently demonstrates the best performance across different classifiers and metrics, particularly in terms of accuracy and MCC. Chi-square also performs well, especially with Decision Tree (accuracy: 75.30 %) and kNN (accuracy: 75.74 %) classifiers. Pearson

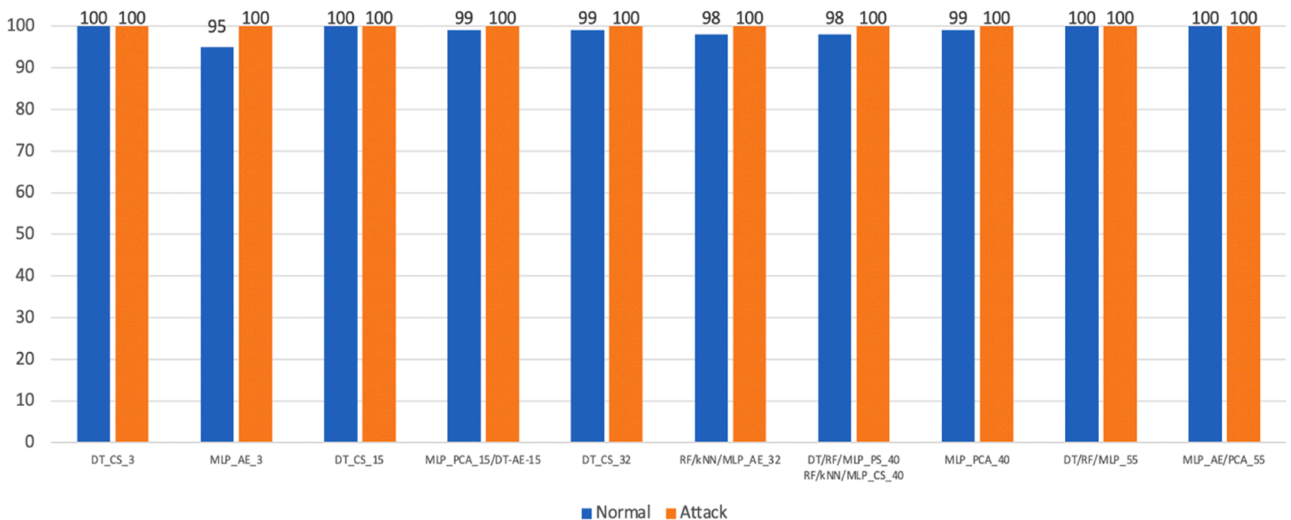


Fig. 15. The class-wise F1-score among the feature reduction techniques for reduced features.

Table 19
Feature reduction techniques for multi-classification with 10 features.

Classifier	FR Time (s)	Training Time (s)	Prediction Time (μs)	Accuracy (%)	Precision (%)	Recall (%)	F1 Score (%)	MCC (%)
Pearson Correlation								
DT	6.03	0.76	10.06	71.39	41.38	38.24	30.30	43.65
RF		8.14	748.31	71.42	33.75	27.65	24.06	41.34
kNN		1.29	698364.41	71.88	37.30	31.66	28.58	43.75
NB		0.80	42.87	19.60	24.01	39.72	19.60	22.03
MLP		36.75	102.20	71.39	41.38	38.24	30.29	43.64
Chi-square								
DT	2.10	0.73	8.26	71.53	36.56	42.18	30.08	44.50
RF		7.64	800.36	72.09	36.69	29.36	25.45	43.33
kNN		1.20	400109.98	72.04	36.57	29.39	25.40	43.24
NB		0.77	36.67	15.65	14.56	42.73	18.47	19.41
MLP		58.43	102.63	71.53	36.56	42.18	30.08	44.50
PCA								
DT	2.22	1.21	9.69	69.59	59.17	41.49	29.78	44.52
RF		15.22	955.51	70.46	42.15	41.20	28.68	45.74
kNN		1.46	12839.28	63.91	42.19	28.47	19.60	35.81
NB		0.80	77.15	55.69	36.92	55.95	39.13	44.46
MLP		383.55	106.45	74.84	54.40	51.37	40.04	53.89
AE								
DT	418.00	1.61	11.18	58.36	46.54	37.08	31.00	36.07
RF		17.83	905.69	74.68	55.09	46.61	39.24	51.49
kNN		1.49	50218.27	53.96	51.68	33.78	23.77	33.32
NB		0.77	72.95	43.59	27.89	51.66	28.69	31.40
MLP		221.44	71.72	76.13	68.82	53.81	42.79	56.27

Table 20
Feature reduction techniques for multi-classification with 22 features.

Classifier	FR Time	Training Time (s)	Prediction Time (μs)	Accuracy (%)	Precision (%)	Recall (%)	F1 Score (%)	MCC (%)
Pearson Correlation								
DT	6.06	1.05	13.87	72.31	59.62	40.38	33.46	46.01
RF		9.27	778.14	69.24	28.63	21.32	16.82	34.46
kNN		0.66	46414.78	72.55	60.57	30.39	28.82	44.51
NB		0.90	108.87	19.36	34.90	41.03	17.70	20.63
MLP		154.28	108.65	72.30	59.59	40.40	33.45	46.01
Chi-square								
DT	2.20	0.99	11.73	75.30	54.59	52.66	40.80	54.82
RF		11.21	939.17	70.62	34.10	33.59	25.48	40.43
kNN		0.65	54065.84	75.74	53.88	39.66	35.80	53.41
NB		0.90	110.57	34.19	29.43	47.14	25.75	29.82
MLP		263.61	197.56	75.30	54.59	52.66	40.80	54.82
PCA								
DT	3.17	2.78	12.09	76.25	70.14	54.31	43.22	56.39
RF		25.57	975.86	76.03	62.18	52.97	42.36	55.80
kNN		0.66	47434.46	49.15	42.76	34.07	24.59	30.58
NB		0.88	152.88	55.60	41.29	62.76	42.27	45.78
MLP		168.58	85.80	76.16	68.76	53.81	42.82	56.30
AE								
DT	476.64	5.10	11.38	60.99	50.31	33.97	22.80	37.15
RF		32.85	947.91	74.75	64.78	49.43	37.95	54.27
kNN		0.64	46885.69	54.55	46.18	34.83	24.66	32.99
NB		0.83	165.65	56.87	39.37	62.80	42.90	46.22
MLP		126.00	75.48	76.13	68.72	54.00	42.87	56.29

Correlation provides moderate accuracy and is relatively faster in training time but less efficient in prediction time (kNN: 698,364.41 μs). AE technique shows significant improvements as the number of features increases, making it a viable option for larger feature sets. The runtime results indicate that PCA and Chi-square are more suitable for resource-constrained IoT environments due to their balanced performance and moderate prediction times. AE, while accurate with more features, requires higher computational resources, making it less ideal for real-time applications.

We first investigate how performance changes with an increasing number of features for both feature selection and feature reduction methods. Fig. 16 illustrates the impact of increasing the number of features on various performance metrics for different feature reduction methods and models. The performance metrics driven by the feature selection method show significant improvement as the number of

features increases from 10 to 34. We observe that the overall performance metrics generally improve. For instance, the decision tree with chi-square (DT_CS_10) starts with lower performance metrics, particularly accuracy (71.53) and F1-score (30.08). In addition, as we move to models with more features like DT_PCA_22 and DT_CS_34, there is a noticeable improvement in these metrics, reaching up to an accuracy of 76.25 and an F1-score of 43.22. This trend suggests that adding more relevant features in feature selection allows the models to capture more relevant information, leading to enhanced classification performance. However, different from the trend of feature selection, the performance metrics driven by feature extraction are stable, and there is no significant performance increase for feature extraction for both PCA and AE extraction techniques.

Moreover, regarding the preferred models for each feature reduction method, several key observations can be made about the preferred

Table 21
Feature reduction techniques for multi-classification with 34 features.

Classifier	FR Time	Training Time (s)	Prediction Time (μs)	Accuracy (%)	Precision (%)	Recall (%)	F1 Score (%)	MCC (%)
Pearson Correlation								
DT	6.10	1.12	15.44	76.01	68.45	53.46	42.41	56.04
RF		10.39	837.31	69.69	39.94	22.38	18.32	35.89
kNN		0.72	54322.28	76.26	67.38	40.68	37.79	54.48
NB		0.99	156.85	32.71	32.67	48.52	24.22	27.71
MLP		175.13	129.95	76.00	68.51	53.58	42.58	56.06
Chi-square								
DT	2.12	1.20	13.75	76.01	68.42	53.82	42.77	56.10
RF		11.90	923.60	69.15	35.81	28.34	21.52	35.45
kNN		0.75	65102.88	76.40	67.98	40.92	38.03	54.77
NB		0.96	166.36	38.64	36.34	48.90	27.47	32.96
MLP		206.08	127.54	75.98	68.40	53.79	42.74	56.04
PCA								
DT	4.38	4.48	14.24	76.59	70.37	54.72	44.00	57.04
RF		32.83	1002.42	76.35	70.84	53.96	43.56	56.52
kNN		0.67	54812.86	68.39	48.92	41.51	34.53	48.20
NB		0.95	214.19	43.51	51.29	58.93	35.57	37.26
MLP		177.93	111.56	76.23	68.87	53.61	42.84	56.41
AE								
DT	467.75	7.15	11.90	59.42	35.43	33.21	20.62	36.01
RF		38.14	972.83	75.11	52.33	51.37	38.99	55.56
kNN		0.64	53990.23	55.98	51.83	35.95	23.84	34.62
NB		0.84	212.04	57.18	39.36	62.72	42.92	46.28
MLP		195.91	67.31	76.18	68.73	53.87	42.88	56.36

Table 22
Feature reduction techniques for multi-classification with 49 features.

Classifier	FR Time	Training Time (s)	Prediction Time (μs)	Accuracy (%)	Precision (%)	Recall (%)	F1 Score (%)	MCC (%)
Pearson Correlation								
DT	6.12	1.68	21.16	74.64	61.50	52.77	44.06	54.48
RF		14.75	905.59	69.74	37.28	22.63	18.81	36.04
kNN		0.77	60620.25	73.82	60.47	49.83	42.99	52.27
NB		1.14	234.02	31.87	29.60	42.21	21.05	25.98
MLP		226.03	187.57	76.15	68.63	53.85	42.86	56.30
Chi-square								
DT	2.18	1.72	19.11	74.56	61.91	53.95	44.96	54.86
RF		14.60	942.15	69.17	39.73	28.44	21.52	35.48
kNN		0.71	60879.82	74.00	59.84	49.39	42.92	52.39
NB		1.05	231.56	30.32	27.14	41.95	22.10	24.93
MLP		232.19	186.56	76.14	68.80	53.84	42.81	56.29
PCA								
DT	6.05	8.06	17.00	59.83	54.06	45.01	34.68	40.27
RF		49.85	1015.13	75.58	53.71	51.92	41.14	54.98
kNN		0.70	60084.51	58.25	52.69	41.81	30.34	38.47
NB		1.00	284.36	28.32	46.25	52.27	32.63	28.73
MLP		411.06	293.80	76.19	68.96	54.25	42.90	56.40
AE								
DT	495.97	9.01	12.73	54.83	50.64	31.02	21.98	29.61
RF		45.61	1001.74	75.39	61.81	52.31	39.94	56.15
kNN		0.65	60322.93	54.17	46.08	32.64	22.55	31.25
NB		0.91	412.78	56.92	39.30	62.47	42.58	45.83
MLP		131.68	82.16	76.22	68.91	53.62	42.83	56.41

models for each feature reduction method based on the figure. For Chi-square feature selection, the Decision Tree (DT) models show a steady improvement in performance metrics as more features are added, indicating that DT is a reliable choice for this feature reduction method. When using Autoencoder (AE) for feature extraction, the Random Forest (RF_AE_10) and Multi-Layer Perceptron (MLP_AE_49) models stand out with high performance metrics across all categories, highlighting AE's ability to enhance complex models. Principal Component Analysis (PCA) also shows significant performance improvements, especially with MLP_PCA_77 achieving high scores in Accuracy (76.28) and F1-score (42.95), suggesting that MLP combined with PCA is particularly effective for multi-class classification tasks. Overall, while each feature reduction method benefits from specific models, it is evident that incorporating more features and using advanced models like RF and MLP with AE and PCA can lead to superior performance in multi-class

classification.

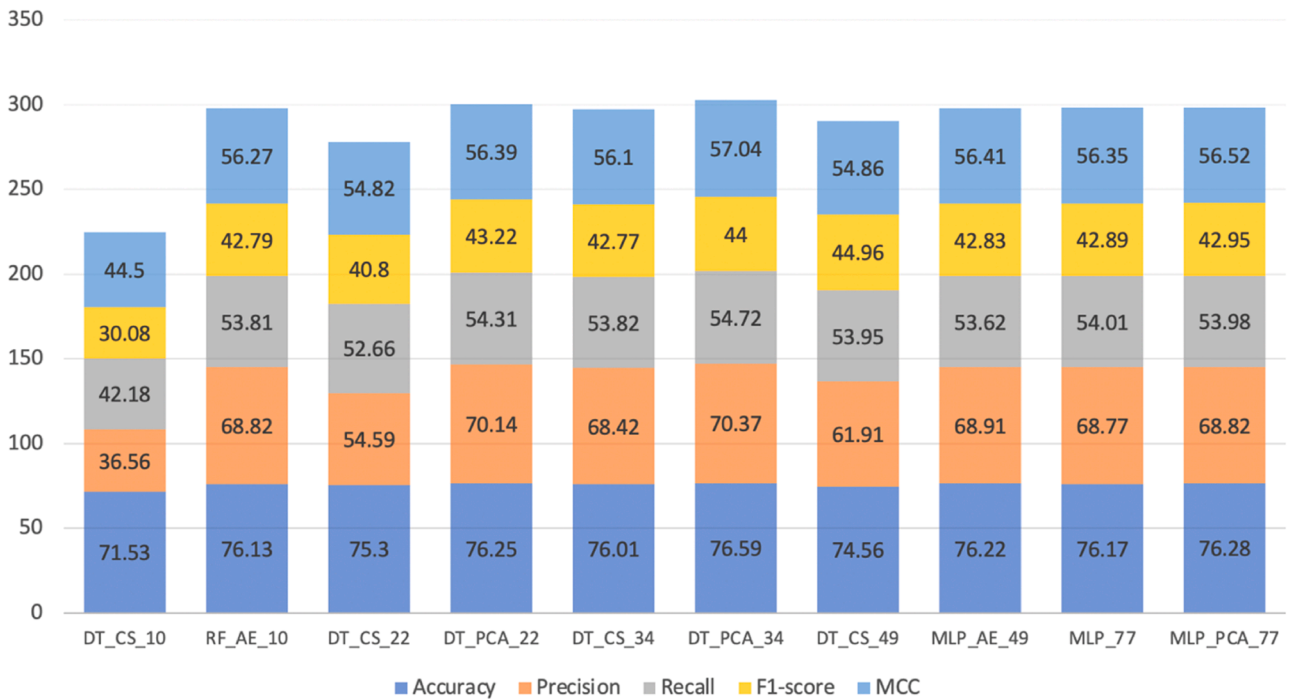
As for run time performance, The Fig. 17 illustrates the feature reduction (FR) times for different methods (FS_PC, FS_CS, FE_PCA, FE_AE) across varying numbers of features in multi-classification tasks. Compared with the Fig. 8 in binary classification, this pattern remains consistent regardless of the classification type. While both figures display similar trends, the multi-classification tasks tend to have slightly higher FR times for all methods, reflecting the added complexity of handling multiple classes. Overall, the comparison underscores that FE_AE remains the most resource-intensive method across both classification types, whereas FS methods maintain their advantage in computational efficiency, making them more suitable for applications with limited computational resources.

The runtime of model training and test time is evaluated as presented from Tables 19 to 23. Compared with the models in binary classification,

Table 23

Feature reduction techniques for multi-classification with 55 features.

Classifier	FR Time	Training Time (s)	Prediction Time (μ s)	Accuracy (%)	Precision (%)	Recall (%)	F1 Score (%)	MCC (%)
Full features without FS applied								
DT	N/A	3.09	25.81	38.06	21.30	14.47	12.10	06.20
RF		17.39	931.25	66.67	21.37	23.20	14.35	27.87
kNN		0.85	73337.88	55.17	49.33	36.72	25.20	35.60
NB		1.30	361.21	20.98	27.37	41.44	19.88	19.81
MLP		379.90	274.88	76.17	68.77	54.01	42.89	56.35
PCA								
DT	7.27	9.67	17.83	54.01	56.48	35.08	26.02	31.56
RF		52.42	1051.87	75.52	53.56	51.78	40.88	54.83
kNN		0.69	63780.04	56.30	54.49	36.47	25.42	34.27
NB		1.05	319.67	23.33	44.82	45.11	26.78	23.56
MLP		638.15	268.42	76.28	68.82	53.98	42.95	56.52
AE								
DT	511.47	10.79	13.46	57.48	46.91	41.48	31.64	37.15
RF		42.71	1022.50	74.78	54.86	49.74	38.61	54.28
kNN		0.64	63153.82	52.69	45.24	34.84	23.88	31.02
NB		0.92	379.76	57.32	49.44	63.10	43.56	46.60
MLP		251.41	70.38	76.16	68.40	53.88	42.89	56.34

**Fig. 16.** The performance comparison of FS and FE models for multi-classification.

the same pattern is presented in multi-classification on runtime of the models. Simpler models like Decision Tree (DT) and Naive Bayes (NB) consistently show shorter training and prediction times compared to more complex models like Multi-Layer Perceptron (MLP) and Random Forest (RF). For instance, in multi-classification with 10 features, DT training time is 1.61 seconds, while MLP is 221.44 seconds. Similarly, in binary classification with 77 features, DT training time is 3.67 seconds, whereas MLP takes 502.21 seconds. k-Nearest Neighbors (kNN) shows the highest prediction times across both tasks. These patterns highlight the computational efficiency of simpler models, making them more suitable for time-sensitive applications.

As for the run time of model training and test among the feature reduction methods, feature reduction methods exhibit distinct differences in runtime efficiency. Chi-square and Pearson Correlation consistently show shorter feature reduction (FR) times and efficient training times across both multi-classification and binary classification tasks. For example, Chi-square with 22 features and DT in multi-

classification shows an FR time of 2.20 seconds. In contrast, Autoencoder (AE) exhibits significantly higher FR times, often exceeding 400 seconds, impacting overall training time. Principal Component Analysis (PCA) has moderate FR and training times. While AE enhances classification performance, Chi-square and Pearson Correlation offer a more efficient alternative for resource-constrained environments, balancing computational efficiency and performance.

The attack detection performance among the feature reduction techniques, we evaluate and compare the class-wise f1-score of various attack types, using the best model of the feature reduction techniques respectively in each feature setting. The Fig. 18 illustrates the class-wise F1-scores for various attack types and the normal class across different feature reduction techniques and models. Each model demonstrates varying levels of effectiveness in detecting specific attack types. For instance, the Random Forest with Autoencoder (RF_AE_10) and Multi-Layer Perceptron with PCA (MLP_PCA_77) consistently achieve high F1-scores across multiple attack categories, including DDoS, scanning,

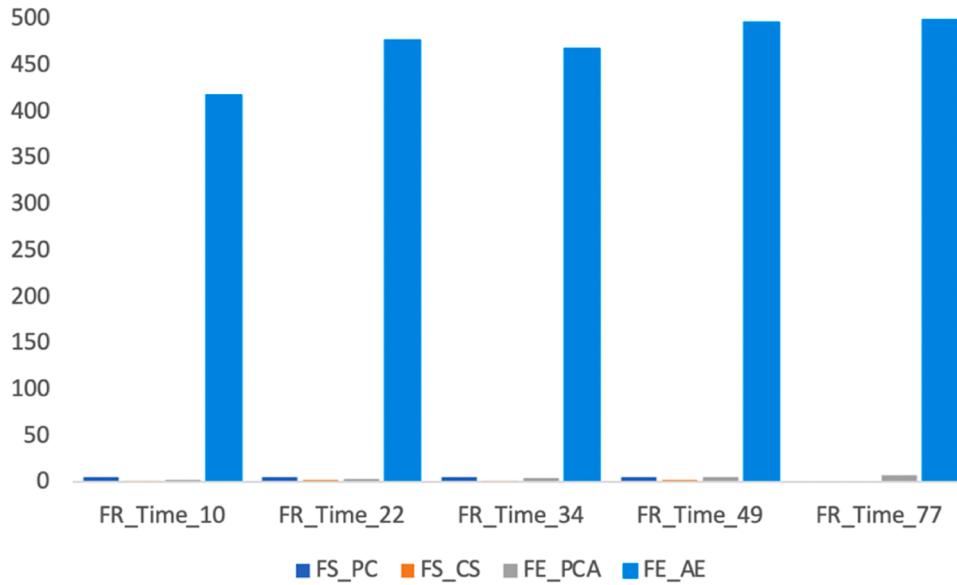


Fig. 17. The run time of FS and FE with reduced features in binary classification.

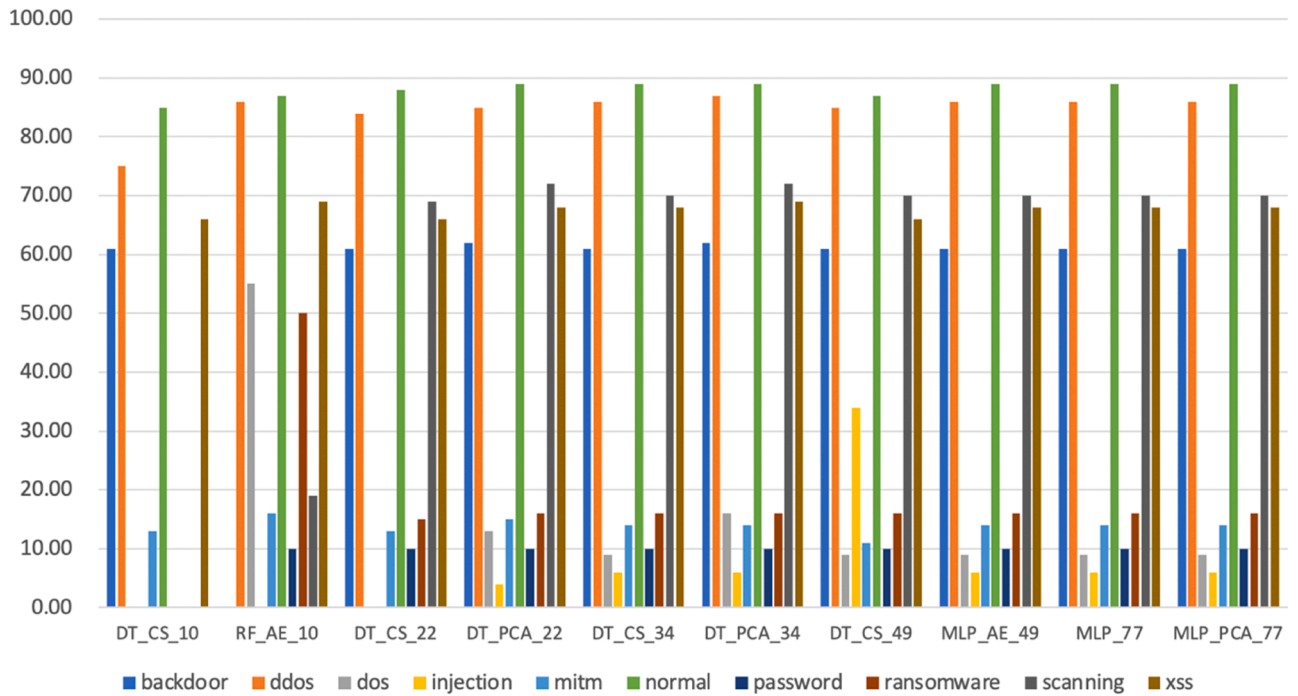


Fig. 18. The class-wise F1-score among the feature reduction techniques for reduced features.

and normal traffic, indicating their robust detection capabilities. In contrast, models like Decision Tree with Chi-square (DT_CS_10) show more variability in performance, excelling in some categories like normal traffic but underperforming in others such as injection and backdoor attacks. Overall, feature reduction techniques such as PCA and Autoencoder significantly enhance the models' ability to accurately detect a wide range of attack types, providing a comprehensive security solution.

Moreover, The Fig. 18 illustrates the performance differences between feature extraction (FE) and feature selection (FS) methods across multiple attack types. Feature extraction methods, such as PCA and Autoencoders (AE), consistently achieve higher F1-scores across a broader range of attacks, including complex and diverse types like backdoor, DDoS, injection, and ransomware. This superior performance

can be attributed to FE techniques' ability to transform high-dimensional data into a more informative and discriminative lower-dimensional space, effectively capturing the essential features needed for accurate detection. In contrast, FS methods, while efficient in reducing the feature set, may miss out on capturing intricate patterns and relationships within the data, leading to lower detection rates for certain attack types. The comprehensive coverage and higher accuracy of FE methods make them particularly effective in detecting a wider variety of cyber threats, ensuring robust performance in network intrusion detection systems (NIDS) for IoT environments.

As for the low performance metrics in certain attack classes, despite the overall effectiveness of feature reduction techniques, certain attack classes, such as backdoor and injection attacks, often show lower F1-scores compared to others like DDoS and scanning attacks. There are

several reasons for this disparity. First, some attack types, such as injection attacks, may exhibit more subtle and complex patterns that are harder to detect and distinguish from normal traffic. These attacks might involve smaller variations in data that are not as easily captured by the models, even with advanced feature reduction techniques. Second, the imbalanced nature of the dataset can contribute to lower performance metrics. If certain attack classes are underrepresented in the training data, the models may not learn their characteristics effectively, leading to poorer detection rates. Finally, the variability and adaptability of certain attacks, such as backdoor attacks, can pose a significant challenge, as these attacks can evolve and disguise themselves, making it difficult for the models to maintain high detection accuracy. Addressing these issues requires not only advanced feature reduction techniques but also strategies like data augmentation, anomaly detection frameworks, and continual learning to improve the robustness of the models against diverse and evolving attack patterns.

In summary, when evaluating binary and multiclass classification within the NIDS utilizing the Network TON-IoT dataset, FE emerges as not only offering superior classification performance but also reduced feature reduction time compared to its FS counterpart, particularly as the number of reduced features increases. The advantage of FE is notably higher and more consistent, especially with smaller quantities of features, such as 10 and 22. However, FS generally demonstrates shorter model training and inference times than feature extraction, which is significant to lightweight model design of NIDS in IoT network.

5.4. Multi classification for BoT-IoT

Here, we use Tables 24–27 to examine the multi-class classification performance and computing time analysis of both the FS and FE approaches. Applying the same criteria as the binary classification described in subsection 5.1, the tables emphasize the best values in bold and the superior results for both feature selection and extraction in red.

For the dataset with 3 features, the Decision Tree classifier shows a balanced performance with an accuracy of 68.90 %, precision of 66.14 %, and recall of 73.95 %. The Random Forest classifier, however, shows lower accuracy (54.73 %) and precision (57.14 %). The k-Nearest Neighbors (kNN) classifier performs similarly to the Decision Tree with an accuracy of 68.87 % but requires a significantly longer prediction time (5147.75 μ s). Naive Bayes shows poor performance with an accuracy of 29.35 %, while the Multi-Layer Perceptron (MLP) classifier

shows a modest accuracy of 54.66 %.

When increasing the number of features to 16, the Decision Tree's performance improves significantly with an accuracy of 95.78 %, precision of 94.60 %, and recall of 95.36 %. The kNN classifier also performs well with an accuracy of 95.73 %, but with a substantial prediction time of 30417.38 μ s. The Random Forest and MLP classifiers show significant improvements, with the Random Forest achieving an accuracy of 79.58 % and the MLP achieving 83.45 %. Naive Bayes continues to underperform with an accuracy of 42.82 %.

With 33 features, the Decision Tree achieves an impressive accuracy of 98.81 %, while kNN and MLP classifiers also show high performance with accuracies of 98.60 % and 96.74 %, respectively. The Random Forest classifier achieves an accuracy of 91.62 %, while Naive Bayes shows slight improvement with an accuracy of 46.54 %.

With 40 features, the Decision Tree classifier achieves almost perfect accuracy at 99.91 %, and the kNN classifier also performs extremely well with an accuracy of 99.41 %. The Random Forest and MLP classifiers maintain high accuracies of 91.24 % and 97.73 %, respectively. Naive Bayes still struggles with an accuracy of 47.31 %.

With the full 55 features, the Decision Tree reaches an accuracy of 99.97 %, the kNN classifier performs very well with 99.86 %, and the MLP classifier achieves 99.65 %. The Random Forest classifier has an accuracy of 98.19 %, while Naive Bayes, although improved, remains less accurate with 76.35 %.

Overall, increasing the number of features generally improves the performance of classifiers across the board, with Decision Tree, kNN, and MLP classifiers showing the highest accuracies as the number of features increases. However, this improvement in accuracy often comes with an increase in prediction time, particularly for kNN, which demonstrates significantly longer prediction times compared to other classifiers. Naive Bayes consistently shows lower performance in terms of accuracy, precision, and recall across different feature sets, suggesting it may not be the best choice for this dataset.

In resource-constrained IoT environments, the trade-off between accuracy and computational efficiency is critical. While classifiers like kNN offer high accuracy, their substantial prediction time may not be suitable for real-time applications. Conversely, Decision Tree and Random Forest classifiers, while providing slightly lower accuracy, offer a more balanced trade-off with reasonable prediction times. The Multi-Layer Perceptron classifier shows a good balance of high accuracy and moderate prediction time, making it a viable option for such

Table 24
Feature reduction techniques for multi-classification with 3 features.

Classifier	FR Time (s)	Training Time (s)	Prediction Time (μ s)	Accuracy (%)	Precision (%)	Recall (%)	F1 Score (%)	MCC (%)
Pearson Correlation								
DT	2.62	2.16	42.47	68.90	66.14	73.95	68.71	40.31
RF		17.05	578.71	54.73	57.14	28.59	28.08	14.55
kNN		0.80	5147.75	68.87	68.63	72.46	69.70	40.39
NB		0.65	12.19	29.35	26.60	42.50	12.21	0.32
MLP		367.48	115.22	54.66	38.09	42.64	34.05	10.49
Chi-square								
DT	1.69	1.11	12.23	66.96	67.88	83.89	72.10	37.57
RF		13.15	449.21	57.20	87.12	54.39	56.81	21.98
kNN		0.75	15715.46	60.86	71.66	71.70	69.85	24.65
NB		0.63	9.46	52.92	60.95	46.89	43.65	7.89
MLP		454.50	97.23	55.87	61.04	54.12	54.88	14.08
PCA								
DT	1.95	2.73	14.89	99.08	93.31	95.08	94.13	98.23
RF		30.58	613.66	78.32	67.18	61.34	63.33	58.49
kNN		0.84	3839.15	98.87	94.75	96.56	95.59	97.83
NB		0.64	24.42	54.11	31.93	51.40	35.48	18.25
MLP		319.91	94.07	79.09	71.01	85.51	73.62	60.03
AE								
DT	380.00	2.84	14.25	99.22	96.99	94.28	95.55	98.50
RF		29.78	619.62	80.77	68.18	63.60	65.47	64.47
kNN		0.81	3627.94	99.03	93.08	95.19	94.07	98.14
NB		0.62	23.25	59.82	37.07	65.57	41.70	29.42
MLP		294.34	75.01	88.57	82.24	88.97	84.79	78.13

Table 25
Feature reduction techniques for multi-classification with 16 features.

Classifier	FR Time (s)	Training Time (s)	Prediction Time (μs)	Accuracy (%)	Precision (%)	Recall (%)	F1 Score (%)	MCC (%)
Pearson Correlation								
DT	2.55	2.09	17.07	95.78	94.60	95.36	94.93	91.94
RF		18.93	583.03	79.58	83.32	47.63	55.60	61.20
kNN		0.57	30417.38	95.73	87.93	93.18	90.35	91.84
NB		0.68	34.91	42.82	49.64	51.22	19.95	-2.92
MLP		594.97	110.12	83.45	82.11	79.48	78.14	68.73
Chi-square								
DT	1.20	2.29	8.22	99.59	98.65	99.56	99.09	99.22
RF		17.90	528.56	85.57	94.84	91.57	92.76	73.47
kNN		1.16	297916.52	98.44	96.79	96.29	96.54	97.01
NB		0.42	11.92	41.51	53.25	62.99	44.63	-0.26
MLP		265.27	55.16	87.41	91.36	91.33	91.33	75.92
PCA								
DT	2.18	15.64	11.38	99.86	98.37	98.29	98.33	99.73
RF		77.89	628.90	92.08	96.81	88.08	92.01	84.83
kNN		1.92	4667.67	99.72	99.55	97.14	98.26	99.46
NB		0.70	51.64	78.36	61.02	76.63	62.99	60.37
MLP		534.39	63.68	98.36	99.02	97.64	98.30	96.88
AE								
DT	411.67	16.09	9.82	99.93	95.88	96.38	96.11	99.87
RF		77.71	601.33	92.12	96.74	65.84	73.72	84.78
kNN		2.03	4535.78	99.94	98.45	98.61	98.53	99.89
NB		0.67	54.27	62.85	53.74	78.11	53.96	37.93
MLP		484.29	90.36	98.89	96.85	97.49	97.15	97.90

Table 26
Feature reduction techniques for multi-classification with 33 features.

Classifier	FR Time (s)	Training Time (s)	Prediction Time (μs)	Accuracy (%)	Precision (%)	Recall (%)	F1 Score (%)	MCC (%)
Pearson Correlation								
DT	2.56	3.84	14.52	98.81	99.48	99.25	99.37	97.72
RF		24.32	620.94	91.62	96.03	90.41	93.00	83.92
kNN		0.63	40388.71	98.60	98.85	96.14	97.41	97.33
NB		0.92	90.95	46.54	48.16	73.94	39.11	16.89
MLP		745.09	224.69	96.74	94.23	94.03	94.11	93.84
Chi-square								
DT	1.11	4.41	7.28	99.77	99.89	99.66	99.77	99.57
RF		25.58	493.83	93.28	97.39	94.88	96.06	87.28
kNN		0.43	25458.92	99.73	97.52	98.56	98.02	99.49
NB		0.54	53.48	74.95	72.14	86.24	69.31	57.00
MLP		347.21	48.34	96.39	98.57	96.95	97.72	93.16
PCA								
DT	3.69	37.96	12.68	99.93	96.61	97.65	97.11	99.86
RF		107.39	632.01	97.69	99.02	90.27	94.25	95.57
kNN		0.58	36113.42	99.86	99.62	96.75	98.10	99.73
NB		0.80	89.36	59.79	56.33	75.83	54.66	38.44
MLP		246.00	113.41	99.65	98.50	97.05	97.75	99.34
AE								
DT	378.83	37.47	11.02	99.91	97.38	96.35	96.85	99.82
RF		110.11	619.61	94.36	97.65	85.19	90.52	89.33
kNN		0.54	31610.86	99.92	98.56	97.12	97.82	99.85
NB		0.68	82.67	62.02	49.13	78.68	49.64	40.30
MLP		482.34	50.05	98.87	98.11	96.72	97.39	97.84

environments, provided the computational resources can support its training time.

We first investigate how performance changes with an increasing number of features for both feature selection and feature reduction methods. Fig. 19 illustrates the performance metrics of different feature selection (FS) and feature extraction (FE) models in multi-classification tasks as more features are added. The performance metrics of all the feature schemes of various feature reduction techniques present a high score and are consistent with the increasing number of selected or extracted features, except for the case of DT_PC_3, which is caused by less relevant predictors selected for classification models; however, with more relevant features selected, the performance increases significantly. For example, the Decision Tree with Principal Component Analysis (DT_PCA) model shows a substantial improvement in metrics as the features increase from 3 to 55. Initially, with 3 features, the accuracy

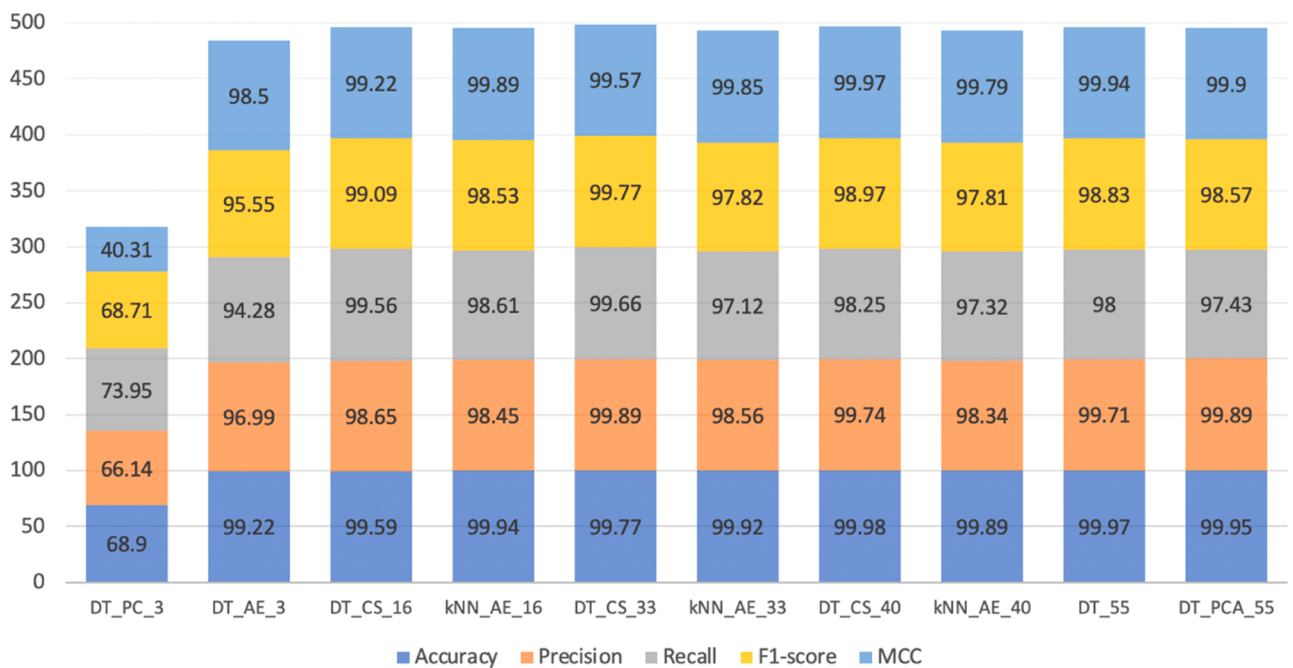
was 40.31 %, but it rose significantly to 99.22 % with 16 features. In contrast, the performance with a small number of features (3 feature sets) driven by feature extraction using AE achieves a competitive performance score compared to those with a larger number of features (16, 33, 40, and 55).

Moreover, regarding the preferred models for each feature reduction method, several key observations can be made about the preferred models for each feature reduction method based on the figure. The Autoencoder (AE) consistently delivers superior performance across various feature settings. For instance, k-Nearest Neighbors with AE (kNN_AE_33) and Decision Tree with AE (DT_AE_3) achieve high metrics, showing the effectiveness of AE in enhancing model performance. Similarly, Principal Component Analysis (PCA) also demonstrates strong results, particularly when combined with Decision Trees (DT_PCA_55). Chi-square (CS) is another effective method, as seen in models like

Table 27

Feature reduction techniques for multi-classification with 40 features.

Classifier	FR Time (s)	Training Time (s)	Prediction Time (μ s)	Accuracy (%)	Precision (%)	Recall (%)	F1 Score (%)	MCC (%)
Pearson Correlation								
DT	2.60	5.61	14.11	99.91	98.78	99.45	99.09	99.82
RF		31.31	596.61	91.24	96.38	92.83	94.47	83.45
kNN		0.64	40525.01	99.41	99.43	96.49	97.87	98.86
NB		0.86	83.56	47.31	58.04	74.95	45.90	20.65
MLP		522.48	107.04	97.73	97.51	94.92	96.13	95.72
Chi-square								
DT	1.14	6.23	6.83	99.98	99.74	98.25	98.97	99.97
RF		32.71	497.57	98.39	99.32	97.79	98.53	96.92
kNN		0.42	29235.08	99.88	97.55	98.44	97.97	99.78
NB		0.53	38.17	76.53	72.37	86.29	70.21	59.74
MLP		301.92	45.32	99.43	99.70	98.20	98.93	98.91
PCA								
DT	3.15	39.57	9.98	99.96	99.71	97.44	98.49	99.93
RF		113.76	512.79	97.84	99.08	93.02	95.87	95.87
kNN		0.39	23743.14	99.85	97.06	96.78	96.92	99.72
NB		0.56	68.10	63.66	56.46	76.73	56.80	42.99
MLP		192.85	59.42	99.76	97.13	96.90	97.01	99.55
AE								
DT	375.65	42.29	11.28	99.93	99.90	98.14	98.99	99.86
RF		126.18	661.88	94.05	97.64	77.83	83.06	88.64
kNN		0.55	38599.82	99.89	98.34	97.32	97.81	99.79
NB		0.70	100.38	66.83	52.49	82.98	57.63	41.09
MLP		432.42	64.55	98.83	93.75	96.43	94.97	97.76

**Fig. 19.** The performance comparison of FS and FE models for multi-classification.

DT_CS_16 and DT_CS_40, which maintain high performance metrics across different feature counts. Overall, AE and PCA emerge as the most preferred feature reduction methods due to their ability to consistently enhance model accuracy and other performance metrics, demonstrating their robustness in handling multi-classification tasks.

As for run time performance, The Fig. 20 illustrates the feature reduction (FR) times for different methods (FS_PC, FS_CS, FE_PCA, FE_AE) across varying numbers of features in multi-classification tasks using BoT-IoT dataset. Compared with the Fig. 8 in binary classification of TON-IoT and Fig. 17 in multi-classification of TON-IoT, this pattern also remains consistent regardless of the classification type. It's clear that FE_AE remains the most resource-intensive method across both classification types.

The runtime of model training and test time is evaluated as presented from Tables 24 to 28. The results reveal notable variations in training and prediction times across different classifiers and feature reduction methods. For example, in multi-classification with 3 features, training times range from 0.62 seconds for Naive Bayes (NB) with AE to 380 seconds for Decision Tree (DT) with AE. Prediction times also show substantial differences, with k-Nearest Neighbors (kNN) taking up to 5147.75 microseconds using Pearson Correlation, while Decision Trees using Chi-square take only 12.23 microseconds. As the number of features increases to 55, training times for models like Multilayer Perceptron (MLP) with full features extend to 678.82 seconds, whereas simpler models like Naive Bayes with PCA remain below 1 second. These discrepancies highlight the trade-offs between model complexity, feature

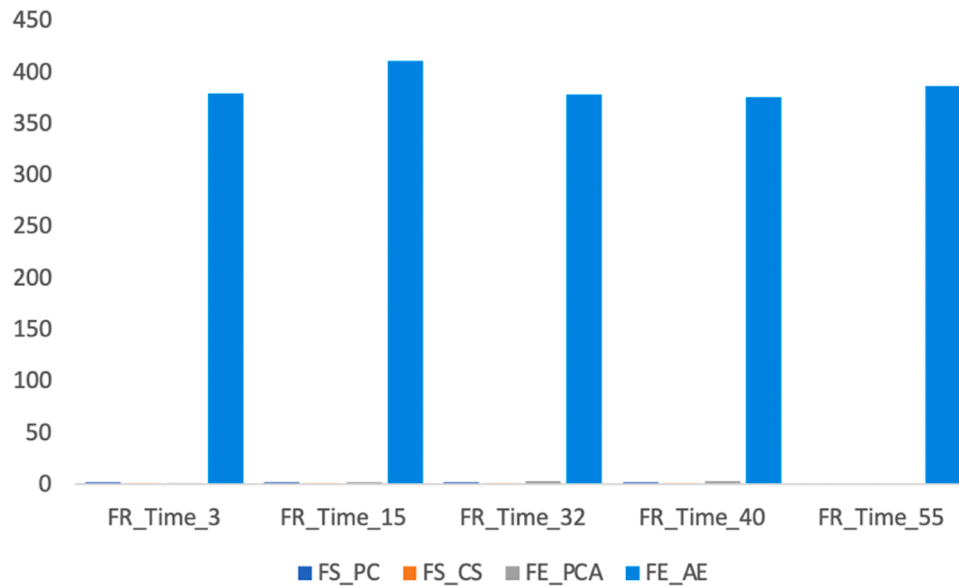


Fig. 20. The run time of FS and FE with reduced features in multi-classification.

Table 28

Feature reduction techniques for multi-classification with 55 features.

Classifier	FR Time (s)	Training Time (s)	Prediction Time (μ s)	Accuracy (%)	Precision (%)	Recall (%)	F1 Score (%)	MCC (%)
Full features without FS applied								
DT	N/A	9.73	15.21	99.97	99.71	98.00	98.83	99.94
RF		39.37	602.57	98.19	99.23	96.25	97.70	96.54
kNN		0.69	42688.11	99.86	99.62	96.75	98.10	99.73
NB		0.95	109.95	76.35	68.22	86.76	69.19	59.36
MLP		678.82	133.50	99.65	97.49	98.53	97.99	99.34
PCA								
DT	0.77	43.67	9.74	99.95	99.89	97.43	98.57	99.90
RF		113.06	532.08	97.70	99.02	90.38	94.33	95.59
kNN		0.41	28179.18	99.86	97.07	96.78	96.92	99.72
NB		0.63	86.91	63.66	56.74	77.03	57.05	43.08
MLP		229.36	54.00	99.68	97.26	98.32	97.77	99.39
AE								
DT	386.63	57.78	11.42	99.93	96.38	96.14	96.22	99.86
RF		141.58	631.85	95.69	98.20	88.18	92.70	91.73
kNN		0.55	37391.68	99.91	99.00	97.10	98.42	99.82
NB		0.73	129.72	71.60	42.71	85.09	48.43	51.08
MLP		393.57	64.42	99.52	99.71	98.03	98.85	99.08

reduction technique, and computational efficiency.

As for the run time of model training and test among the feature reduction methods, feature reduction methods exhibit distinct differences in runtime efficiency, it is evident that AE generally incurs higher training times compared to other methods. For instance, AE with DT in the 3-feature setting takes 380 seconds, while Pearson Correlation takes only 2.62 seconds. In contrast, methods like Chi-square and PCA offer more balanced performance, with relatively lower training times and competitive prediction times. For example, PCA with DT in the 33-feature setting achieves a high accuracy of 99.93 % with a training time of 37.96 seconds and a prediction time of 12.68 microseconds. These patterns suggest that while AE can enhance model performance, it comes at the cost of increased computational load. On the other hand, PCA and Chi-square methods strike a better balance between performance metrics and computational efficiency, making them more practical for real-world applications where runtime is a critical factor.

The attack detection performance among the feature reduction techniques, we evaluate and compare the class-wise f1-score of various attack types, using the best model of the feature reduction techniques respectively in each feature setting. The Fig. 18 illustrates that the results of the minimal number of features (3 feature sets) driven by feature

extraction (DT_AE_3) is significantly better than those of feature selection (DT_PC_3), which is caused lack of enough relevant features selected by the minimal feature selection technique. In addition, it is evident that the techniques involving 16, 33, 40 and 55 features consistently achieve high F1-scores across the attack types, including DDoS, DoS, normal, and Reconnaissance. However, the Theft attack is completely classified only in DT_CS_33 feature scheme, while other feature schemes achieve more than 90 % except for the minimal feature scheme DT_PC_3. This can be due to the reasons, such as the complexity and subtlety of these attack patterns, which make them harder to distinguish from normal behavior. Additionally, the imbalance in the dataset, where some attack types are underrepresented, can lead to models that are less effective at detecting these specific attacks.

Moreover, the Fig. 21 shows their effectiveness across several attack types, including DDoS, DOS, reconnaissance, and theft. Feature extraction (FE) methods like Principal Component Analysis (PCA) and Autoencoders (AE) demonstrate consistently high F1-scores across these attack types, highlighting their superior ability to detect diverse and complex cyber threats. The same as the reasoning of the results of multiclassification on TON-IoT, this advantage stems from FE techniques' capability to transform the original high-dimensional feature

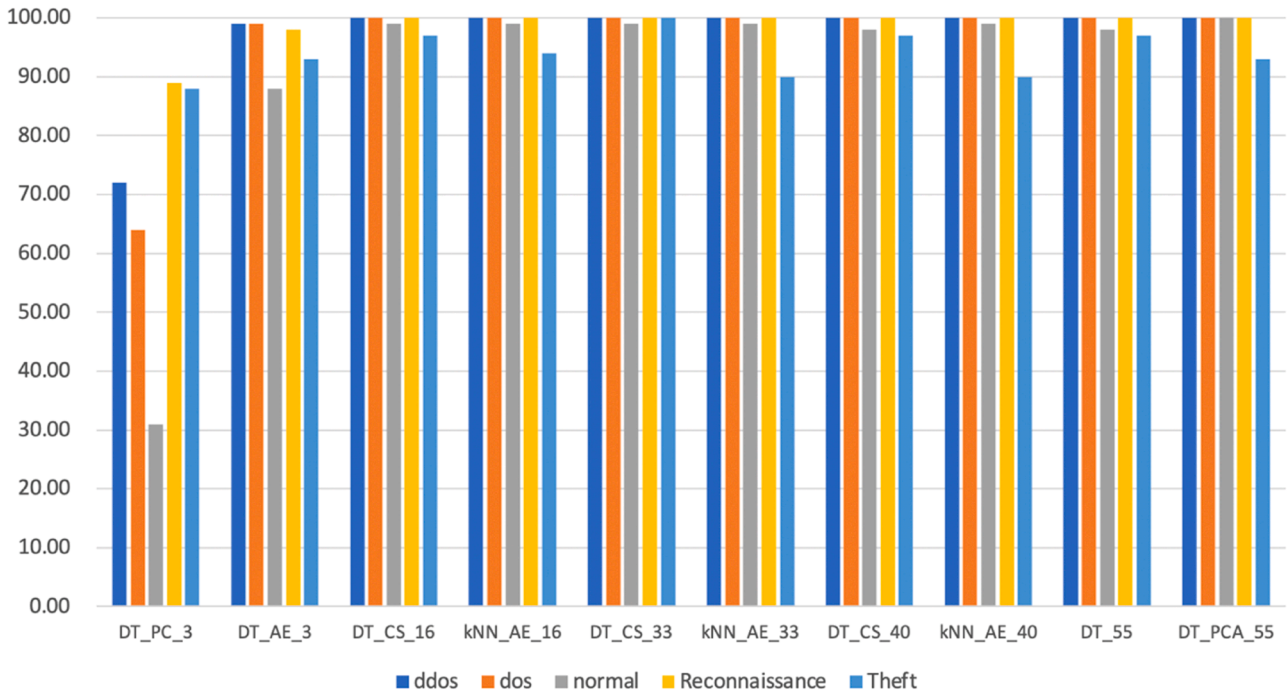


Fig. 21. The class-wise F1-score among the feature reduction techniques for reduced features.

space into a lower-dimensional representation that retains critical information, enhancing the model's ability to distinguish between different types of attacks. Conversely, feature selection (FS) methods, while effective in reducing feature set size, may not capture the same level of intricate patterns and dependencies present in the data, leading to slightly lower detection rates for certain attack types. Therefore, FE methods provide a more comprehensive and robust approach for identifying a wide range of cyber threats in network intrusion detection systems (NIDS), especially in IoT environments where attack diversity and complexity are significant concerns.

In summary, the analysis of multi-class classification performance using the BoT-IoT dataset, as presented as above, reveals significant insights into the effectiveness of various feature selection (FS) and feature extraction (FE) methods. Initially, with only 3 features, models such as Decision Tree (DT) and k-Nearest Neighbors (kNN) exhibit moderate performance, while Naive Bayes (NB) performs poorly. As the number of features increases, substantial improvements are observed, particularly in DT and kNN classifiers, which achieve near-perfect accuracies with 33 to 55 features. The Autoencoder (AE) and Principal Component Analysis (PCA) methods consistently enhance model performance, with AE showing a marked ability to improve accuracy even with a smaller number of features. However, these improvements often come with increased training and prediction times, especially for kNN, highlighting a trade-off between accuracy and computational efficiency. Among all methods, AE emerges as the most resource-intensive, while PCA and Chi-square offer a more balanced performance, making them practical for real-time applications in resource-constrained environments. Overall, the study underscores the importance of selecting appropriate feature reduction techniques to achieve high accuracy and computational efficiency in multi-class classification tasks.

5.5. Results summarized

The study systematically evaluates feature selection (FS) and feature extraction (FE) techniques for binary and multi-class classification using TON-IoT and BoT-IoT datasets. It highlights that FE methods, such as PCA and AE, typically offer higher classification accuracy and robustness across various classifiers compared to FS methods. Random Forest

(RF) emerges as a consistently high-performing classifier, particularly when used with AE, achieving the best accuracy and F1 scores, although it requires more computational resources. Decision Trees (DT) are optimal for smaller feature sets, whereas Multi-Layer Perceptron (MLP) excels with larger sets. Chi-square is identified as the most efficient FS technique, providing a balance of performance and computational efficiency, while PCA is noted for balancing performance and runtime better than AE. The findings indicate that detection performance increases with more informative features and that simpler datasets benefit more from FE methods. The study also notes that FE methods are less sensitive to changes in the number of features, offering stable performance, whereas FS methods show significant improvement with more informative features. Overall, FE methods show greater potential for improving performance, particularly with small feature sets, but come with higher computational costs particularly for deep learning-based AE extraction technique, making FS methods preferable for resource-constrained environments for specific scenarios. The main findings are presented in Table 29, in order to underscore the importance of selecting appropriate feature reduction techniques to optimize detection performance and computational efficiency in network intrusion detection systems (NIDS) for IoT environment.

6. Conclusions and future works

We conducted a thorough comparison between two dimensionality reduction methods, feature selection methods including Pearson correlation and Chi-square techniques, and feature extraction methods including PCA and AE, with five classic machine learning models including DT, RF, NB, kNN, and MLP as the classifiers, using the contemporary and heterogenous IoT-specific TON-IoT and BoT-IoT datasets for both binary and multi-classifications for NIDS. Firstly, our extensive analysis revealed that FE methods, such as PCA and AE, generally offer higher accuracy and robustness across various classifiers compared to FS methods. Among classifiers, Random Forest (RF) consistently performs well, particularly when paired with AE, achieving superior accuracy and F1 scores despite higher computational requirements. Decision Trees (DT) are most effective with smaller feature sets, while Multi-Layer Perceptron (MLP) excels with larger sets. Chi-

Table 29

A comparison between FE and FS in various feature scheme scenarios.

No.	Main Findings	FS	FE
1	Higher accuracy when no. of features is extremely limited, such as 3, 8, 10 feature sets.		✓
2	The same or higher accuracy with the number of feature sets increasing till full number of features.		✓
3	The performance metrics can benefit more from less complex dataset (BoT-IoT with 296480/55 instances/features is less complex than TON-IoT with 462262/77 instances/features)	✓	
4	Chi-square achieves better performance scores with runtime efficiency than Pearson correlation for feature selection	✓	
5	PCA achieves better than AE on balance between performance and runtime efficiency		✓
6	Lower feature reduction time (CS is better than PCA with increasing number of feature sets)	✓	
7	Lower model training time	✓	
8	Lower model prediction time	✓	
9	kNN and MLP can be the significant classifiers considering performance		✓
10	DT is the most efficient classifier with competitive performance scores	✓	✓
11	RF is the most favorite classifier considering performance with moderate efficiency	✓	✓
12	Less sensitive to the number of selected/extracted features		✓
13	Less sensitive to various machine learning models		✓
14	Detection performance degrades when number of features is reaching full number of features		✓
15	Detection performance increase when more informative features added	✓	
16	Detect more diverse attack types when using the same classifier		✓
17	Higher detection rate in detecting DoS, DDoS, Injection, Password, Ransomware, Scanning and XSS attack among all feature schemes		✓
18	Higher detection rate in detecting Backdoor, Reconnaissance, Theft attack among all feature schemes	✓	
19	More potential to improve performance when the number of features is small	✓	
20	More potential to improve performance when the number of features is large		✓

square (CS) emerges as the most efficient FS technique, balancing performance and computational efficiency, whereas PCA outperforms AE in terms of runtime efficiency. Moreover, the study reveals that detection performance improves with more informative features and that simpler datasets benefit more from FS methods, while FE methods favor more complex datasets compared with their counterparts. FE techniques show less sensitivity to the number of features and offer stable performance, whereas FS techniques exhibit significant performance improvements with additional informative features. However, FE methods come with higher computational costs, particularly for deep learning-based AE extraction techniques, making FS methods more suitable for resource-constrained environments in specific scenarios. Lastly, FE methods exhibit the capability to detect more diverse attack types than FS methods, particularly in cases of a small number of feature schemes. The findings are significant to both academia and industry as they provide a detailed comparison of dimensionality reduction methods, highlighting the superior accuracy and robustness of feature extraction techniques, particularly in complex IoT environments, while also addressing the practical computational constraints faced in real-world NIDS applications.

Our study has several limitations that should be acknowledged. Firstly, the evaluation of feature reduction techniques was limited to specific methods: feature selection methods included Pearson correlation and Chi-square, while feature extraction methods comprised PCA and Autoencoders (AE). Secondly, the datasets used, TON-IoT and BoT-IoT, included a limited range of attack types, which may not fully represent the diversity of real-world cyber threats. Thirdly, the study only applied five classic machine learning classifiers (Decision Tree, Random Forest, Naive Bayes, k-Nearest Neighbors, and Multi-Layer Perceptron), potentially overlooking the capabilities of other advanced

classifiers. Additionally, the number of features was determined based on Pearson correlation within the proposed 3-phase pipeline, which might not cover the full spectrum of feature selection possibilities. Lastly, the data preprocessing and class balancing methods employed in the study may have impacted the results, suggesting that alternative techniques may yield different outcomes.

For future work, we propose several key directions to enhance the effectiveness and applicability of NIDS in IoT environments. Firstly, reducing the computational cost of FE techniques, particularly AE, is crucial for real-time applications in resource-constrained settings. Exploring hybrid approaches that combine FS and FE methods could leverage the strengths of both, improving overall performance and efficiency. Additionally, integrating more advanced machine learning and deep learning classifiers may further enhance detection accuracy and robustness. Extensive real-world testing and validation are necessary to ensure these methods can handle diverse and evolving IoT network traffic and cyber threats. Expanding the datasets to include a broader range of attack types, including emerging and sophisticated vectors, will ensure comprehensive detection capabilities. Finally, developing adaptive NIDS that can dynamically adjust feature sets and classifiers based on the current network context and threat landscape will help maintain optimal performance and efficiency.

CRedit authorship contribution statement

Jing Li: Conceptualization, Investigation, Methodology, Software, Writing – original draft, Writing – review & editing. **Hewan Chen:** Funding acquisition, Formal analysis, Visualization, Validation. **Mohd Othman Shahizan:** Supervision, Project administration. **Lizawati Mi Yusuf:** Resources, Data curation.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

Data will be made available on request.

References

- Abdulhammed, R., Faezipour, M., Musafer, H., & Abuzneid, A. (2019). Efficient network intrusion detection using PCA-based dimensionality reduction of features. In *2019 International Symposium on Networks, Computers and Communications (ISNCC)* (pp. 1–6). Istanbul, Turkey: IEEE. <https://doi.org/10.1109/ISNCC.2019.8909140>. Jun.
- Aburomman, A. A., & Ibne Reaz, M. B. (2016). A novel SVM-kNN-PSO ensemble method for intrusion detection system. *Applied Soft Computing*, 38, 360–372. <https://doi.org/10.1016/j.asoc.2015.10.011>. Jan.
- Al-Fuqaha, A., Guizani, M., Mohammadi, M., Aledhari, M., & Ayyash, M. (2015). Internet of things: A survey on enabling technologies, protocols, and applications. *IEEE Communication Survey and Tutorials*, 17(4), 2347–2376. <https://doi.org/10.1109/COMST.2015.2444095>.
- Almseidin, M., Alzubi, M., Kovacs, S., & Alkasasbeh, M. (2017). Evaluation of machine learning algorithms for intrusion detection system. In *2017 IEEE 15th International Symposium on Intelligent Systems and Informatics (SISY)* (pp. 000277–000282). <https://doi.org/10.1109/SISY.2017.8080566>.
- Amato, F., Mazzocca, N., Moscato, F., & Vivenzio, E. (2017). Multilayer perceptron: An intelligent model for classification and intrusion detection. In *2017 31st International Conference on Advanced Information Networking and Applications Workshops (WAINA)* (pp. 686–691). <https://doi.org/10.1109/WAINA.2017.134>.
- Ambusaidi, M. A., He, X., Nanda, P., & Tan, Z. (2016). Building an intrusion detection system using a filter-based feature selection algorithm. *IEEE Transaction of Computer*, 65(10), 2986–2998. <https://doi.org/10.1109/TC.2016.2519914>. Oct.
- Aminanto, M. E., Choi, R., Tanuwidjaja, H. C., Yoo, P. D., & Kim, K. (2018). Deep abstraction and weighted feature selection for Wi-Fi impersonation detection. *IEEE Transaction Information Forensic Security*, 13(3), 621–636. <https://doi.org/10.1109/TIFS.2017.2762828>. Mar.
- Amiri, F., Rezaei Yousefi, M., Lucas, C., Shakeri, A., & Yazdani, N. (2011). Mutual information-based feature selection for intrusion detection systems. *Journal of*

- Network and Computer Applications, 34(4), 1184–1199. <https://doi.org/10.1016/j.jnca.2011.01.002>. Jul.
- Aslahi-Shahri, B. M., et al. (2016). A hybrid method consisting of GA and SVM for intrusion detection system. *Neural Computer & Application*, 27(6), 1669–1676. <https://doi.org/10.1007/s00521-015-1964-2>. Aug.
- Chaabouni, N., Mosbah, M., Zemmar, A., Sauvignac, C., & Faruki, P. (2019). Network intrusion detection for IoT security based on learning techniques. *IEEE Communication Survey and Tutorials*, 21(3), 2671–2701. <https://doi.org/10.1109/COMST.2019.2896380>
- Chohra, A., Shirani, P., Karbab, E. B., & Debbabi, M. (2022). Chameleon: Optimized feature selection using particle swarm optimization and ensemble methods for network anomaly detection. *Computers & Security*, 117, Article 102684. <https://doi.org/10.1016/j.cose.2022.102684>. Jun.
- D'Angelo, G., & Palmieri, F. (2021). Network traffic classification using deep convolutional recurrent autoencoder neural networks for spatial-temporal features extraction. *Journal of Network and Computer Applications*, 173, Article 102890. <https://doi.org/10.1016/j.jnca.2020.102890>
- Dao, T.-N., & Lee, H. (2022). Stacked autoencoder-based probabilistic feature extraction for on-device network intrusion detection. *IEEE Internet of Things Journal*, 9(16), 14438–14451. <https://doi.org/10.1109/JIOT.2021.3078292>
- Dina, A. S., & Manivannan, D. (2021). Intrusion detection based on machine learning techniques in computer networks. *Internet of Things*, 16, Article 100462. <https://doi.org/10.1016/j.iot.2021.100462>. Dec.
- Disha, R. A., & Waheed, S. (2022). Performance analysis of machine learning models for intrusion detection system using gini impurity-based weighted random forest (GIWRF) feature selection technique. *Cybersecurity*, 5(1), 1. <https://doi.org/10.1186/s42400-021-00103-8>. Dec.
- Fatani, A., Dahou, A., Al-qaness, M. A. A., Lu, S., & Abd Elaziz, M. A. (2021). Advanced feature extraction and selection approach using deep learning and aquila optimizer for IoT intrusion detection system. *Sensors*, 22(1), 140. <https://doi.org/10.3390/s22010140>. Dec.
- Gad, A. R., Nashat, A. A., & Barkat, T. M. (2021). Intrusion detection system using machine learning for vehicular ad hoc networks based on ToN-IoT dataset. *IEEE Access : Practical Innovations, Open Solutions*, 9, 142206–142217. <https://doi.org/10.1109/ACCESS.2021.3120626>
- Gavel, S., Raghuvanshi, A. S., & Tiwari, S. (2022). An optimized maximum correlation based feature reduction scheme for intrusion detection in data networks. *Wireless Network*, 28(6), 2609–2624. <https://doi.org/10.1007/s11276-022-02988-w>. Aug.
- Gharehchopogh, F. S., Abdollahzadeh, B., Barshandeh, S., & Arasteh, B. (2023). A multi-objective mutation-based dynamic Harris Hawks optimization for botnet detection in IoT. *Internet of Things*, 24, Article 100952. <https://doi.org/10.1016/j.iot.2023.100952>. Dec.
- Guo, G. (2022). An intrusion detection system for the internet of things using machine learning models. In *2022 3rd International Conference on Big Data, Artificial Intelligence and Internet of Things Engineering (ICBAIE)* (pp. 332–335). Xi'an, China: IEEE. <https://doi.org/10.1109/ICBAIE56435.2022.9985800>. Jul.
- Halim, Z., et al. (2021). An effective genetic algorithm-based feature selection method for intrusion detection systems. *Computers & Security*, 110, Article 102448. <https://doi.org/10.1016/j.cose.2021.102448>. Nov.
- Hall, M. A. "Correlation-based feature selection for machine learning," p. 198.
- Hosseini, F., Gharehchopogh, F. S., & Masdari, M. (2022). A botnet detection in IoT using a hybrid multi-objective optimization algorithm. *New Generation Computing*, 40(3), 809–843. <https://doi.org/10.1007/s00354-022-00188-w>. Sep.
- Hosseini, F., Gharehchopogh, F. S., & Masdari, M. (2023). MOAEOSCA: An enhanced multi-objective hybrid artificial ecosystem-based optimization with sine cosine algorithm for feature selection in botnet detection in IoT. *Multimedia Tools and Applications*, 82(9), 13369–13399. <https://doi.org/10.1007/s11042-022-13836-6>. Apr.
- Ingre, B., Yadav, A., & Soni, A. K. (2018). Decision tree based intrusion detection system for NSL-KDD dataset. In S. C. Satapathy, & A. Joshi (Eds.), *Information and Communication Technology for Intelligent Systems (ICTIS 2017) - Volume 2* (pp. 207–218). Cham: Springer International Publishing.
- Ismail, S., & Reza, H. (2022). Evaluation of naive Bayesian algorithms for cyber-attacks detection in wireless sensor networks. In *2022 IEEE WORLD AI IOT CONGRESS (AIOT)* (pp. 283–289). <https://doi.org/10.1109/AIOT54504.2022.9817298>
- Kasong, S. M., & Sun, Y. (2020). Performance analysis of intrusion detection systems using a feature selection method on the UNSW-NB15 dataset. *Journal of Big Data*, 7(1), 105. <https://doi.org/10.1186/s40537-020-00379-6>. Dec.
- Khammassi, C., & Krichen, S. (2017). A GA-LR wrapper approach for feature selection in network intrusion detection. *Computers & Security*, 70, 255–277. <https://doi.org/10.1016/j.cose.2017.06.005>. Sep.
- Khan, F. A., Gumaei, A., Derhab, A., & Hussain, A. (2019). A novel two-stage deep learning model for efficient network intrusion detection. *IEEE Access : Practical Innovations, Open Solutions*, 7, 30373–30385. <https://doi.org/10.1109/ACCESS.2019.2899721>
- Koroniotis, N., Moustafa, N., Sitnikova, E., & Turnbull, B. (2019). Towards the development of realistic botnet dataset in the Internet of Things for network forensic analytics: Bot-IoT dataset. *Future Generation Computer Systems*, 100, 779–796. <https://doi.org/10.1016/j.future.2019.05.041>. Nov.
- Kotsiantis, S. B., Kanellopoulos, D., & Pintelas, P. E. "Data preprocessing for supervised learning," vol. 1, no. 12, p. 6, 2007.
- Kuang, F., Xu, W., & Zhang, S. (2014). A novel hybrid KPCA and SVM with GA model for intrusion detection. *Applied Soft Computing*, 18, 178–184. <https://doi.org/10.1016/j.asoc.2014.01.028>. May.
- Leevy, J. L., Hancock, J., Khoshgoftaar, T. M., & Peterson, J. M. (2022). IoT information theft prediction using ensemble feature selection. *Journal of Big Data*, 9(1), 6. <https://doi.org/10.1186/s40537-021-00558-z>. Dec.
- Liu, G., Yi, Z., & Yang, S. (2007). A hierarchical intrusion detection model based on the PCA neural networks. *Neurocomputing*, 70(7–9), 1561–1568. <https://doi.org/10.1016/j.neucom.2006.10.146>. Mar.
- Liu, J., Yang, D., Lian, M., & Li, M. (2021). Research on intrusion detection based on particle swarm optimization in IoT. *IEEE Access : Practical Innovations, Open Solutions*, 9, 38254–38268. <https://doi.org/10.1109/ACCESS.2021.3063671>
- Mishra, P., Varadharajan, V., Tupakula, U., & Pilli, E. S. (2019). A detailed investigation and analysis of using machine learning techniques for intrusion detection. *IEEE Communication Survey and Tutorials*, 21(1), 686–728. <https://doi.org/10.1109/COMST.2018.2847722>
- Moustafa, N. (2021). A new distributed architecture for evaluating AI-based security systems at the edge: Network TON IoT datasets. *Sustainable Cities and Society*, 72, Article 102994. <https://doi.org/10.1016/j.scs.2021.102994>. Sep.
- Moustafa, N., & Slay, J. (2015). A hybrid feature selection for network intrusion detection systems: Central points. In *Proceedings of the 16th Australian Information Warfare Conference* (pp. 5–13) (p. 2015). <https://doi.org/10.4225/75/57A84D4BFEFBB>. held on the 30 November–2 December.
- Moustafa, N., & Slay, J. (2015). UNSW-NB15: a comprehensive data set for network intrusion detection systems (UNSW-NB15 network data set. In *2015 Military Communications and Information Systems Conference (MilCIS)* (pp. 1–6). Canberra, Australia: IEEE. <https://doi.org/10.1109/MilCIS.2015.7348942>. Nov.
- Moustafa, N., & Slay, J. (2016). The evaluation of network anomaly detection systems: statistical analysis of the UNSW-NB15 data set and the comparison with the KDD99 data set. *Information Security Journal: A Global Perspective*, 25(1–3), 18–31. <https://doi.org/10.1080/19393555.2015.1125974>. Apr.
- Moustafa, N., Turnbull, B., & Choo, K.-K. R. (2019). An ensemble intrusion detection technique based on proposed statistical flow features for protecting network traffic of internet of things. *IEEE Internet of Things Journal*, 6(3), 4815–4830. <https://doi.org/10.1109/JIOT.2018.2871719>. Jun.
- Mukherjee, S., & Sharma, N. (2012). Intrusion detection using naive bayes classifier with feature reduction. *Procedia Technology*, 4, 119–128. <https://doi.org/10.1016/j.protcy.2012.05.017>
- Naseri, T. S., & Gharehchopogh, F. S. (2022). A feature selection based on the farmland fertility algorithm for improved intrusion detection systems. *Journal of Network System Management*, 30(3), 40. <https://doi.org/10.1007/s10922-022-09653-9>. Jul.
- Negandhi, P., Trivedi, Y., & Mangrulkar, R. (2019). Intrusion detection system using random forest on the NSL-KDD dataset. In N. R. Shetty, L. M. Patnaik, H. C. Nagaraj, P. N. Hamsavath, & N. Nalini (Eds.), *Emerging Research in Computing, Information, Communication and Applications* (pp. 519–531). Singapore: Springer Singapore.
- Ngo, V.-D., Vuong, T.-C., Van Luong, T., & Tran, H. "Machine learning-based intrusion detection: feature selection versus feature extraction." arXiv, Jul. 04, 2023. Accessed: Jul. 10, 2023. [Online]. Available: <http://arxiv.org/abs/2307.01570>.
- Padmashree, A., & Krishnamoorthi, M. (2022). Decision tree with pearson correlation-based recursive feature elimination model for attack detection in IoT environment. *Information Technology and Control*, 51(4), 771–785. <https://doi.org/10.5755/joi.ite.51.4.31818>
- Pajouh, H. H., Dastghaibafard, G., & Hashemi, S. (2017). Two-tier network anomaly detection model: a machine learning approach. *Journal of Intelligent Information Systems*, 48(1), 61–74. <https://doi.org/10.1007/s10844-015-0388-x>. Feb.
- Pajouh, H. H., Javidan, R., Khayami, R., Dehghantanha, A., & Choo, K.-K. R. (2019). A two-layer dimension reduction and two-tier classification model for anomaly-based intrusion detection in IoT backbone networks. *IEEE Transaction Emergency Topics Computer*, 7(2), 314–323. <https://doi.org/10.1109/ETC.2016.2633228>. Apr.
- Pavaiyarkarasi, R., Manimegalai, T., Satheeshkumar, S., Dhivya, K., & Ramkumar, G. (2022). A productive feature selection criterion for Bot-IoT recognition based on random forest algorithm. In T. G. S (Ed.), *Proceedings - 2022 IEEE 11th International Conference on Communication Systems and Network Technologies, CSNT 2022* (pp. 539–545). Institute of Electrical and Electronics Engineers Inc.. <https://doi.org/10.1109/CSNT54456.2022.9787583>
- Peterson, J. M., Leevy, J. L., & Khoshgoftaar, T. M. (2021). A review and analysis of the Bot-IoT dataset. In *2021 IEEE International Conference on Service-Oriented System Engineering (SOSE)* (pp. 20–27). Oxford, United Kingdom: IEEE. <https://doi.org/10.1109/SOSE52839.2021.00007>. Aug.
- Popoola, S. I., Adebisi, B., Hammoudeh, M., Gui, G., & Gacanin, H. (2021). Hybrid deep learning for botnet attack detection in the internet-of-things networks. *IEEE Internet of Things Journal*, 8(6), 4944–4956. <https://doi.org/10.1109/JIOT.2020.3034156>
- Qi, L., Yang, Y., Zhou, X., Rafique, W., & Ma, J. (2022). Fast anomaly identification based on multiaspect data streams for intelligent intrusion detection toward secure industry 4.0. *IEEE Transaction Industry Information*, 18(9), 6503–6511. <https://doi.org/10.1109/TII.2021.3139363>. Sep.
- Saied, M., Guirguis, S., & Madbouly, M. (2024). Review of artificial intelligence for enhancing intrusion detection in the internet of things. *Engineering Applications of Artificial Intelligence*, 127, Article 107231. <https://doi.org/10.1016/j.engappai.2023.107231>. Jan.
- Samadi Bonab, M., Ghaffari, A., Soleimani Gharehchopogh, F., & Alemi, P. (2020). A wrapper-based feature selection for improving performance of intrusion detection systems. *International Journal of Communication Systems*, 33(12), e4434. <https://doi.org/10.1002/dac.4434>. Aug.
- Shafiq, M., Tian, Z., Bashir, A. K., Du, X., & Guizani, M. (2021). CorrAUC: A malicious Bot-IoT traffic detection method in IoT network using machine-learning techniques. *IEEE Internet of Things Journal*, 8(5), 3242–3254. <https://doi.org/10.1109/JIOT.2020.3002255>. Mar.

- Song, J., Takakura, H., Okabe, Y., Eto, M., Inoue, D., & Nakao, K. (2011). Statistical analysis of honeypot data and building of Kyoto 2006+ dataset for NIDS evaluation. In *Proceedings of the First Workshop on Building Analysis Datasets and Gathering Experience Returns for Security* (pp. 29–36). Salzburg Austria: ACM. <https://doi.org/10.1145/1978672.1978676>. Apr.
- Tama, B. A., Comuzzi, M., & Rhee, K.-H. (2019). TSE-IDS: A two-stage classifier ensemble for intelligent anomaly-based intrusion detection system. *IEEE Access : Practical Innovations, Open Solutions*, 7, 94497–94507. <https://doi.org/10.1109/ACCESS.2019.2928048>
- Tan, Z., Jamdagni, A., He, X., & Nanda, P. (2010). Network intrusion detection based on LDA for payload feature selection. In *2010 IEEE Globecom Workshops* (pp. 1545–1549). Miami, FL, USA: IEEE. <https://doi.org/10.1109/GLOCOMW.2010.5700198>. Dec.
- Tan, Z., Jamdagni, A., He, X., & Nanda, P. (2010). Network intrusion detection based on LDA for payload feature selection. In *2010 IEEE Globecom Workshops* (pp. 1545–1549). <https://doi.org/10.1109/GLOCOMW.2010.5700198>
- Wang, C., Xu, D., Li, Z., & Niyato, D. “Effective intrusion detection in highly imbalanced IoT networks with lightweight S2CGAN-IDS.” arXiv, Jun. 06, 2023. Accessed: Jun. 12, 2023. [Online]. Available: <http://arxiv.org/abs/2306.03707>.
- Xu, X., & Wang, X. (2005). An adaptive network intrusion detection method based on PCA and support vector machines. In X. Li, S. Wang, & Z. Y. Dong (Eds.), *Advanced Data Mining and Applications* (pp. 696–703). Berlin, Heidelberg: Springer Berlin Heidelberg.
- Yan, B., & Han, G. (2018). Effective feature extraction via stacked sparse autoencoder to improve intrusion detection system. *IEEE Access : Practical Innovations, Open Solutions*, 6, 41238–41248. <https://doi.org/10.1109/ACCESS.2018.2858277>
- Zachos, G., Essop, I., Mantas, G., Porfyrakis, K., Ribeiro, J. C., & Rodriguez, J. (2021). Generating IoT edge network datasets based on the TON IoT telemetry dataset. In *2021 IEEE 26th International Workshop on Computer Aided Modeling and Design of Communication Links and Networks (CAMAD)* (pp. 1–6). Porto, Portugal: IEEE. <https://doi.org/10.1109/CAMAD52502.2021.9617799>. Oct.
- Zhou, L., Zhu, Y., Zong, T., & Xiang, Y. (2022). A feature selection-based method for DDoS attack flow classification. *Future Generation Computer Systems*, 132, 67–79. <https://doi.org/10.1016/j.future.2022.02.006>. Jul.
- Zhou, W., Jia, Y., Peng, A., Zhang, Y., & Liu, P. (2019). The effect of IoT new features on security and privacy: New threats, existing solutions, and challenges yet to be solved. *IEEE Internet of Things Journal*, 6(2), 1606–1616. <https://doi.org/10.1109/JIOT.2018.2847733>
- Zhou, X., Hu, Y., Liang, W., Ma, J., & Jin, Q. (2021). Variational LSTM enhanced anomaly detection for industrial big data. *IEEE Transaction Industry Information*, 17(5), 3469–3477. <https://doi.org/10.1109/TII.2020.3022432>. May.